Cours : Circuits programmables FPGA (Field Programmable Gate Array)

Semestre : 1

Unité d'enseignement: UEF 1.1.2

Matière 4 : Circuits programmables FPGA

Crédits : 4

Coefficient : 2

Public cible : Master 1 Télécommunications option : Systèmes des télécommunications.

Objectifs de l'enseignement :

Dans cette matière, les étudiants auront à étudier les différents types de circuits programmables, ainsi que les différentes méthodes de conception en particulier la programmation en utilisant les langages de description matérielle

Connaissances préalables recommandées :

Electronique numérique (combinatoire et séquentielle)

Année universitaire : 2017-2018

Table des matières

Chapitre 1. Les Réseaux Logiques Programmables : PLD……3
I. Introduction……...……...4
II. Architectures classiques des circuits numériques……...……4
III. Classification des circuits numériques……...……..4
1. Les circuits standards……...5
2. Les circuits spécifiques à l'application ASIC……..6
3. Les circuits programmables PLD……7
Chapitre 2. Les technologies des éléments programmables……12
I. Généralité sur les technologies des éléments programmables……13
II. Les technologies à fusibles……13
III. Les technologies à anti fusible et SRAM……13
IV. Les technologies à EPROM/FLASH……15
V. Technologies utilisées par les différents fabricants……15
Chapitre 3. Architecture des FPGA……...16
I. Structure des FPGA……...17
II. Architecture des circuits FPGA……18
III. Les éléments des FPGA……..……...21
Chapitre 4. Programmation VHDL…….. 24
I. Le HDL (Hardware Description Language)……..25
II. But du HDL……...25
1. La simulation……25
2. La synthèse……26
III. Les Principaux Langages HDLs……26
IV. VHDL (VHSIC Hardware Description Language) ……..26
1. Les avantages du langage VHDL……..27
2. Environnement du VHDL…… 27
3. Les unités de compilation VHDL……28
4. La sémantique du langage vhdl ……..……..30
5. Les instructions……..33
6. Modélisation en vhdl……...…….....37
7. Simulations…… ????……38
Chapitre 5. Applications : Implémentation de quelques circuits logiques dans les circuits FPGA……40
I. Introduction……...41
II. Les différentes portes logiques……..42
III. Multiplexeur……..53
IV. Demi-additionneur ……56
V. Additionneur complet ……...58
VI. Bascule D……..63
VII. REGISTRE……65
VIII. Compteur/Décompteur……..68
Série TD n°1 : (Circuits combinatoires)……70
Série TD n°2 (Circuits combinatoires)……...77
Série TD n°3 (Circuits séquentiels)……85
Série TD n°4 (Circuits séquentiels)……89
Bibliographies…...……..……..95

Chapitre 1.
Les Réseaux Logiques Programmables : PLD

I. Introduction

Un circuit logique programmable, ou autrement dit un réseau logique programmable, est un circuit intégré qui fonctionne de manière logique et qui permet ça reprogrammation à volonté même après sa fabrication. Il n'y a pas de programmation en logiciel (contrairement à un microprocesseur). On utilise plutôt le terme « reconfiguration » plutôt que « reprogrammation » (car ses connexions ou ses composants, sont à base des portes logiques et des bascules). Le terme programmation est utilisé fréquemment, pour personnaliser les réseaux logiques reconfigurables et modifiables. Nous allons voir dans ce chapitres les différents circuits numériques et logiques programmables.

II. Architectures classiques des circuits numériques

Le rôle de la technologie micro-électronique est la réalisation et l'intégration des transistors nécessaires à la réalisation des circuits électroniques.

Cette technologie d'intégration se présente sous formes de plusieurs possibilités :

- ✓ Utiliser des circuits standards.
- ✓ Concevoir des circuits spécifiques à l'application (ASIC).
- ✓ Utiliser des circuits programmables (PLD).

Les systèmes électroniques modernes sont de plus en plus complexes, Les contraintes de taille, de puissance dissipée et de performances sont de plus en plus sévères (téléphonie mobile, ordinateurs, traitement du signal, de l'image, etc.…).

Accroissement spectaculaire des densités d'intégrations :

- 1964 Intégration à petite échelle (SSI de 1 à 10 transistors).
- 1968 Intégration à moyenne échelle (MSI de 10 à 500 transistors).
- 1971 Intégration à grande échelle (LSI de 500 à 20 000 transistors).
- 1980 Intégration à très grande échelle (VLSI plus de 20 000 à 1000 000 transistors).
- 1984 Intégration à très grande échelle (ULSI plus de 1000000 transistors). [1]

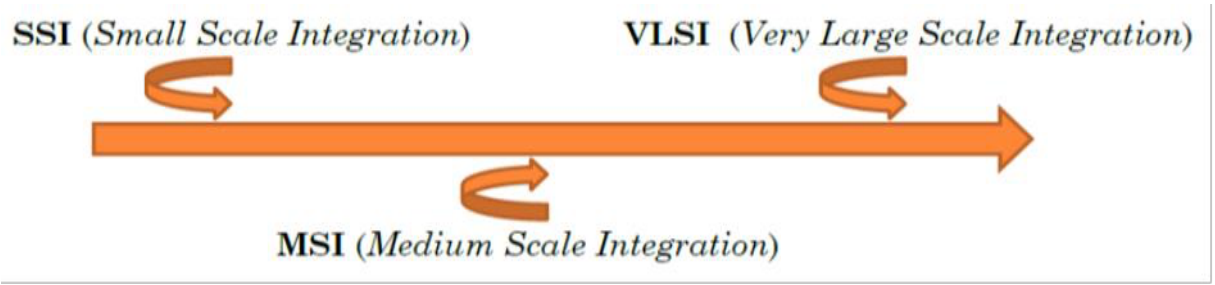


Figure 1 : les différentes échelles d'intégrations

III. Classification des circuits numériques

Les circuits numériques se réparties sur trois grandes familles des circuits bien distincte, comme le montre la figure suivante :

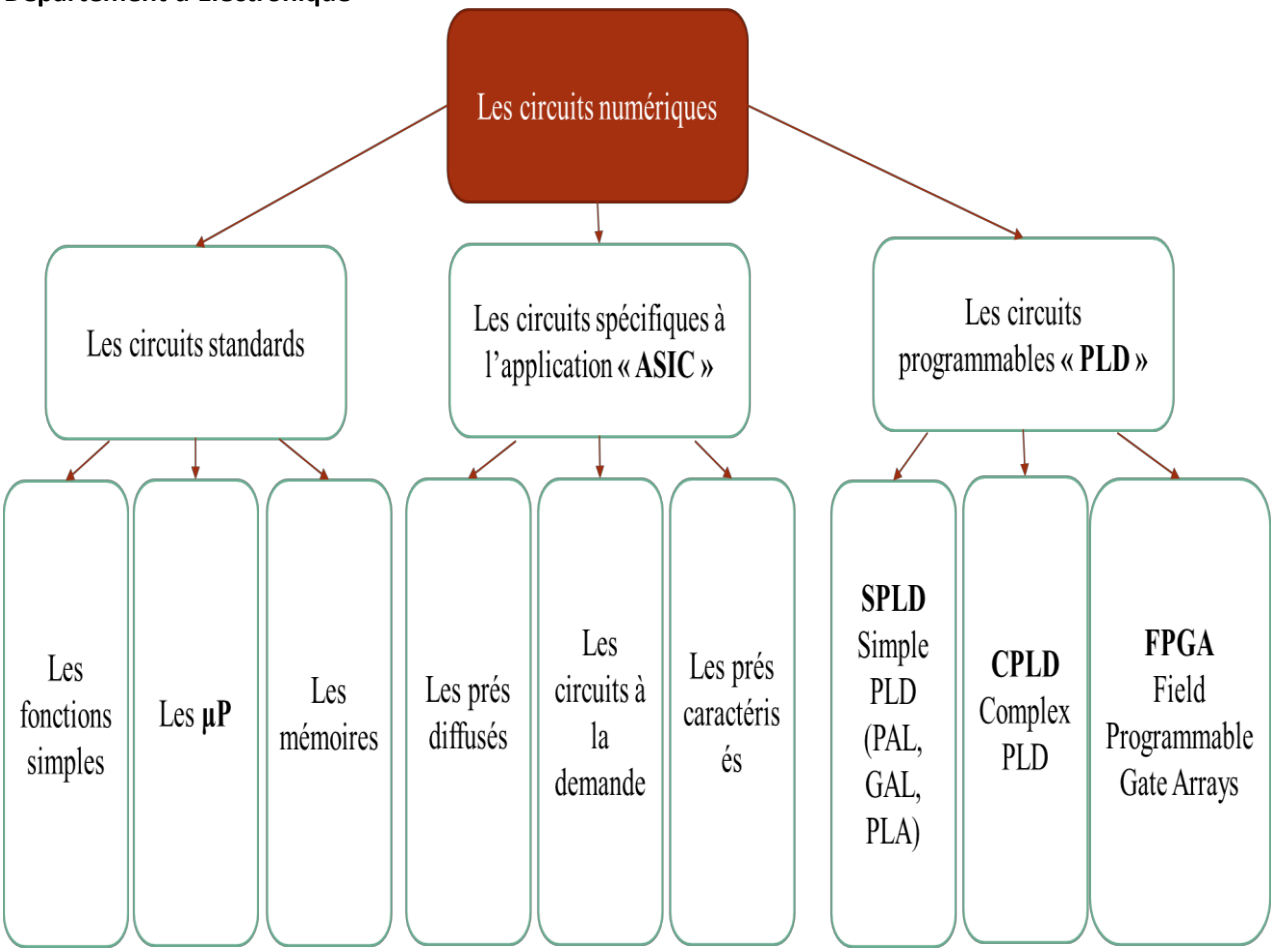


Figure 2 : classification des circuits numériques

1. Les circuits standards

Des fabricants de circuits tels que MATRA, MOTOROLA, SGS THOMSON, … proposent des composants standards ayant des fonctions plus ou moins complexes. L'association de ces composants sur un circuit imprimé permet de réaliser un système numérique.

Ces circuits standards se présente sous forme de trois critères à savoir :

- ✓ **Les fonctions simples.**
  - Certains circuits combinatoires de moyenne complexité MSI (Medium Scale Integration) sont considérés comme des circuits standards de base.
  - On peut trouver aussi les circuits SSI (Small Scale Integration), qui réalisent des fonctions combinatoires ou séquentielles élémentaires.
  - Les fonctions correspondantes réalisées en circuits intégrés se retrouvent comme composants dans les bibliothèques (librairies) de conception des circuits logique programmables.
- ✓ **Les microprocesseurs**.
  - *Processeur à usage général :* Pour minimiser l'impact du coût de conception et de fabrication des circuits intégrés les plus complexes : Créer un circuit de traitement numérique dont l'usage final (l'application) n'est pas connu à la fabrication.

Réaliser un circuit intégré ayant quelques ressources de traitement assez génériques (addition de deux nombres, stockage d'un nombre en mémoire, lecture d'un nombre d'une mémoire...).
Avec de tels circuits l'augmentation de complexité des applications est gérée simplement par l'augmentation de la taille des programmes.

- *DSP (Digital Signal Processor) :*
  Le DSP est un microprocesseur optimisé, pour exécuter des applications de traitement numériques du signal (filtrage, extraction de signaux, etc) le plus rapidement possible.
  Les DSP sont utilisés dans la plupart des applications du traitement numérique du signal en temps réel.
  On les trouve dans les Modems (RTC, ADSL), les télephones mobiles, les appareils multimédia (lecteur MP3), les récepteurs GPS, Etc…

✓ **Les mémoires.**
Une mémoire est un dispositif permettant de stocker puis de restituer une information. On distingue deux classes de mémoires à semi-conducteur :

- *Les mémoires vives :* sont des mémoires volatiles, car on peut perdre l'information en cas de coupure d'alimentation électrique. Elles peuvent être lues et écrites.
- *Les mémoires mortes* : sont des mémoires qui conservent l'information même en absence de l'alimentation. Donc on peut les considérées comme un circuit logique programmable.

2. Les circuits spécifiques à l'application ASIC (Application Specific Integration circuit)

- Les circuits ASIC constituent la troisième génération des circuits intégrés, apparu au début des années 80.
- l'ASIC présente une personnalisation de son fonctionnement, selon l'utilisation:

➠ une réduction du temps de développement

➠ une augmentation de la densité d'intégration et de la vitesse de fonctionnement.

- l'ASIC est un circuit intégré qui permet un câblage direct des applications spécifiques sur le silicium.

✓ **Les prés diffusés (Gate Array):**
Une "mer" de portes est routée. Les éléments logiques existent déjà physiquement sur le circuit, seules les connexions peuvent être définies

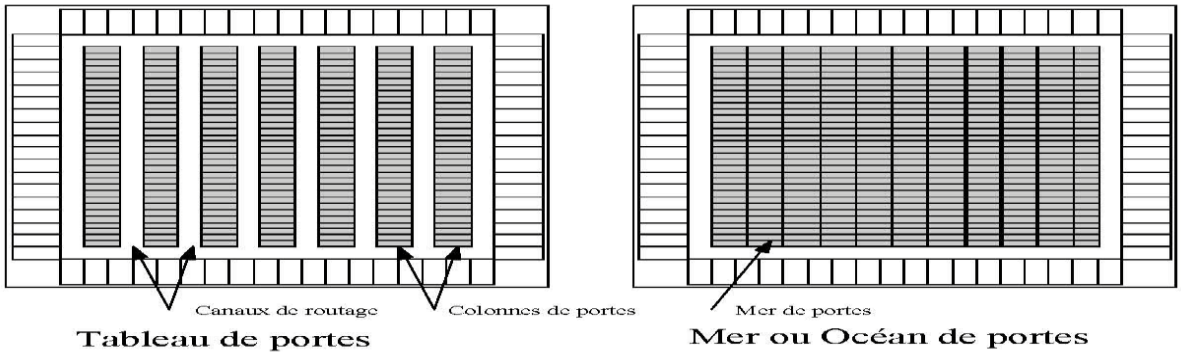


Figure 3 : ASIC (Près diffusés) [2]

✓ **Les circuits à la demande (Full-Custom) :**

Tous les éléments utilisés sont développés par le concepteur ; le fabricant réalise tous les niveaux de masque. Tout est modifiable : transistors (type, caractéristiques), connexions…

L'Intérêt du Full Custom :

- Création de circuits standards, de circuits programmables, de bibliothèques de cellules standards ou d' IPs.
- Utilisation des technologies les plus récentes.
- Contraintes de performances trop fortes pour les composants existants.

✓ **Les prés caractérisés (Standard Cell) :**

- Le circuit est un assemblage de cellule placées/routées.
- Les éléments logiques sont choisis dans une bibliothèque de portes, les connexions sont libres.

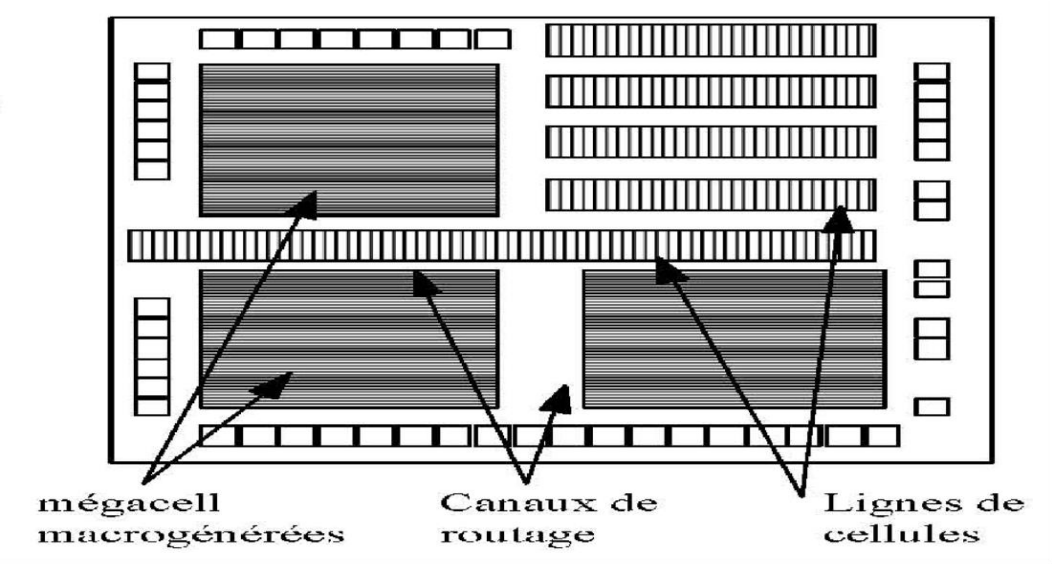


Figure 4 : ASIC (prés caractérisés) [2]

3. Les circuits programmables PLD (Programmble Logic Device)

Nom générique donné à l'ensemble des circuits monolithiques formés de cellules logiques (comportant des fusibles, des antis fusibles ou de la mémoire) qui peuvent être programmés et parfois reprogrammés par l'utilisateur.

Il existe trois principaux circuits programmables :

✓ SPLD (Simple PLD)

Les Simple Programmable Logic Device sont des circuits programmables élémentaires, constitué d'un ensemble de portes « ET » sur lesquelles viennent se connecter les variables d'entrée et leurs compléments. Ainsi qu'un ensemble de portes « OU » sur lesquelles les sorties des opérateurs « ET » sont connectées les variables d'entrée. Comme le montre la figure suivante :

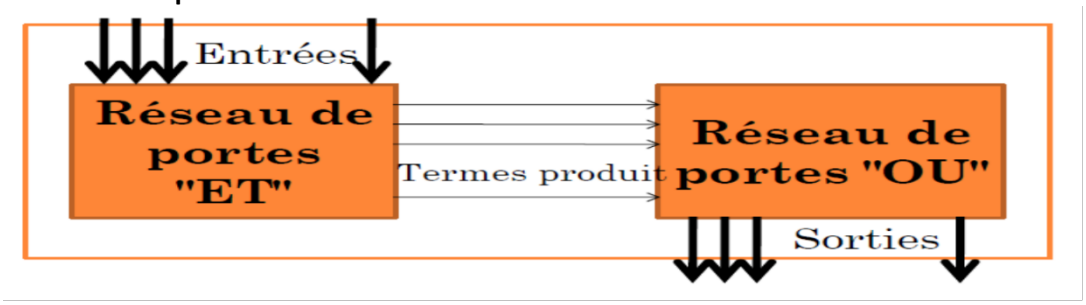


Figure 5 : Architecture globale d'un SPLD

Il existe trois type des SPLD :

- PAL (Programmable Array Logic) : réseaux logiques programmables
  - ✓ Développés au début des années 70 par MMI (ex-AMD).
  - ✓ La programmation se fait par destruction de fusibles.
  - ✓ Aucun fusible n'est grillé à l'achat de la PAL. [2]

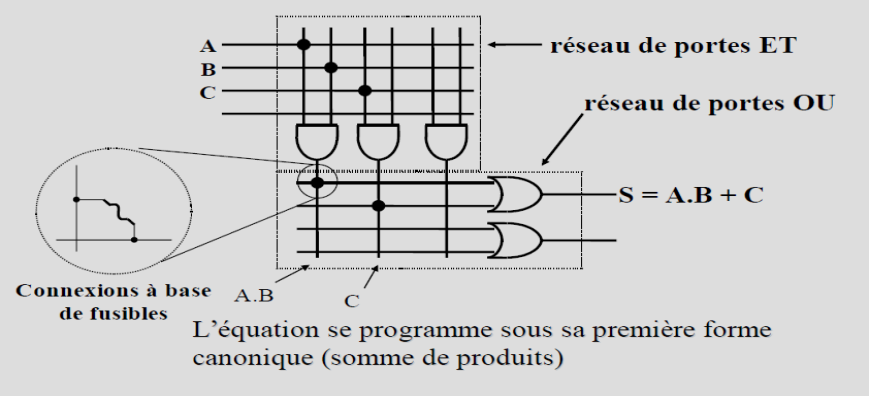


Figure 6 : PAL Architecture [2]

Remarque : Les fonctions ET sont programmables

- GAL (Généric Array Logic)

L'inconvénient majeur des PAL est qu'ils ne sont programmables qu'une seule fois. Ce qui a ramené la firme LATTICE a pensé de remplacer les fusibles irréversibles des PAL par des transistors MOS FET qui peuvent être régénérés. Ceci a donné naissance aux GAL « Réseau Logique Générique ». Ces circuits peuvent donc être reprogrammés à volonté, la figure 7 présente une implémentation d'un circuit GAL.

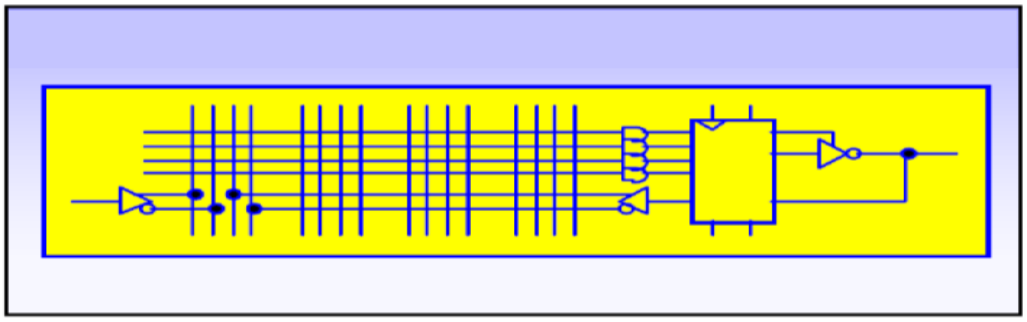


Figure 7 : Implémentation ET-OU-bascule D d'une cellule de base d'un circuit GAL [3]

- **Les PLA (Programmable Logic Arrays)**

Les PLA diffèrent des appareils logiques programmables (PAL et GAL) dans la mesure où les fonctions de porte ET et OU sont programmables. Comme le montre la figure suivante :

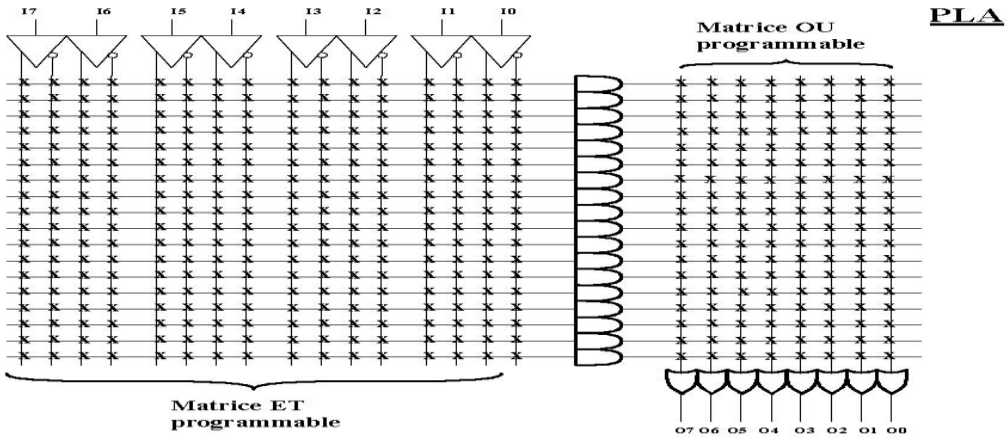


Figure 8 : Circuit PLA programmable des deux portes ET et OU [3]

- Le tableau suivant va récapituler les trois technologies des circuits SPLD

Type	Plan ET	Plan OU	Technologie	Utilisation classique
PAL	Programmable	Fixe	Bipolaire	Décodage, machine à état
GAL	Reprogrammable	Fixe	CMOS	Décodage, machine à état
PLA	Programmable	Programmable	CMOS	Fonctions logiques complexes

Tableau 1 : Comparaison entre les SPLD

- **LIMITATIONS DES SPLD**
  - Impossibilité d'implémenter des fonctions multi-niveaux,
  - Impossibilité de partager des produits entre fonctions,
  - Avec les CPLDs on peut maintenant non seulement programmer la fonctionnalité mais aussi l'interconnexion entre 2 cellules !
  - Contrairement aux FPGAs, il n'y a qu'un seul chemin entre 2 points,
  - Les CPLDs perdent en flexibilité mais gagnent en prédictibilité !

✓ CPLD (Complex PLD)

Les CPLD peuvent être vu comme une intégration de plusieurs PLD simples (SPLD) dans une structure à deux dimensions, ils sont composés de blocs logiques répartis autour d'une matrice d'interconnexion PI (Programmable Interconnect). Comme le montre la figure 9.

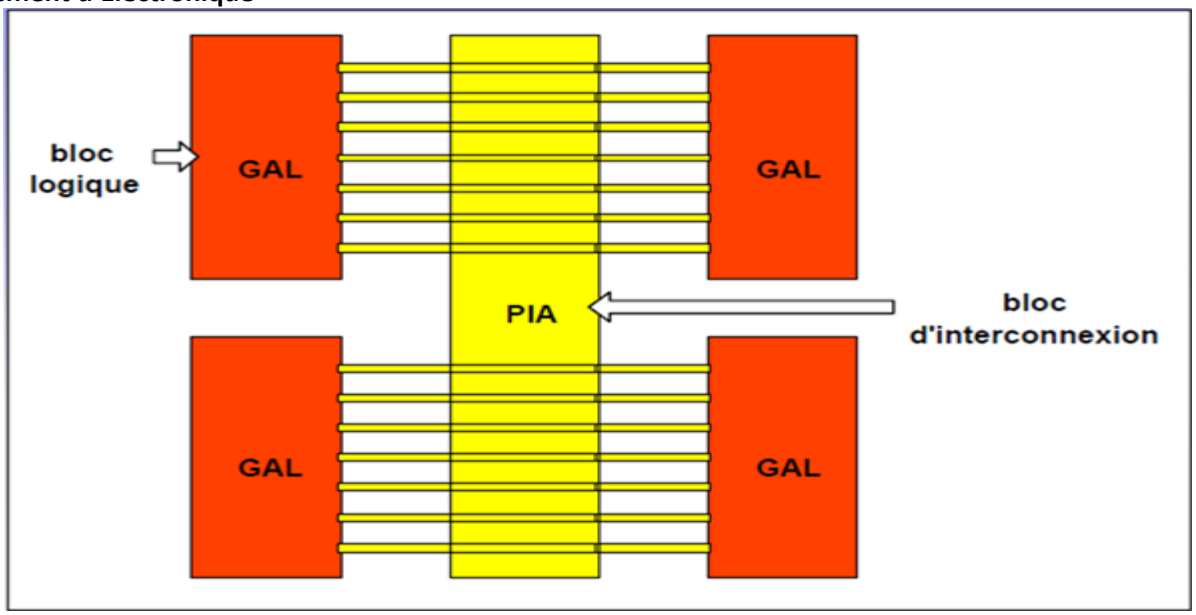


Figure 9 : Circuit CPLD [3]

Ça structure interne (Macro-cellule) est composée d'une zone de portes logiques et une bascule.

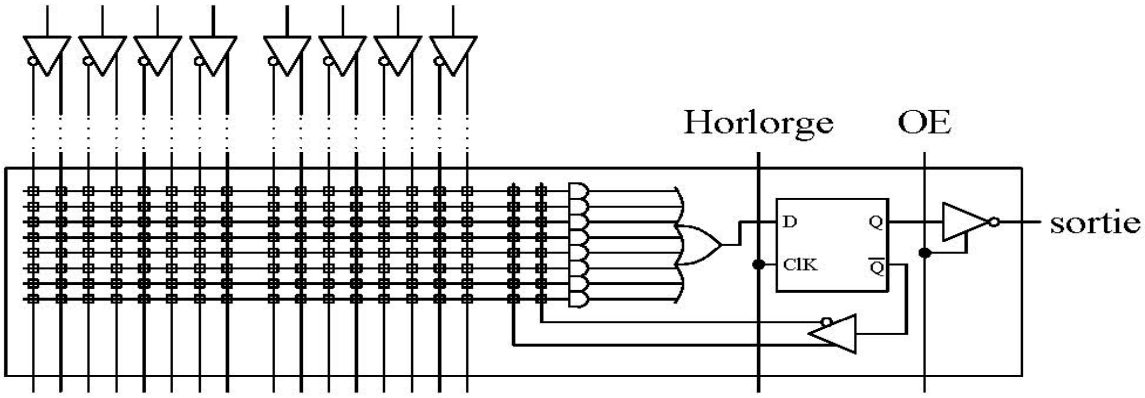


Figure 10 : Macro-cellule Programmable [3]

Dans les années 80, Altera sont les premiers à fournir une solution utilisable de CPLD:

- Utilisé pour le prototypage rapide,
- Un problème restait pourtant au niveau de la matrice d'interconnexion entre les SPLD,
- Limitant la taille des designs à prototyper
- En 1984 Xilinx, lance le premier Field Gate Array (FPGA), le XC2064.
- La principale différence est sa flexibilité sans perte de performances.
- Son inconvénient est le manque de prédictibilité des temps d'interconnexion.

- Ressemblance entre CPLD et FPGA
  - Les cellules logiques sont placées dans une topologie donnée, et reliées par une infrastructure d'interconnexion.
  - Leur fonction est programmable,

- Les chemins entre 2 cellules sont multiples et les temps ne sont connues qu'après le routage !
- Les cellules externes (IO cells) ne sont pas programmable fonctionnellement mais en :Direction,Voltage, Bufferisation.
- Avantage principal : Le temps de conception

- Différence entre CPLD et FPGA
  - FPGA contient jusqu'à 100 000 de minuscules blocs logiques, tandis que CPLD ne contient que quelques blocs de logique allant jusqu'à quelques milliers.
  - En termes d'architecture, les FPGA sont considérés comme des dispositifs à « grain fin » alors que les CPLD sont des « grains grossiers ».
  - Les FPGA sont parfaits pour des applications plus complexes, tandis que les CPLD sont mieux pour les plus simples.
  - Les FPGA sont constitués de minuscules blocs logiques tandis que les CPLD sont constitués de blocs plus gros.
  - FPGA est une puce logique numérique basée sur la RAM, tandis que CPLD est basé sur EEPROM.
  - Normalement, les FPGA sont plus coûteuses alors que les CPLD sont beaucoup moins chers.
  - Les délais sont beaucoup plus prévisibles dans les CPLD que dans les FPGA.

✓ FPGA (Field Programmable Gate Arrays)

Circuit programmable composé d'un réseau de blocs logiques élémentaires (plusieurs milliers de portes), de cellules d'entrée-sortie et de ressources d'interconnexion totalement flexibles
Ce circuit, qui nécessite un outil de placement-routage, est caractérisé par son architecture, sa technologie de programmation et les éléments de base de ses blocs logiques
La puissance de ces circuits est telle qu'ils peuvent être composés de plusieurs milliers voire millions de portes logiques et de bascules. Les dernières générations de FPGA intègrent même de la mémoire vive (RAM). Les deux plus grands constructeurs de FPGA sont XILINX et ALTERA.

Chapitre 2.
Les technologies des éléments programmables

I. Généralité sur les technologies des éléments programmables

On trouve les éléments programmables dans les blocs logiques des PLDs, afin de leur donner une fonctionnalité, mais aussi dans les matrices d'interconnexions entre ces blocs. Un élément programmable peut être considéré comme un interrupteur. Afin de respecter les contraintes imposées à l'ingénieur, les éléments programmables doivent posséder plusieurs qualités :

- Ils doivent occuper une surface la plus petite possible (Ce point s'explique pour des raisons évidentes de coût. Ceci est d'autant plus vrai que l'on désire en disposer d'un grand nombre).
- Ils doivent posséder une résistance de passage faible et une résistance de coupure très élevée.
- Ils doivent apporter un minimum de capacité parasite.

Les deux derniers points s'expliquent quant à eux pour des raisons de performance en terme de fréquence de fonctionnement du PLD. Plus la résistance et la capacité sur le chemin d'un signal sont faibles, plus la fréquence de ce signal peut être élevée (RC effet). [4]

II. Les technologies à fusibles

-Circuits de faible densité.
-La programmation permet de supprimer la connexion.
-Toutes les connexions sont établies à la fabrication.
- Son principe est d'appliquer une tension de 12 à 25v (tension de claquage) aux bornes du fusible.
-La programmation dans cette technologie est irréversible.

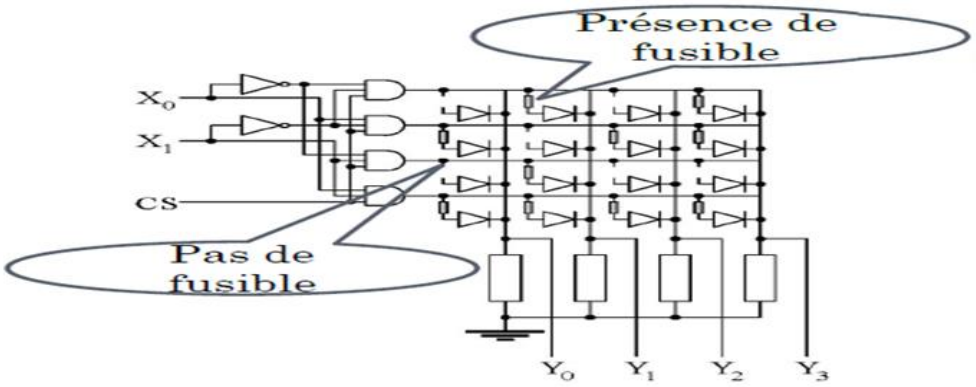


Figure 11 : les Fusibles [2]

Remarque :

Technologies maintenant abandonnées pour des raisons de manque de fiabilité. Le fait de "griller" les fusibles provoque des perturbations qui peuvent affecter le reste du circuit. De plus, cette programmation est irréversible et ne permet donc pas la re-programmabilité.

III. Les technologies à anti fusible et SRAM

Malgré qu'entend plus le terme de la reprogrammation dans les circuit FPGA, il existe deux différentes classifications de FPGA soit à base d'anti fusible soit à celle d'une mémoire, la figure suivante nous montre la classification de ses dernières.

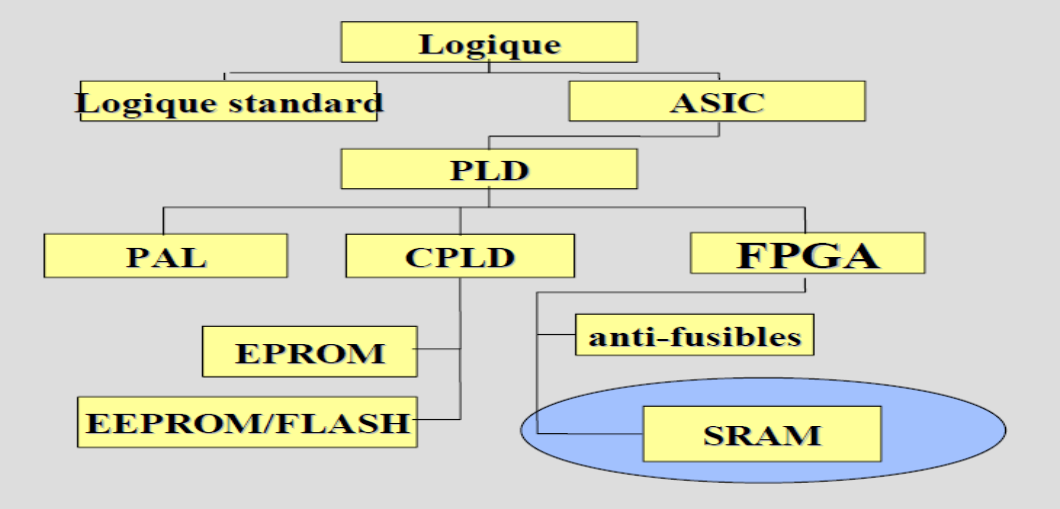


Figure 12 : Classification des technologies [2]

✓ **Les Anti-Fusibles**

Elaborer par la société ACTEL, La technique de l'anti-fusible consiste à isoler deux lignes de connexions par une fine couche d'oxyde.

Si une impulsion de haute tension (une vingtaine de volts) est appliquée à cet anti-fusible, la couche d'oxyde est trouée et les deux lignes sont reliées.

La programmation permet d'établir la connexion mais dans cette technologie est irréversible. Par déduction cette technique est non reprogrammable.

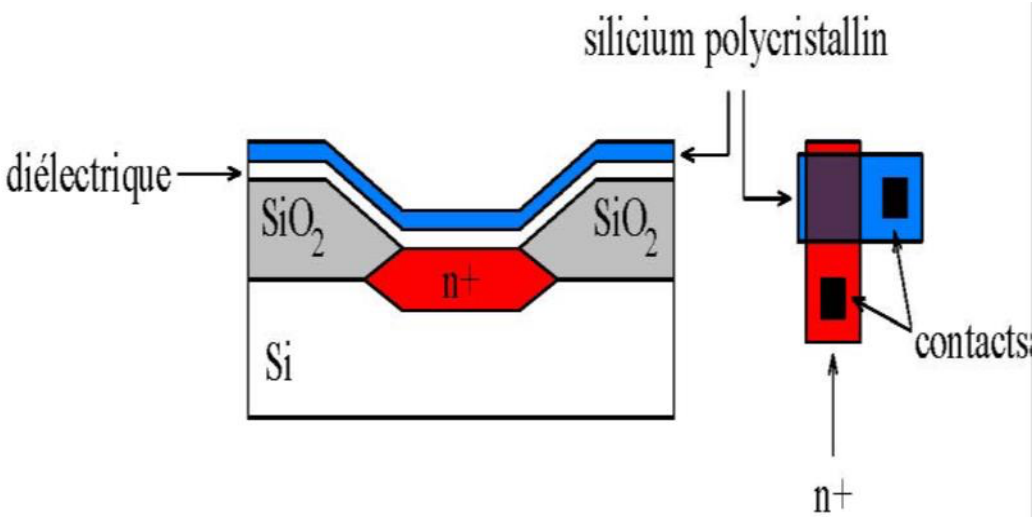


Figure 13 : Technologie de l'anti fusible [3]

✓ **Les SRAM (Static Random Access Memory)**

Les circuits FPGA SRAM deviennent des solutions de remplacement avantageuses pour les systèmes numériques à haute intégration. Les cycles de développement et de prototypage (test et vérification en conditions réelles) sont accélérés et même confondus (figure 14).

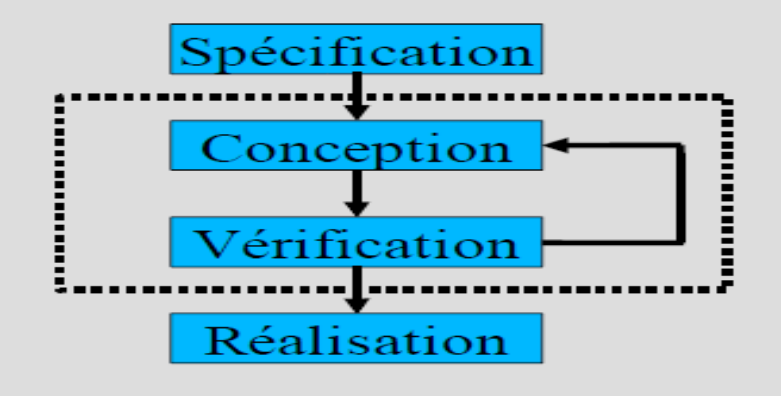


Figure 14 : technologie de FPGA SRAM [3]

IV. Les technologies à EPROM/FLASH

Les technologies des éléments programmables II. Les technologies des éléments programmables

Il existe deux variantes de l'EPROM,

- Erasable Programmable Read Only Memory classique (EPROM)

- Electrically Erasable Programmable Read Only Memory (EEPROM).

La technologie EEPROM

- Les PLD à EPROM se programment électriquement et s'effacent aux UV, Par contre Les PLD à EEPROM se programment quasi instantanément, et gardent la configuration jusqu'à une nouvelle programmation (même en l'absence de tension)
- La technologie EEPROM est Facile et rapide à programmer, la configuration disparaît sans alimentation. [4]

V. Technologies utilisées par les différents fabricants

Le tableau 2, nous donne les différents fabricants dans le monde qui utilise ces technologies des éléments programmables.

FABRICANTS	TECHNOLOGIES UTILISEES
ACTEL	Anti fusible, SRAM
ALTERA	EPROM, EEPROM,SRAM
AMD	EEPROM
ATMEL	SRAM
LATTICE	EPROM,EEPROM
XILINX	SRAM, Anti fusible, EPROM, EEPROM

Tableau 2 : Technologies utilisées par les différents fabricants [4].

Chapitre 3.
Architecture des FPGA

Comme nous venons de le voir dans les chapitres précédents, les éléments programmables sont à la base des caractéristiques des FPGA.

Le choix d'un PLD dépendra dont :

- La densité d'intégration.
- De la rapidité de fonctionnement.
- De la facilité de mise en œuvre (programmation, reprogrammation…).
- De la possibilité de maintien de l'information. [6]

I. Structure des FPGA

Les FPGA à la différence des CPLD sont assimilables à des ASIC programmables par l'utilisateur.

• La puissance de ces circuits est telle qu'ils peuvent être composés de plusieurs milliers voire des millions de portes logiques et de bascules. Les dernières générations de FPGA intègrent même de la mémoire vive (RAM). Les deux plus grands constructeurs de FPGA sont XILINX et ALTERA.

• Ils sont composés de blocs logiques élémentaires (plusieurs milliers de portes) qui peuvent être interconnectés.

• Critère de choix : vitesse de fonctionnement plus élevées pour les CPLD [4].

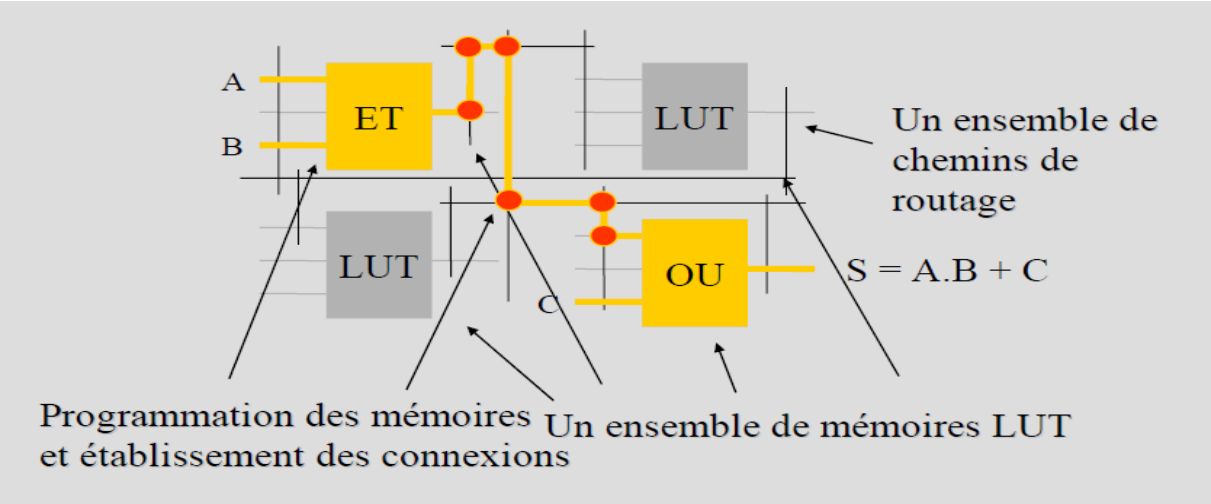


Figure 15 : Structure d'un circuit FPGA

La structure des LUT est caractérisée par une table de vérité ainsi qu'une mémoire pour router les différentes interconnexions entre les entrées et sorties des circuits FPGA.

Dans la figure suivante nous allons voir deux type de circuit FPGA à deux dimension et à mer de porte.

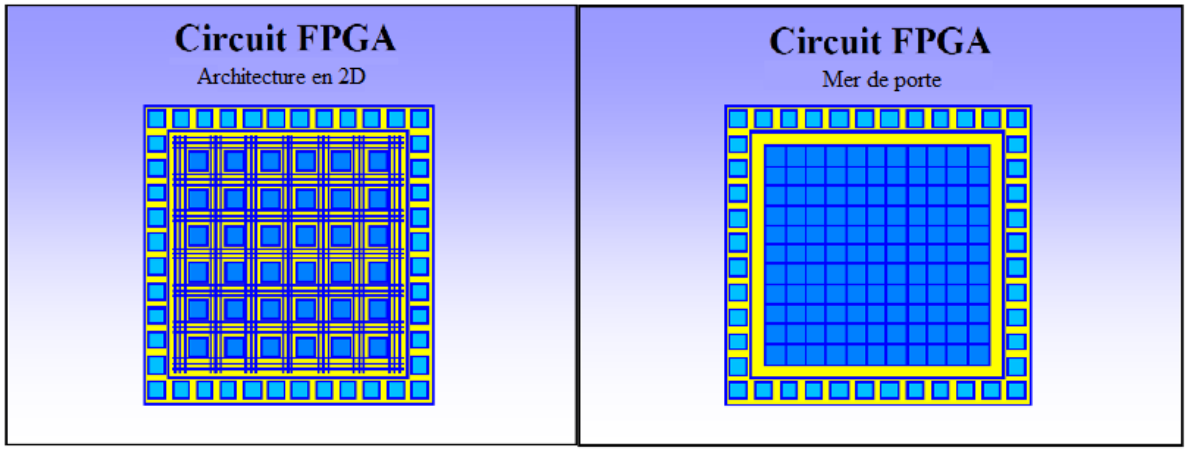


Figure 16 : Architecture en 2 dimension et à Mer de porte [4]

Le FPGA est présenté pour la 1er fois par XILINX avec des structures :

• blocs logiques configurables.

• blocs d'I/O configurables.

• des interconnexions entre bloc configurables. [4]

La figure 17, Nous montre un exemple d'un circuit FPGA créer par la société XILINX.

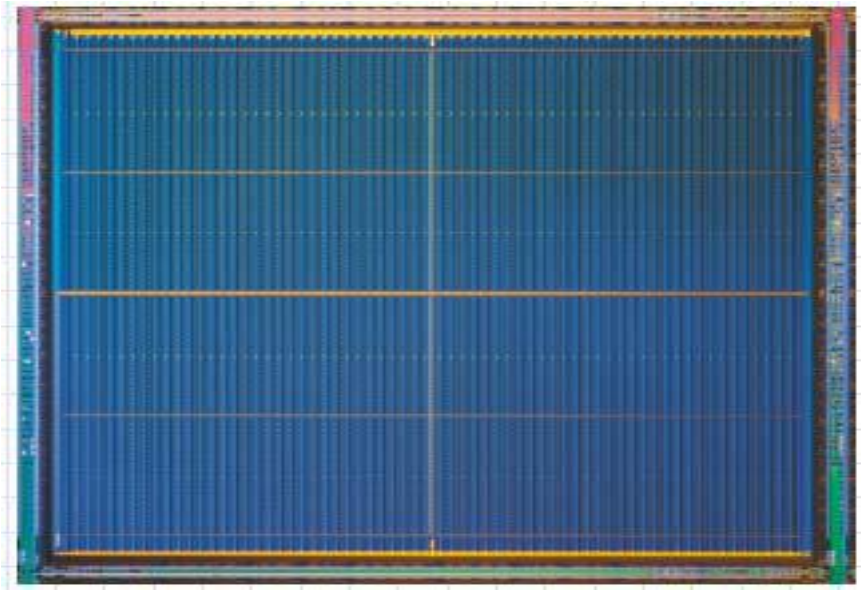


Figure 17 : circuit FPGA Xilinx virtex II [4]

II. Architecture des circuits FPGA

Un FPGA est un réseau (matrice) de blocs combinatoires et séquentiels (CLB).

- Des blocs d'entrée/sortie (IOB) sont associés aux broches du circuit.
- Les CLB et IOB sont interconnectés entre eux par des dispositifs variés.
- Les matrices s'organisent de 8x8 à 128x120. [2]

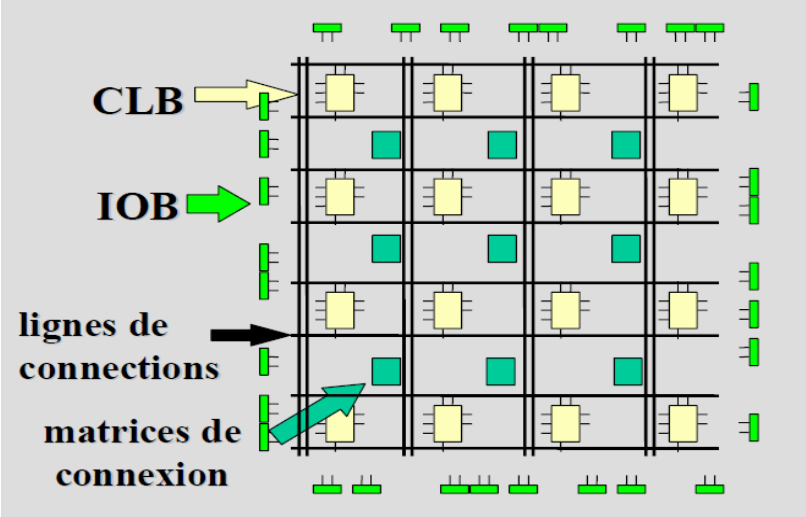


Figure 18 : Architecture générale d'un FPGA [2]

✓ Les Interconnexions

Il existe trois type d'interconnections entre les différents blocs des circuits FPGA

- Interconnexion directe : entre les différents blocs logiques.
- Interconnexions par le biais d'une matrice.
- Interconnexion par les grandes lignes relient tous les CLB dans les extrémités des circuits FPGA.

La figure 19 nous donne un récapitulatif des trois méthodes de connexion.

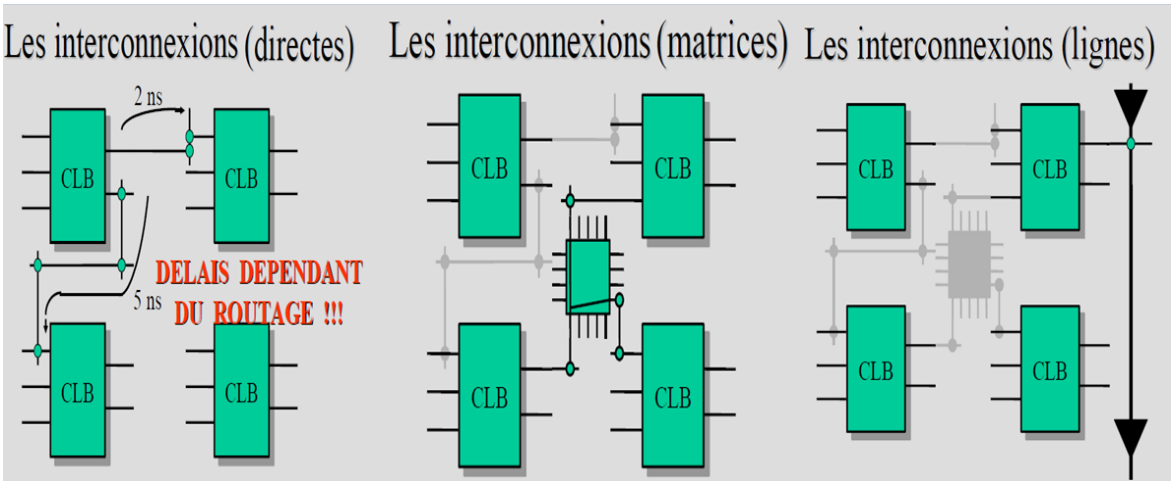


Figure 19 : Les différentes méthodes d'interconnexions entres blocs logiques [2]

✓ **Structure des CLB (CONFIGURABLE LOGIC BLOCS)**

Une table de transcodage combinatoire (LUT) pouvant implanter :

– Deux fonctions indépendantes à 4 variables.
– Une fonction complète à 5 variables.
– Une fonction incomplète à 6 variables.

Elle permet aussi d'implanter, deux cellules séquentielles (bascules D) et des multiplexeurs de configuration. [2]

La figure 20, nous montre un exemple d'un CLB SPARTRAN avec ses différents composants.

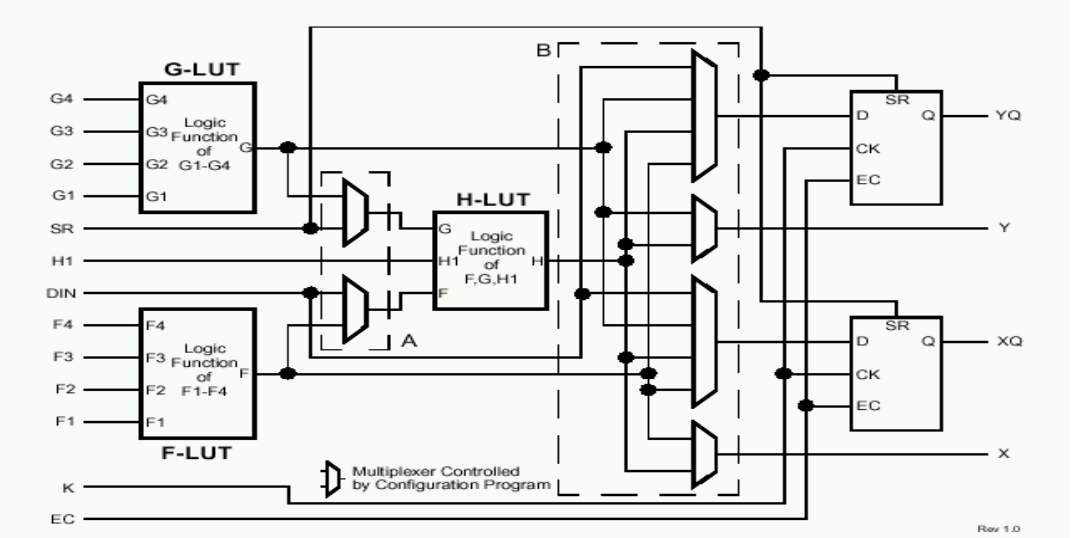


Figure 20 : Structure d'un CLB SPARTAN [2]

✓ **Structure des IOB (INPUT/OUTPUT BLOCKS)**

Les Ports d'entrées/sortie des circuits FPGA sont totalement programmables, et :

• Le seuil d'entrée est soit TTL ou CMOS.

• Le slew-rate est programmable (La vitesse de balayage ou Slew rate représente la vitesse de variation maximale que peut reproduire un amplificateur).

• Le buffer de sortie est programmable en haute impédance

• Les entrées et sorties sont directes ou mémorisées

• L'inverseur est aussi programmable. [2]

La figure 21, nous montre un exemple des IOB SPARTRAN avec ses différents composants.

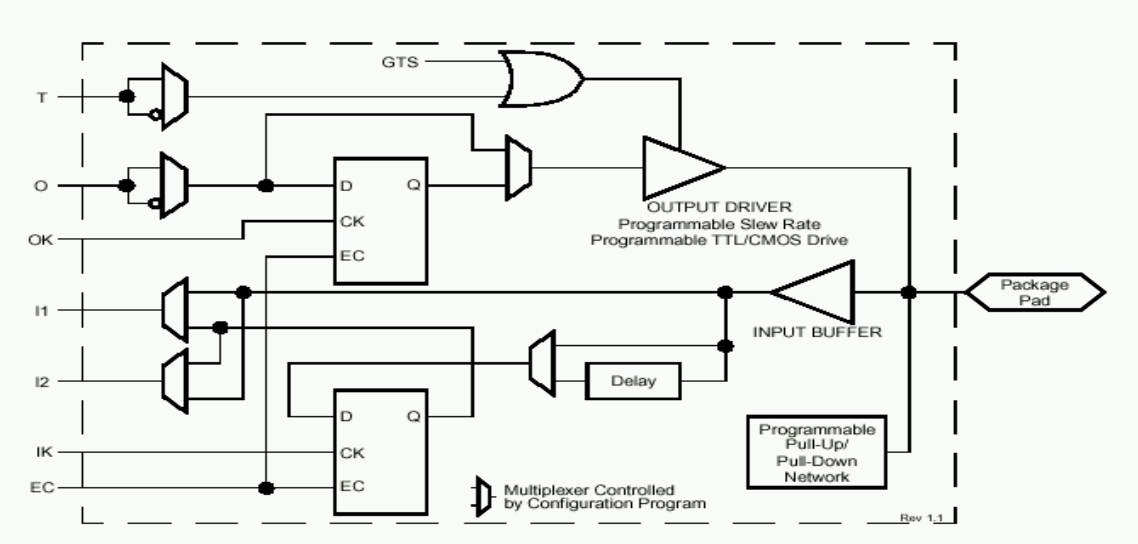


Figure 21 : Structure des IOB SPARTAN [2]

✓ **Canaux de routage**

Les canaux de routage sont présentés soit sous formes de 8 pour les interconnexions en matrices soit sous forme de doublé pour les interconnexions directes soit sous forme de 3 pour les grandes lignes comme le montre la figure suivante

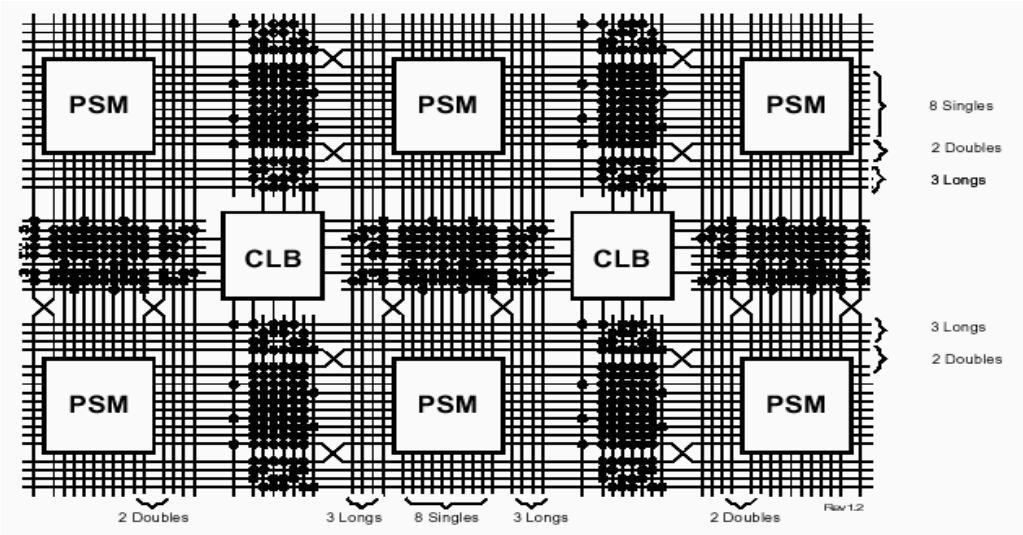


Figure 22 : canaux de routage dans un FPGA [2]

III. Les éléments des FPGA

✓ L'horloge :

Un élément essentiel pour le bon fonctionnement d'un système électronique. Les circuits FPGA sont prévus pour recevoir une ou plusieurs horloges.

Des entrées peuvent être spécialement réservées à ce type de signaux. Ainsi que des ressources de routage spécialement adaptées au transport d'horloges sur de longues distances.

L'horloge est un oscillateur à quartz : Placé dans un angle de la puce, il peut être activé lors de la phase de programmation pour réaliser un oscillateur.

Il utilise deux IOB voisins, pour réaliser l'oscillateur. Cet oscillateur ne peut être réalisé que dans un angle de la puce où se trouve l'amplificateur prévu à cet effet.
Il est évident que si l'oscillateur n'est pas utilisé, les deux IOB sont utilisables au même titre que les autres IOB.

- ✓ Les Critères De Choix :
  - Coût de développement et fabrication : C'est le coût des dépenses engagées pour concevoir le système et réaliser les outils nécessaires à sa fabrication et son test.

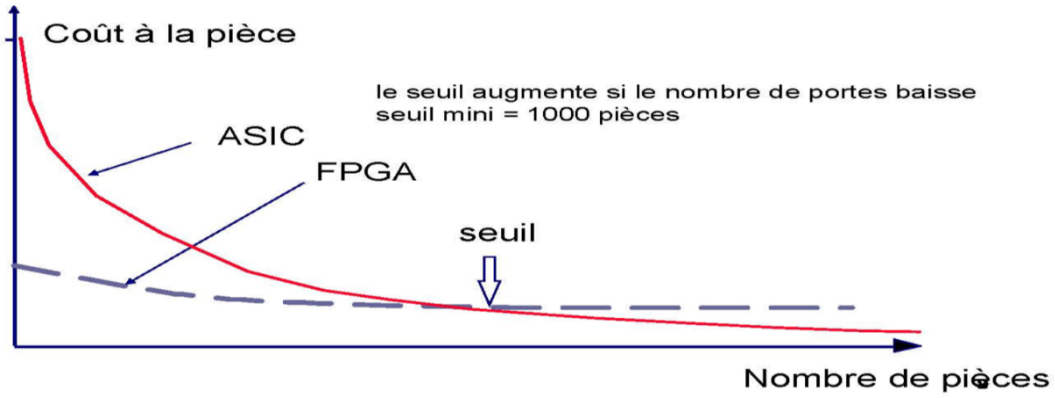


Figure 23 : Coût de développement et fabrication des technologies [2]

  - Temps de développement : Le temps de fabrication à l'aide d'un circuit FPGA se résume à sa programmation ce qui est négligeable.
    SPEC = Standard Performance Evaluation Corporation (licence ou cahier de charge des systèmes informatiques).

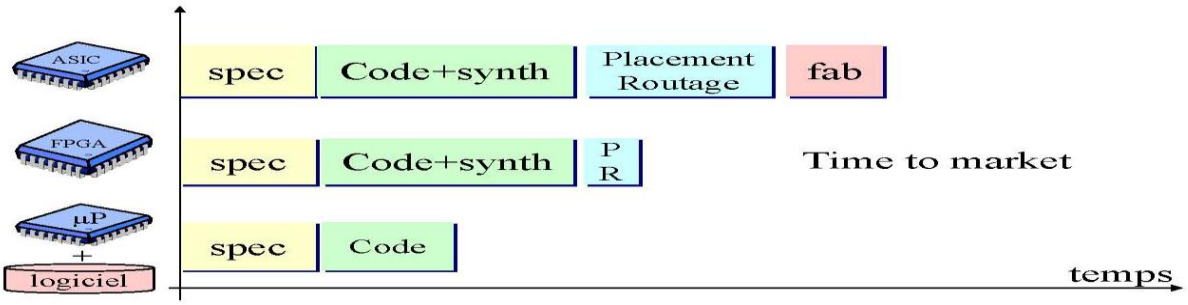


Figure 24 : temps de développement de différentes technologies [2]

  - Taille : Il y a une forte dépendance entre la taille du système et la densité d'intégration. L'augmentation de la densité d'intégration produit des systèmes de taille réduite.

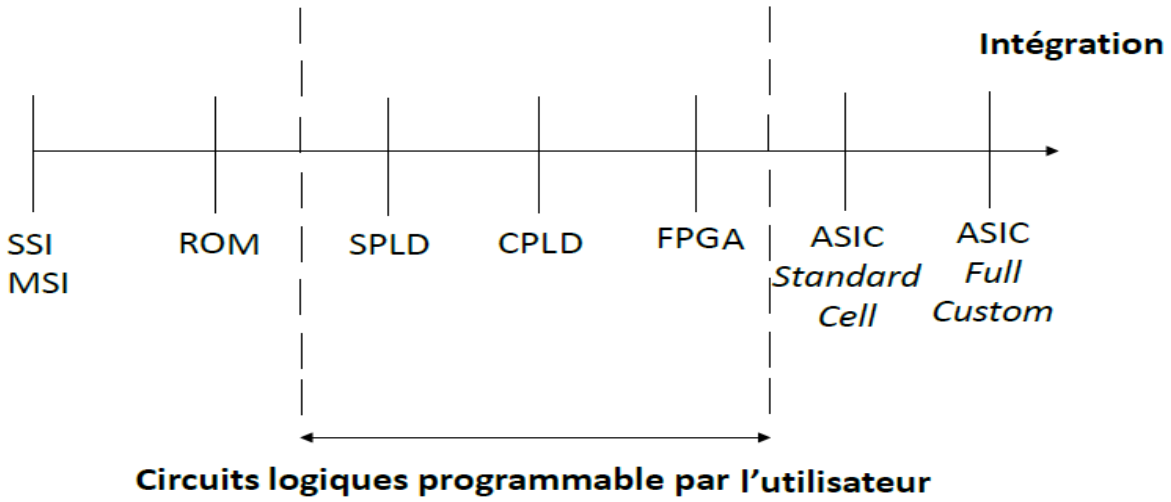


Figure 25 : Taille des différentes technologies

- Souplesse d'utilisation : Favorise les circuits programmables (SPLD, CPLD, FPGA) dont on peut modifier plus facilement des fichiers que des circuits.
- Consommation : Critère particulièrement sensible dans les applications possédant une alimentation autonome. Il conduit à favoriser des solutions ASIC.
- Vitesse de fonctionnement : Les CPLD sont des composants pour la plupart reprogrammables électriquement ou à fusibles, peu chers et très rapides (fréquence de fonctionnement élevée) mais avec une capacité fonctionnelle moindre que les FPGA.
- Capacité mémoire: Les FPGA à SRAM contiennent des mémoires pour stocker leur configuration. La plupart des familles récentes offrent à l'utilisateur la possibilité d'utiliser certaines de ces mémoires en tant que telles. [2]

Chapitre 4 :
Programmation VHDL

I. Le HDL (Hardware Description Language)

Est une instance d'une classe de langage informatique ayant pour but la description formelle d'un système électronique.

Il peut généralement :

- Décrire le fonctionnement du circuit,
- Décrire sa structure,
- Et servir à vérifier sa fonctionnalité par simulation ou preuve formelle.

Ils avaient pour but de modéliser, donc de simuler, mais aussi de concevoir, des outils informatiques permettant de traduire automatiquement une description textuelle en dessins de transistors. [5]

À la différence d'un langage de programmation logiciel, la syntaxe et la sémantique d'un HDL incluent des notations explicites pour exprimer le temps et la concurrence qui sont les attributs principaux du matériel.

Les classes de langages dont la seule caractéristique est de décrire un circuit par une hiérarchie de blocs interconnectés est appelée une netlist. [6]

Un synthétiseur logique permet de transformer un circuit décrit dans un langage de description de matériel en une netlist. [5]

II. But du HDL

Le but des HDL est double :

1. La simulation

L'un des objectifs des HDL est d'aboutir à une représentation exécutable d'un circuit, soit sous forme autonome, soit à l'aide d'un programme externe appelé simulateur. Cette forme exécutable comporte :

- Une description du circuit à simuler,
- Un générateur de stimuli (vecteurs de test),
- Un dispositif implémentant la sémantique du langage et l'écoulement du temps.

Il existe deux types de simulateurs :

1/ à temps discret, généralement pour le numérique,

2/ à temps continu pour l'analogique.

Remarque : Des HDL existent pour ces deux types de simulations.

2. La synthèse

En n'utilisant qu'un sous-ensemble d'un HDL, un programme spécial appelé synthétiseur peut transformer une description de circuit en une netlist de portes logiques ayant le même comportement que le circuit de départ. Le sous-ensemble du langage utilisé à ce propos est alors dit synthétisable.

La sémantique synthétisable ignore typiquement toutes les constructions ayant un rapport avec le temps. [5]

III. Les Principaux Langages HDLs

- **ABEL** "Advanced Boolean Expression Language", un langage propriétaire développé par Data I/O Corporation, maintenant possédé par Lattice;
- **AHDL** ("Altera HDL", langage propriétaire d'Altera pour la programmation de leurs FPGA) langage propriétaire essentiellement structurel proche d'ABEL ;
- **Verilog** qui mélange description structurelle et algorithmique ;
- **VHDL** légèrement plus abstrait que Verilog qui est inspiré de ADA ;
- **SystemC** utilisant le C++ et qui permet de modéliser les interactions logiciel/matériel ;
- **Confluence**, langage déclaratif GPL pouvant générer du Verilog, du VHDL, des modèles exécutables en C et des modèles de vérification formelle ;
- **CUPL** langage propriétaire de Logical Devices;
- **VHDL-AMS** : extension de VHDL destinée aux circuits analogiques ou mixtes. Les langages mixtes, qui sont souvent des extensions des précédents. Ils permettent la modélisation des systèmes à l'aide d'équations différentielles. [6]

IV. VHDL (VHSIC Hardware Description Language)

Le programme VHSIC (Very High Speed Integrated Circuits), impulsé par le département de la défense des Etats-Unis dans les années 1970-1980, a donné naissance à un langage : VHSIC-HDL, connu sous le nom de VHDL . [7]

VHDL → VHSIC Hardware Description Language

Le langage de description VHDL est ensuite devenu une norme IEEE en 1987. Révisée en 1993 pour supprimer quelques ambiguïtés et améliorer la portabilité du langage, cette norme est vite devenue un standard en matière d'outils de description de fonctions logiques.

A ce jour, on utilise le langage VHDL pour :

- Concevoir des ASIC,
- Programmer des composants programmables du type PLD, CPLD et FPGA,
- Concevoir des modèles de simulations numériques ou des bancs de tests. [8]

Le but d'un langage de description matériel tel que le VHDL est de faciliter le développement d'un circuit numérique en fournissant une méthode rigoureuse de description du fonctionnement et de l'architecture du circuit désirée.

L'étape suivante consiste à synthétiser cette description matérielle pour obtenir un composant réalisant les fonctions désirées, à l'aide d'éléments logiques concrets (portes logiques, bascules ou registres). Ceux-ci seront implémentés, selon la technologie utilisée, soit directement en transistors (dans le cas d'un ASIC), ou en se basant sur les éléments programmables des FPGA.

Le VHDL ayant une double fonction (simulation et synthèse), une partie seulement du VHDL est synthétisable, l'autre existant uniquement pour faciliter la simulation (écriture de modèles comportementaux et de test benches).

1. **Les avantages du langage VHDL**

✓ La portabilité.
✓ La conception de haut niveau.
✓ La possibilité de décrire des systèmes très complexes en quelques lignes de code.
✓ De plus, le VHDL :
- Peut-être simulé,
- Peut être traduit en schéma de portes logiques. [9]

2. **Environnement du VHDL**

✓ plateforme : PC/Station
- Éditeurs de texte.
- Compilateur.
- Synthèse.
- Simulateurs.
- Interfaces graphiques (courbes stimulis & résultats).
- bibliothèques : IEEE.
- paquetages IEEE.
- paquetages standards.
- paquetages propriétaires.

✓ Notion de projets
- Un analyseur (transformation du code VHDL en éléments logiques de base).
- Un Place & Route (placement et connection des éléments logiques dans le composant).

✓ Bibliothèque communes (IEEE) et bibliothèques utilisateur (WORK).

Remarque : Beaucoup de logiciels prennent en charge toutes ces fonctionnalités :

Xillinx ISE, Altera Quartus, Lattice ISP Lever, Altium Designer, etc...

Une description VHDL : est un ensemble de fichiers d'extension. Vhd permettant : Édition des fichiers VHDL, Faire référence aux bibliothèques et paquetages nécessaires (IEEE, WORK), Compilation, Simulation, Synthèse et implémentation sur les ASICs ou les FPGAs.

3. Les unités de compilation VHDL

L'analyse d'un modèle VHDL peut s'effectuer sur des parties du code ou "unités de compilation". Il existe 5 types d'unités de compilation :

- Entité (vue externe)
- Architecture (vue interne)
- Configuration (couple: entité-architecture)
- Paquetage (déclarations globales, …)
- Corps du paquetage (sous-programmes, …)

En VHDL, une structure logique est décrite à l'aide d'une entité et d'une architecture de la façon suivante :

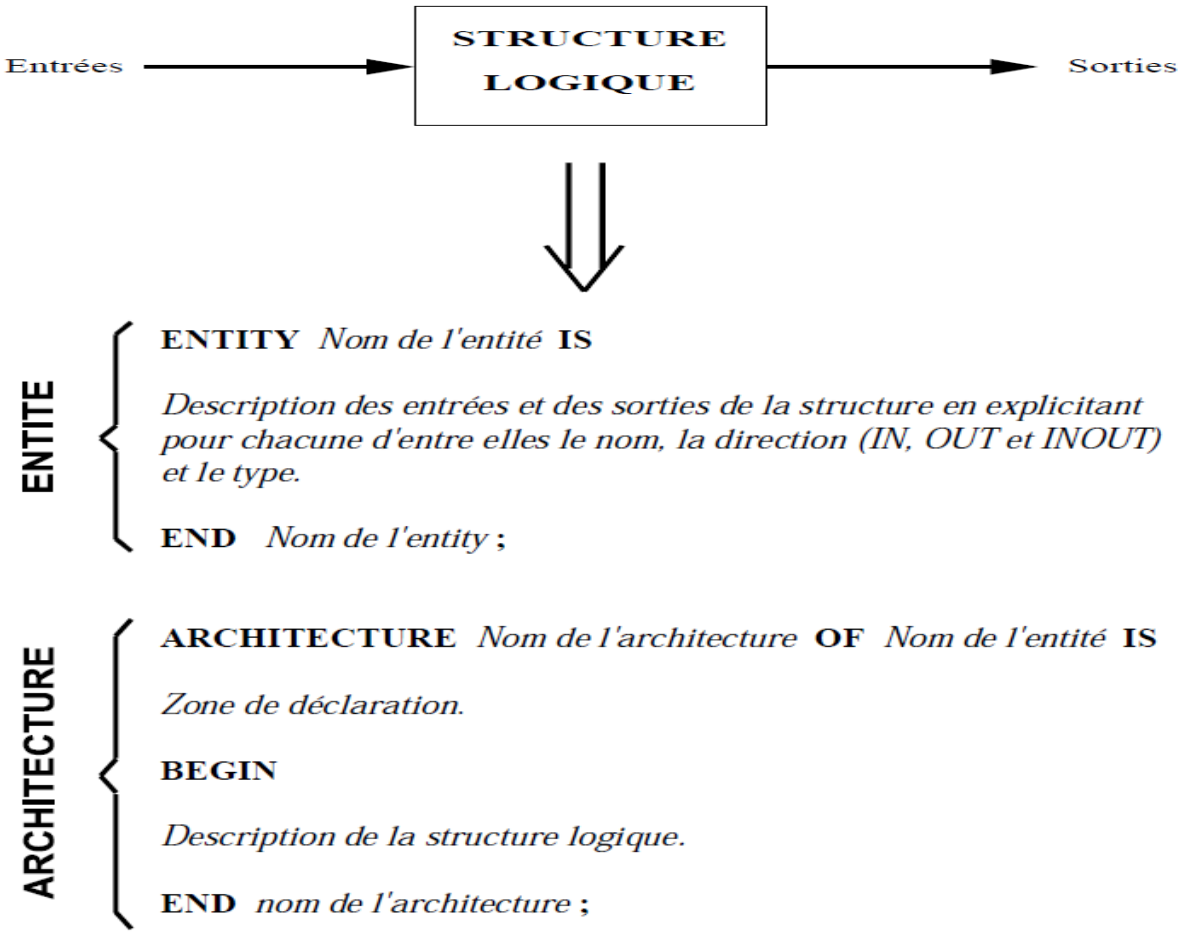


Figure 26 : Structure logique d'un programme VHDL [8]

- ✓ **Syntaxe de l'entité**

entity mon_circuit **is**

port (les signaux d'entrée **: in (type des signaux);**

les signaux de sortie**: out (type des signaux));**

end mon_circuit**;**

- ✓ **Syntaxe de l'architecture**

architecture exemple of mon_circuit **is**

partie déclarative optionnelle : types, constantes, signaux locaux, composants**.**

begin

Corps de l'architecture.

(Suite d'instructions parallèles : affectations de signaux; processus explicites; blocs; instanciation (i.e. importation dans un schéma) de composants.

end exemple **;**

- ✓ **Syntaxe de la configuration :** La déclaration de configuration est utilisée pour lier des entités et des architectures ensemble. Si votre conception a plusieurs entités et architectures multiples pour une logique similaire. Ensuite, vous pouvez utiliser la configuration pour choisir quelle entité vous allez prendre pour votre conception et ensuite lier cette entité avec une architecture de votre choix.

Syntaxe 1:

for label_1 {,label_2} :nom_du_composant **use entity** nom_d_entité{(nom_d_architecture**)}**

{generic map (correspondance_des_paramètres_génériques)**}**

{port map (correspondance_des_ports)**};**

Syntaxe 2:

for label_1 {, label_2 } :nom_du_composant **use configuration** nom_de_la_configuration

{generic map (correspondance_des_paramètres_génériques)**}**

{port map (correspondance_des_ports)**};**

Remarque : Par défaut, si vous avez plusieurs architectures écrites dans un fichier VHDL et que vous ne spécifiez pas la déclaration de configuration, le VHDL choisit la dernière architecture qui est écrite dans votre conception par elle-même.

- ✓ **Syntaxe du paquetage**

package PKG1 **is**

constant C1 **: integer;**

procedure cvt(I **: in std_logic_VECTOR;** o**: out integer);**

function cvt(I **: in std_logic_VECTOR) return POSITIVE);**

component CMP

generic (N : integer);

port (A, B **: in std_logic;** Y **: out std_logic);**

end component;

end PKG1**;**

Remarque : Et il faut faire la référence du paquage au début du fichier x.vhd qui utilise les éléments du paquage.

Exemple : **use work.**PKG1**.all;**

- ✓ **Syntaxe corps de paquetage**

package body PKG1 **is**

constant C1 **: integer** := 16**;**

procedure cvt(I **: in STD_LOGIC_VECTOR;** o**: out integer) is**

begin

o := cvt(I);

end cvt**;**

function cvt(I **: in STD_LOGIC_VECTOR) return integer is**

begin

return (cvt**_std_logic_vector_**int(I) **);**

end cvt**;**

end PKG1 ; [8]

4. La sémantique du langage VHDL

- Règles à respecter en langage VHDL
  - ✓ Ils commencent par une lettre,
  - ✓ Ils contiennent des lettres, des chiffres et des traits bas (_),
  - ✓ Ils ne doivent pas contenir deux traits bas consécutifs (_ _),
  - ✓ Ils ne doivent ni commencer, ni finir par un trait bas,
  - ✓ Un identificateur ne doit pas excéder une ligne de programme,
  - ✓ La différence minuscule majuscule n'est pas significative,
  - ✓ Ils doivent être différents des mots clés.
- ✓ Les commentaires :
  - ✓ 2 traits consécutifs indiquent que le reste de la ligne est du commentaire.
  - ✓ Ils sont liés à la ligne
  - ✓ Ils sont nécessaires à la compréhension du modèle mais totalement ignorés par le compilateur.
- ✓ Le langage VHDL exploite 4 objets différents :

- ✓ **Les signaux :** ils portent les informations des liaisons d'entrée, des liaisons de sortie et des liaisons internes.
- ✓ **Les constantes :** elles reçoivent leurs valeurs au moment de leurs déclarations.
- ✓ **Les variables :** elles ne sont déclarées et utilisées que dans les processus, les fonctions et les procédures. L'assignation initiale est facultative.
- ✓ **Les ports :** ils sont les signaux d'entrées et de sorties de la fonction décrite : ils représentent en quelques sortes les signaux des broches du composant VHDL en cours de description.

✓ **Les types de données :**

✓ Les types énumérés

- Type BIT: ce type peut prendre deux valeurs : '0' ou '1'. Ces deux valeurs ne sont pourtant pas suffisantes pour satisfaire les besoins de la synthèse logique : par exemple, l'état haute impédance n'est pas envisagé !
- Type BIT_VECTOR: ce type permet de manipuler des grandeurs résultant de l'association d'éléments de type BIT (i.e. des vecteurs de bits ou mot).
- Type BOOLEAN: ce type ne peut prendre que deux valeurs différentes : TRUE (vrai) ou FALSE (faux)
- Types STD_LOGIC et STD_LOGIC_VECTOR: ces types sont inclus dans la bibliothèque IEEE 1164,
- STD_LOGIC : 9 valeurs décrivant tous les états d'un signal logique

  'U' : non initialisé**

  'X' : niveau inconnu, forçage fort**

  '0' : niveau 0, forçage fort

  '1' : niveau 1, forçage fort

  'Z' : haute impédance *

  'W' : niveau inconnu, forçage faible**

  'L' : niveau 0, forçage faible

  'H' : niveau 1, forçage faible

  '-' : quelconque (don't care) *

- STD_LOGIC_VECTOR : vecteur de STD_LOGIC
  Pour utiliser ces types il faudra inclure les directives suivantes dans le code source.
  library ieee;
  use ieee.std_logic_1164.all;
- Types CHARACTER et STRING :
  Les variables de type CHARACTER prennent 95 valeurs différentes correspondant à une partie du jeu de caractères ASCII. Le type STRING résulte de l'association de plusieurs éléments de type CHARACTER.
- Type SEVERITY_LEVEL :
  Ce type est employé avec une fonction spécifique aux programmes de tests et permet de caractériser le niveau de gravité programmable lors d'incidents

de simulation. Les variables de ce type peuvent prendre les valeurs suivantes : NOTE, WARNING, ERROR, FAILURE.

- ✓ Les types numériques non énumérés
  - Type INTEGER: valeurs signées codées sur 32 bits.
  - Type REAL: l'étendue des valeurs dépend du compilateur VHDL et la norme VHDL impose des valeurs aux puissances de 10 au moins comprises entre -38 et +38.
  - Sous-types NATURAL: ce type correspond en fait à une restriction du type INTEGER dans on conserve toutes les valeurs supérieures ou égales à 0.
  - Sous-types POSITIVE: De la même façon, le sous-type POSITIVE est une restriction du type INTEGER dont seules les valeurs supérieures à 0 sont conservées.
  - Types UNSIGNED et SIGNED: ces types UNSIGNED et SIGNED utilisent pour base des vecteurs de bits. L'éventail des valeurs que peut couvrir une variable ou un signal de type UNSIGNED ou SIGNED dépendra du nombre de bits qui lui sera attribué lors de la déclaration.
- ✓ Les types utilisateurs
  - Types énumérés définis par l'utilisateur :
    Il est possible de créer de nouveaux types de données.
  - Types non énumérés définis par l'utilisateur :
    Pour la création de type non énuméré, le VHDL dispose du mot clé RANGE faisant office de directive et permettant de spécifier l'intervalle des valeurs valides pour le type créé.
  - Enregistrement : RECORD:
    Lorsque des informations de toute nature doivent être réunies, VHDL permet la création d'un type spécifique qui autorisera la manipulation de toutes ces données sous un même identificateur, appelé enregistrement. On utilise alors le mot clé record.
  - Types vectoriels :
    Les types utilisateurs qui viennent d'être décrits sont des scalaires. On peut aussi créer des types vectoriels dont on fixe le nombre d'éléments dans la déclaration du type ou lors de la déclaration de l'instance. Ces types reposent sur le mot clé array.

- ✓ **Les opérateurs :**
  - ✓ **Opérateurs d'affectation :**
    Le VHDL utilise deux opérateurs d'affectations :
    - Affectation à un signal : pour réaliser une affectation à un signal, on utilise l'opérateur <=
    - Affectation à une variable : l'affectation à une variable exploite l'opérateur **:=**
  - ✓ **Opérateurs relationnels :**
    Ils sont valables avec des signaux et des variables scalaires et vectoriels.
    =, /= (différent), <, <=, >, >=
    La différence entre l'opérateur relationnel <= et l'opérateur d'affectation <= est effectuée par le compilateur en fonction de son contexte d'utilisation.
  - ✓ **Opérateurs logiques :**

Ils s'appliquent à des signaux ou des variables de types booléens, bits et dérivés : BIT, BIT_VECTOR, STD_LOGIC, STD_LOGIC_VECTOR, BOOLEAN.

- AND, OR, NAND, NOR et XOR.
- AND n'est pas prioritaire par rapport à OR (utiliser des parenthèses)
- Dans le cas d'opérandes de types STD_LOGIC_VECTOR ou BIT_VECTOR, l'opération logique s'effectue BIT à BIT (cela implique des opérandes de tailles identiques pour les opérateurs binaires).

✓ **Opérateurs arithmétiques :**

Ils sont définis pour les types entiers et réels.

+ , -, *, / (division entière), ** (exponentiation), mod (modulo), rem (reste de la division), abs (valeur absolue).

✓ **Opérateur de concaténation :**

L'opérateur de concaténation & permet de manipuler des tableaux de tout type et de différentes tailles pour en réaliser l'association sous un autre label de dimension plus i. Exemple : A <= "1100";

B <= "0011";

C <= A & B; C <= "11000011". [8]

5. Les instructions

✓ **Instructions concurrentes :**

- ✓ L'ordre d'écriture des instructions n'a pas d'importance (c'est le parallélisme).
- ✓ Une instruction concurrente décrit une opération qui porte sur des signaux (entrée, interne) pour produire d'autres signaux (interne, sortie).
- ✓ Tous les signaux mis en jeu dans l'architecture sont disponibles au même moment.
- ✓ Le corps d'architecture est décrit dans un ou plusieurs styles.
- ✓ Le style n'est pas imposé par le type de logique (combinatoire ou séquentielle) ou le type de traitement des données (parallèle ou séquentiel).
- ✓ Trois styles peuvent coexister au sein d'une même architecture.
- ✓ Le VHDL est un langage concurrent ➠ le système à modéliser peut-être diviser en plusieurs blocs agissant en parallèle.
- ✓ L'instruction concurrente est exécutée quand un évènement sur certains de ses signaux apparait :
  - Instanciation de composants

    Création d'un composant dont le modèle est défini ailleurs (hiérarchie).

    Exemple de syntaxe

    architecture structurelle **of** adder **is**

    .....

    begin

    C1 : PORTE_ET **port map** (A1,A2,S1);

    C2 : PORTE_OU **port map** (A1,S1,S2);

    C3 : INVERSEUR **port map** (S2,S3);

    ...

    .....

    end structurelle;
  - Process

    Ne contient que des instructions séquentielles

Un process "vit" toujours ➠ cyclique
Doit être controlé par sa liste de sensibilité ou des synchronisations internes (wait).

process (liste de sensibilité) ou **process**
Déclarations Déclarations
begin **begin**
instructions séquentielles **wait** on (liste de signaux)
..... instructions séquentielles
end process; .....
end process;

Remarque : Ne pas mettre une **liste de sensibilité** et un (ou des) **wait**.

- Affectation de signaux

Affectation simple:
Signal **<=** forme d'onde { délai }
Affectation conditionnelle:
Signal <= forme d'onde 1 **when** condition 1 **else** forme d'onde 2
Affectation sélective:
with expression **select**
Signal <= forme d'onde 1 **when** choix 1,
forme d'onde 2 **when** choix 2,

- Appel de procédure

Dans le cas d'un appel concurrent les paramètres sont des signaux ou des constantes sensible à un événement sur les paramètres d'entrée.
{label:} nom_de_la_procedure **(liste des paramètres)**;

- Assertion

Surveille une condition et génère un message si la condition est fausse
On y associe un niveau d'erreur qui peut être pris en compte par le simulateur.
{label:} **assert condition** { report message } { **severity niveau_d'erreur** };

- Bloc

Similaires à un process mais contiennent des instructions concurrentes
Le "block" permet :
– la hiérarchisation des instructions concurrentes et des déclarations locales
– de synchroniser les instructions du bloc avec l'extérieur de ce dernier
label : **block** { condition de garde }
-- déclarations locales
begin
-- instructions concurrentes
end block label;

- Génération

Raccourci d'écriture ➠ élaboration itérative ou conditionnelle d'instructions concurrentes
forme conditionnelle :
➥les instructions sont générées si la condition est vraie
{label :} **if** condition_booléenne **generate**
.....

suite d'instructions concurrentes
end generate {label} ;
forme itérative :
➥génération de structures répétitives indicées
{label :} **for** nom_de_l'index **in** intervalle_discret **generate**
..... -- l'index n'a pas besoin d'être déclaré
instructions concurrentes
end generate {label} ;

- ✓ **Instructions séquentielles :**
  - ✓ Elles sont utilisées dans les process ou les corps de sous-programmes.
  - ✓ Elles permettent des descriptions comportementales :
    - o Affectation de variables
      Exemple :
      resul := 4;
      resul := resul + 1;
    - o Affectation séquentielles de signaux
      Exemple :
      S <= 4 **after** 10 ns;
      S2 <= S **after** 20 ns;
    - o Instruction de synchronisation
    - o Instructions de contrôle
    - o Appels de procédure
      Process
      begin
      ....
      test_valeur (A,B);
      ....
      end Process; -- Appel en séquentiel
    - o Assertion
      assert S_OUT /= "0000"
      report "S_OUT est nul "
      severity warning; -- testé en séquentiel
    - o Null
      case DIVI **is**
      when '0' => ..... ;
      when '1' => **null**;
      end case;
- ✓ **Programmation modulaire**
  - ✓ **Les fonctions**

    Une fonction retourne au programme appelant une valeur unique, elle a donc un type.
    - o **Déclaration :**

    function nom [(liste de paramètres formels)]
    return nom_de_type ;
    - o **Corps de la fonction :**

    function nom [(liste de paramètres formels)]

return nom_de_type is
[déclarations]
begin
instructions séquentielles
end [nom] ;

Le corps d'une fonction ne peut pas contenir d'instruction wait, les variables locales, déclarées dans la fonction, cessent d'exister dès que la fonction se termine.

Utilisation :

nom (liste de paramètres réels)

Lors de son utilisation, le nom d'une fonction peut apparaître partout, dans une expression, où une valeur du type correspondant peut être utilisée.

- ✓ **Les procédures**

Une procédure, comme une fonction, peut recevoir du programme appelant des arguments : constantes, variables ou signaux. Donc une procédure peut renvoyer un nombre quelconque de valeurs au programme appelant.

- o **Déclaration :**

procedure nom [(liste de paramètres formels)];

- o **Corps de la fonction :**

procedure nom [(liste de paramètres formels)] is
[déclarations]
begin
instructions séquentielles
end [nom] ;

Dans la liste des paramètres formels, la nature des arguments doit être précisée :

procedure exemple (signal a, b : in bit ; signal s : out bit) ;

Le corps d'une procédure peut contenir une instruction wait, les variables locales, déclarées dans la procédure, cessent d'exister dès que la procédure se termine.

Utilisation :

nom (liste de paramètres réels)

- ✓ **Les paquetages**

Un paquetage permet de rassembler des déclarations et des sous programmes, utilisés fréquemment dans une application, dans un module qui peut être compilé à part, et rendu visible par l'application au moyen de la clause use.

- ✓ **Les paquetages prédéfinis.**

Un compilateur VHDL est toujours assorti d'une librairie, décrite par des paquetages, qui offre à l'utilisateur des outils variés.

- ✓ **Les paquetages créés par l'utilisateur.**

L'utilisateur peut créer ses propres paquetages. Cette possibilité permet d'assurer la cohérence des déclarations dans une application complexe, évite d'avoir à répéter un grand nombre de fois ces mêmes déclarations et donne la possibilité de créer une librairie de fonctions et procédures adaptée aux besoins des utilisateurs.

✓ **Les librairies**

Une librairie est une collection de modules VHDL qui ont déjà été compilés. Ces modules peuvent être des paquetages, des entités ou des architectures.

Une librairie par défaut, work, est systématiquement associée à l'environnement de travail de l'utilisateur. Ce dernier peut ouvrir ses propres librairies par la clause library:

library nom_de_la_librairie ;

La façon dont on associe un nom de librairie à un, ou des, chemins, dans le système de fichiers de l'ordinateur, dépend de l'outil de développement utilisé. [8]

6. Modélisation en VHDL

✓ Les 3 niveaux de description :

✓ **La description en flot de données.**

- Utiliser des instructions concurrentes d'assignation de signal.
- Description de la manière dont les données circulent de signal en signal, ou d'une entrée vers une sortie
- Trois types d'instructions :

étiquette : … <= …

étiquette : … <= … when … else …

étiquette : with … select … <= … when …

L'ordre d'écriture des instructions d'assignation de signaux est quelconque (le parallélisme).

✓ **La description comportementale.**

- Utiliser des instructions concurrentes d'appel de processus.
- Certains comportements peuvent être décrits de façon algorithmique ; il faut alors les définir comme des processus.
- Dans l'architecture utilisatrice, un processus est considéré comme une instruction concurrente.
- Instruction d'appel de processus :

étiquette: process
déclarations
begin
Instructions séquentielles
end process;

- L'ordre d'écriture des instructions d'appel de processus est quelconque.
- Un processus contient des instructions séquentielles qui ne servent qu'à traduire simplement et efficacement, sous forme d'un algorithme, le comportement d'un sous-ensemble matériel.
- À l'intérieur d'un processus, trois types d'instruction d'assignation de signaux et deux types d'instruction d'itération

… <= … cas d'un signal interne
if … then … else …
case … when …
for … loop …
while … loop …
L'ordre d'écriture des instructions à l'intérieur du processus est déterminant.

- ✓ La description structurelle**.**
  - Instructions concurrentes d'instanciation de composant.
  - Interconnexion de composants (component), à la manière d'un schéma, mais sous forme d'une liste.
  - Dans l'architecture utilisatrice, un composant peut-être considéré comme une boîte noire.
  - Un composant est instancié à l'aide d'une instruction d'appel de composant :

étiquette : nom_composant **port map** (liste des entrées et sorties);

L'ordre d'écriture des instructions d'instanciation de composants est quelconque. Mais la liste doit suivre l'ordre des E/S déclaré auparavant.

  - Par définition, un composant est aussi un système logique (un sous-système) ; à ce titre, il doit aussi être décrit par un couple (entité, architecture) dans lequel sont définis ses entrées-sorties et son comportement.
  - Le corps d'architecture du composant (le comportement) est décrit selon un ou plusieurs Descriptions ; par exemple, un composant peut faire appel à d'autres composants.
  - Un composant peut-être rangé dans une bibliothèque. [9]

7. Simulations

VHDL est à la fois un langage de synthèse et de simulation.

- ✓ Un testbench VHDL est un code VHDL destiné à la vérification, par simulation, du bon fonctionnement d'un système, lui-même décrit en VHDL.
- ✓ Dans l'industrie, les testbenches jouent un rôle très important ; ils sont intégrés dans les spécifications d'un système.
- ✓ Simulation directe (à éviter, sauf si vous avez des simulations déjà tester auparavant) :
  - Charger le composant à simuler dans le simulateur.
  - Faire varier entrée par entrée à chaque pad de simulation.
  - Si on a 1 circuit : horloge + reset + 10 entrées !
    Pour un test exhaustif : 2^{11} +1 possibilités.
    Si 1 possibilité = 5 sec.
    Alors il faut ~ 170 min. = ~ 3heures juste pour fixer les entrées.
- ✓ Simulation avec un fichier de test (testbench) est recommandée :Méthodique et un Gain de temps énorme.
  - En VHDL :
    Les entrées : lecture seulement
    Les sorties : écriture seulement
  - Compilation
  - Chargement dans le simulateur
  - Choix des signaux à observer
  - Lancement de la simulation

- Résultat sur la fenêtre des courbes

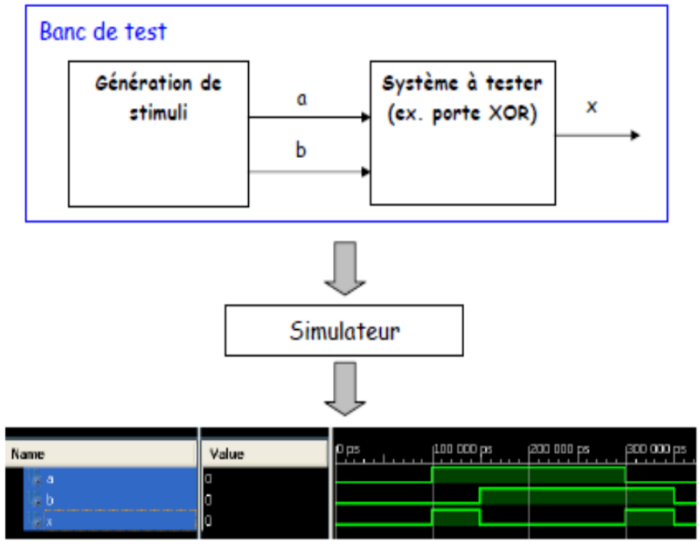


Figure 27 : Procédure de test

✓ Le code VHDL d'un banc de test

- Structure du fichier pour le système à tester

Entité (Entrés,sorties)
Architecture
Déclarations
Instructions

- Structure du fichier pour le banc de test:

Entité (ni entrée, ni sortie)
Architecture
Déclaration de composant du système à tester
Déclaration des signaux de test
Instruction d'instanciation
Instructions de génération de stimuli (entrées).

Chapitre 5.

Applications : Implémentation de quelques circuits logiques dans les circuits FPGA

I. Introduction

L'implantation d'une ou de plusieurs descriptions VHDL dans un circuit FPGA va dépendre de l'affectation que l'on fera des broches d'entrées/sorties et des structures de base du circuit logique programmable.

Le schéma ci-dessous représente un exemple de descriptions VHDL ou de blocs fonctionnels implantés dans un FPGA. Lors de la phase de synthèse chaque bloc sera matérialisé par des portes et/ou des bascules. La phase suivante sera d'implanter les portes et les bascules à l'intérieur du circuit logique. Cette tâche est réalisée par le logiciel placement/routage, au cours de laquelle les entrées et sorties seront affectées à des numéros de broches.

On peut remarquer sur le schéma la fonction particulière du bloc VHDL N°5. En effet dans la description fonctionnelle d'un FPGA on a souvent besoin d'une fonction qui sert à cadencer le fonctionnement de l'ensemble, celle-ci est très souvent réalisée par une machine d'états synchronisée par une horloge.

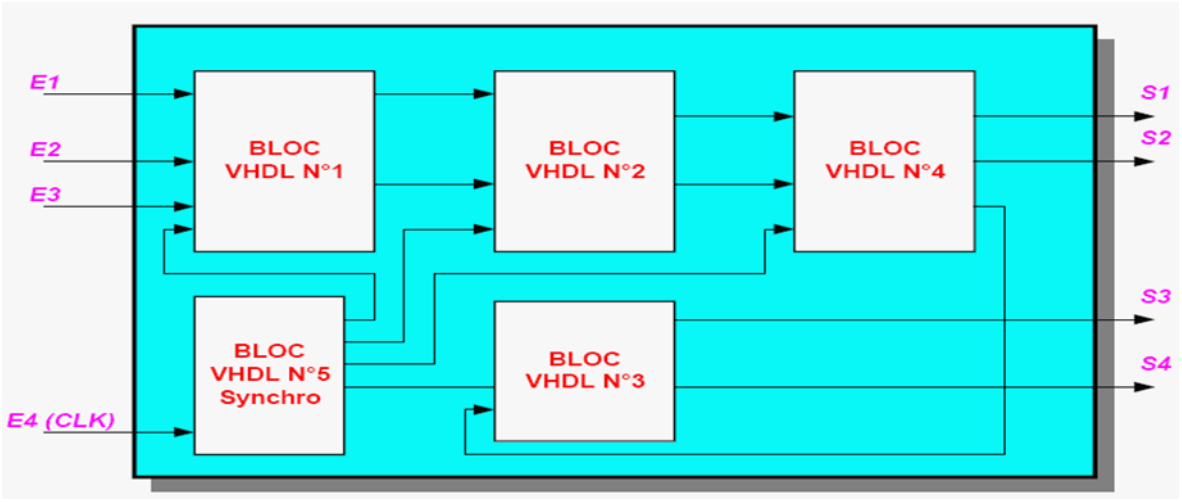


Figure 28 : Relation entre VHDL et FPGA [10]

II. Les différentes portes logiques

Réaliser les opérateurs de base de la logique combinatoire (les portes : Inverseur, ET, OU, NON-ET, NON-OU).

- ✓ Porte Inverseur :

Pour créer la **porte inverseur** vous aller cliquer dans la rubrique **Sources** sur la référence **xc3s1200e-5fg320** du projet combinatoire déjà créé auparavant ensuite dans la rubrique **Processes** vous aller cliquer sur **Create New Source** ensuite sur **VHDL Module** pour créer le fichier VHDL de la **porte_inverseur** nommé dans **file name** comme le montre la figure 28.

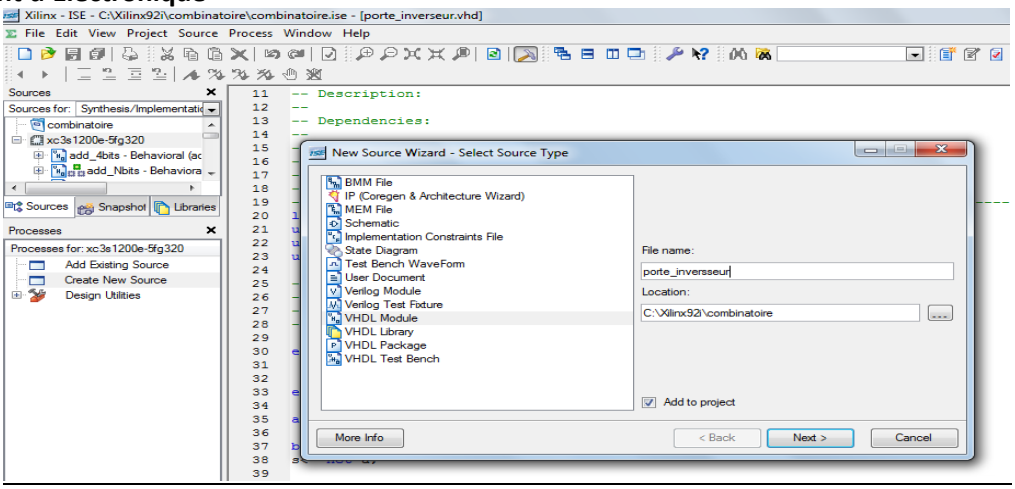


Figure 29 : première étape de la création d'un fichier VHDL

Ensuite vous aller nommé **les entrées/sorties** de la **porte_inverseur** (on choisit la direction de l'**entrée a** en **in** et la **sortie s** en **out**), le nom de l'entité (elle prendra par défaut le **nom du fichier**) et le nom de l'architecture (par défaut c'est **Behavioral**).

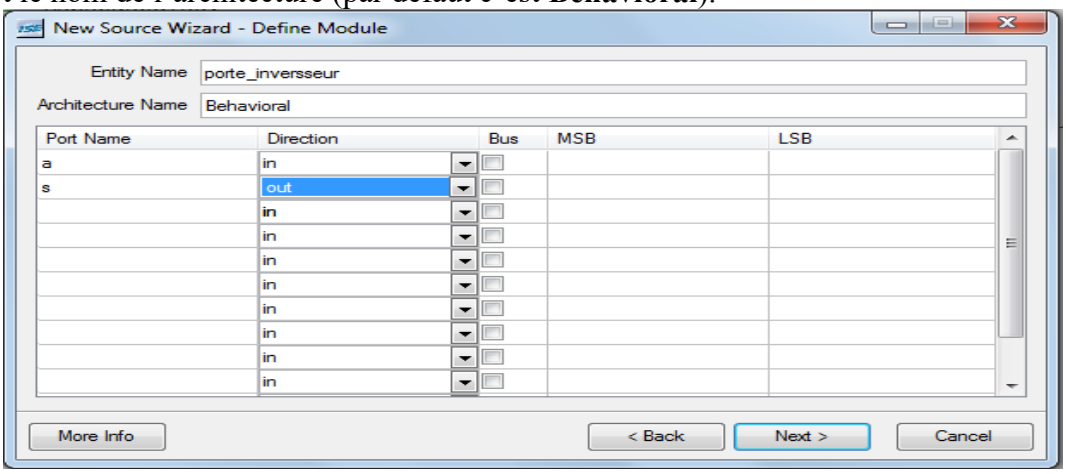


Figure 30 : deuxième étape de la création d'un fichier VHDL

Alors le programme de l'inverseur est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity porte_inverseur is
    Port ( a : in  STD_LOGIC;
           s : out  STD_LOGIC);
end porte_inverseur;
-- Déclaration de l'architecture avec une affectation simple.
architecture Behavioral of porte_inverseur is
begin
s<= not a;
end Behavioral;
```

Pour créer le **Test de la porte inverseur** vous aller cliquer dans la rubrique **Sources** sur la porte **porte_inverseur** du projet combinatoire déjà créé auparavant ensuite dans la rubrique **Processes** vous aller cliquer sur **Create New Source** ensuite sur **VHDL Test Bench** pour créer le fichier Test du VHDL : **test_porte_inverseur** nommé dans **file name** comme le montre la figure 31.

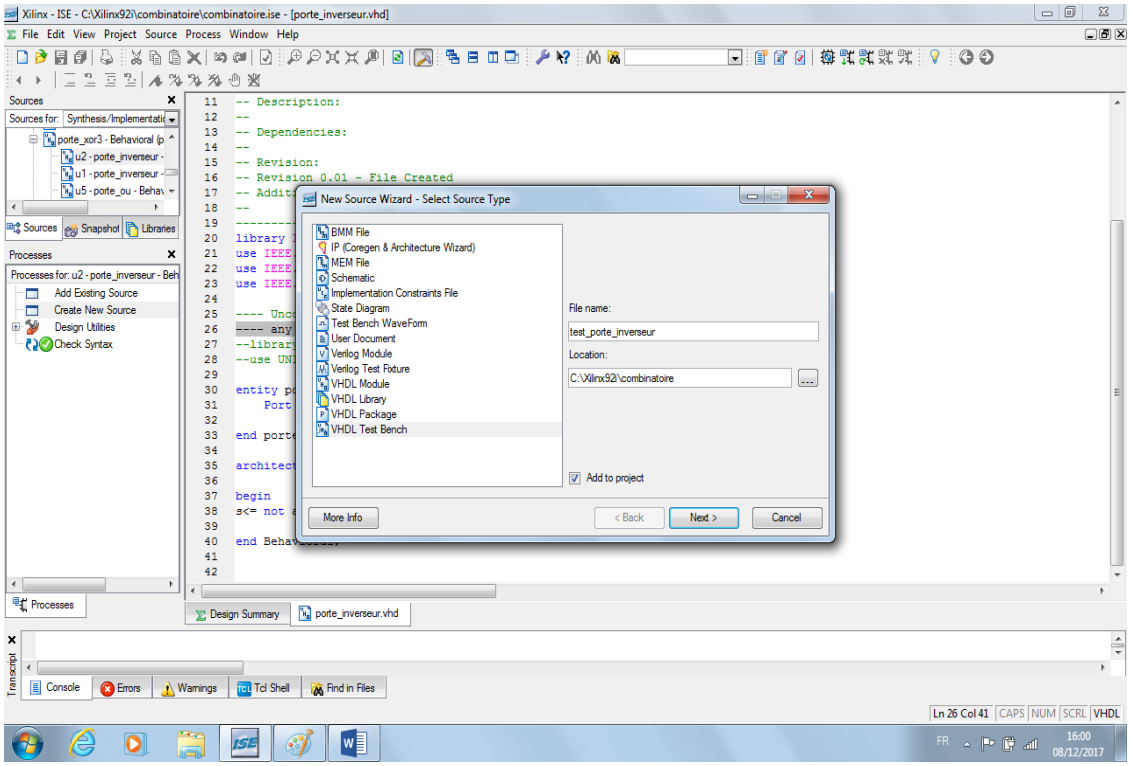


Figure 31 : la création d'un fichier test bench VHDL

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity test_port_invers_vhd is
end test_port_invers_vhd;

architecture behavior of test_port_invers_vhd is

        component porte_inverseur
        port(
                a : in std_logic;
                s : out std_logic);
        end component;

        --inputs
        signal a :  std_logic := '0';

        --outputs
        signal s :  std_logic;
begin
        -- instantiate the unit under test (uut)
```

```
        uut: porte_inverseur port map(
                a => a,
                s => s
        );
        tb : process
        begin

        a<='0';wait for 20 ns;
        a<='1';wait for 20 ns;
        wait;
        end process;
end;
```

Remarque : Que ce soit pour le **VHDL module** ou le **VHDL test bench** il faut cliquer sur **Check Syntax** dans la rubrique **Processes** pour vérifier s'il n'y a pas d'erreur de compilation**.**

Et pour voir le schéma de la porte décrite, Donner la priorité à la **porte_inverseur** en cliquons dessus, bouton droit (**set as Top module**) ensuite dans la rubrique **Processes** cliquer sur **View RTL Schematic** dans la partie Synthesize-XST.

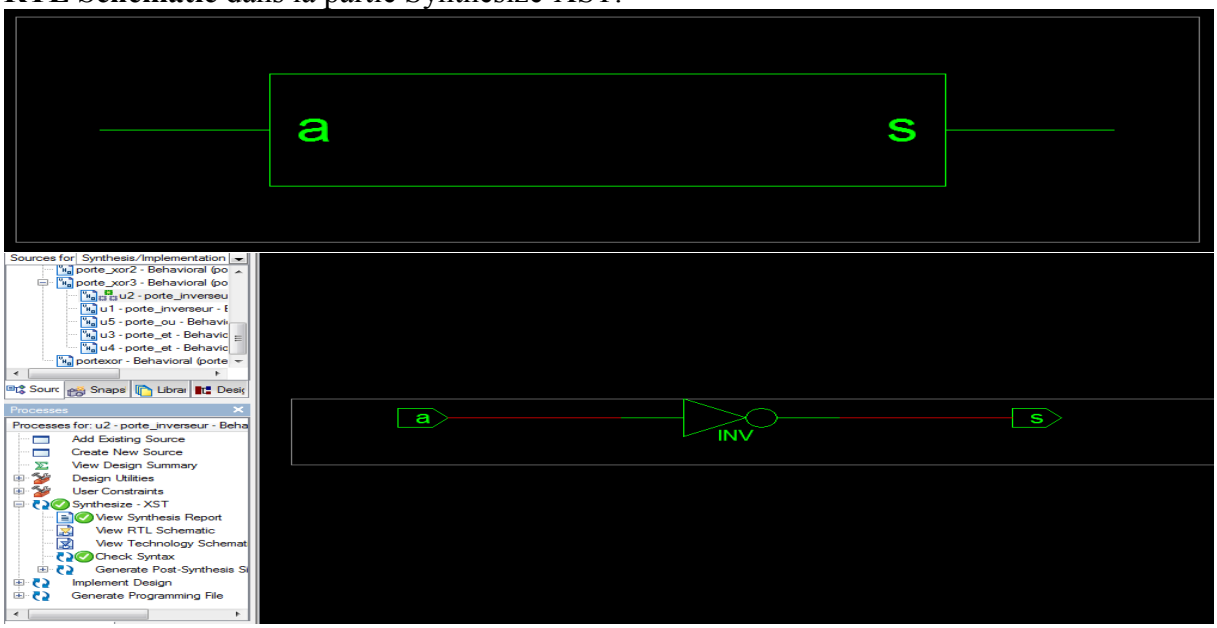


Figure 32 : le schéma de la porte_inversseur

Et enfin dans la rubrique **behavioral Simulation**, on simule notre porte en donnant des valeurs pour l'entrée **a** et voir la sortie **S** :

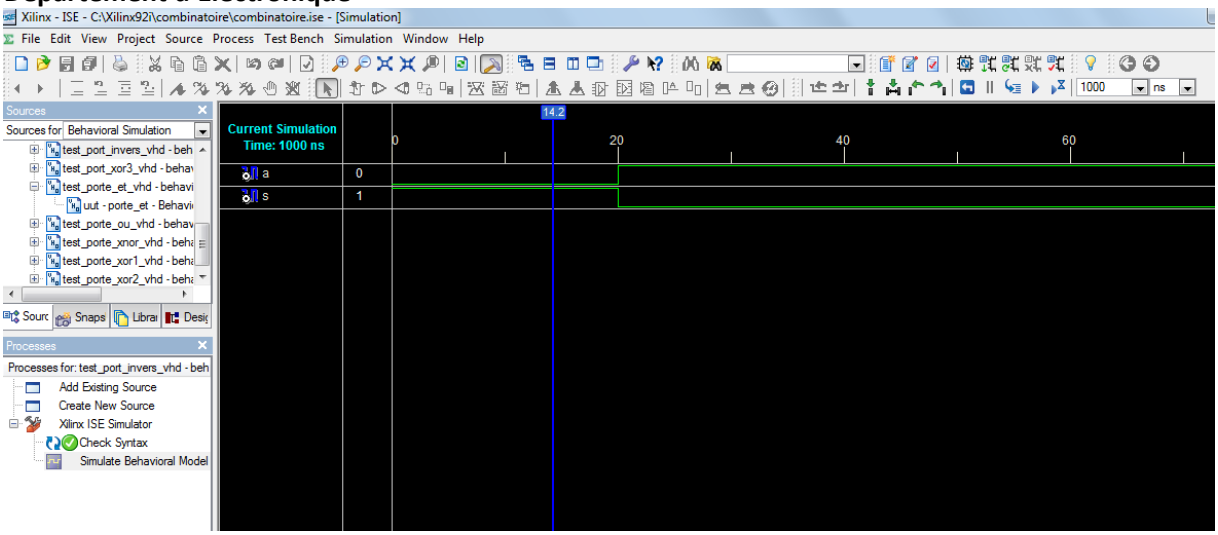


Figure 33 : la simulation de la porte_inverseur

De la même manière que dans la porte inverseur nous allons créer les autres portes :

- ✓ Porte AND :

Alors le programme de la porte_et est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity porte_et is
    Port ( a : in  STD_LOGIC;
           b: in  STD_LOGIC;
           s : out  STD_LOGIC);
end porte_et;
-- Déclaration de l'architecture avec une affectation simple.
architecture Behavioral of porte_et is
begin
s<=  a and b;
end Behavioral;
```

Le fichier Test du VHDL : **test_porte_et** nommé dans **file name** on change juste la partie du **tb :process**

```
tb : process
begin
a<= '0'; b<= '0'; wait for 20 ns;
a<= '1'; b<= '0'; wait for 20 ns;
a<= '0'; b<= '1'; wait for 20 ns;
a<= '1'; b<= '1'; wait for 20 ns;
wait;
end process;
end;
```

Et pour voir le schéma de la porte décrite, Donner la priorité à la **porte_et** en cliquons dessus, bouton droit (**set as Top module**) ensuite dans la rubrique **Processes** cliquer sur **View RTL Schematic** dans la partie Synthesize-XST.

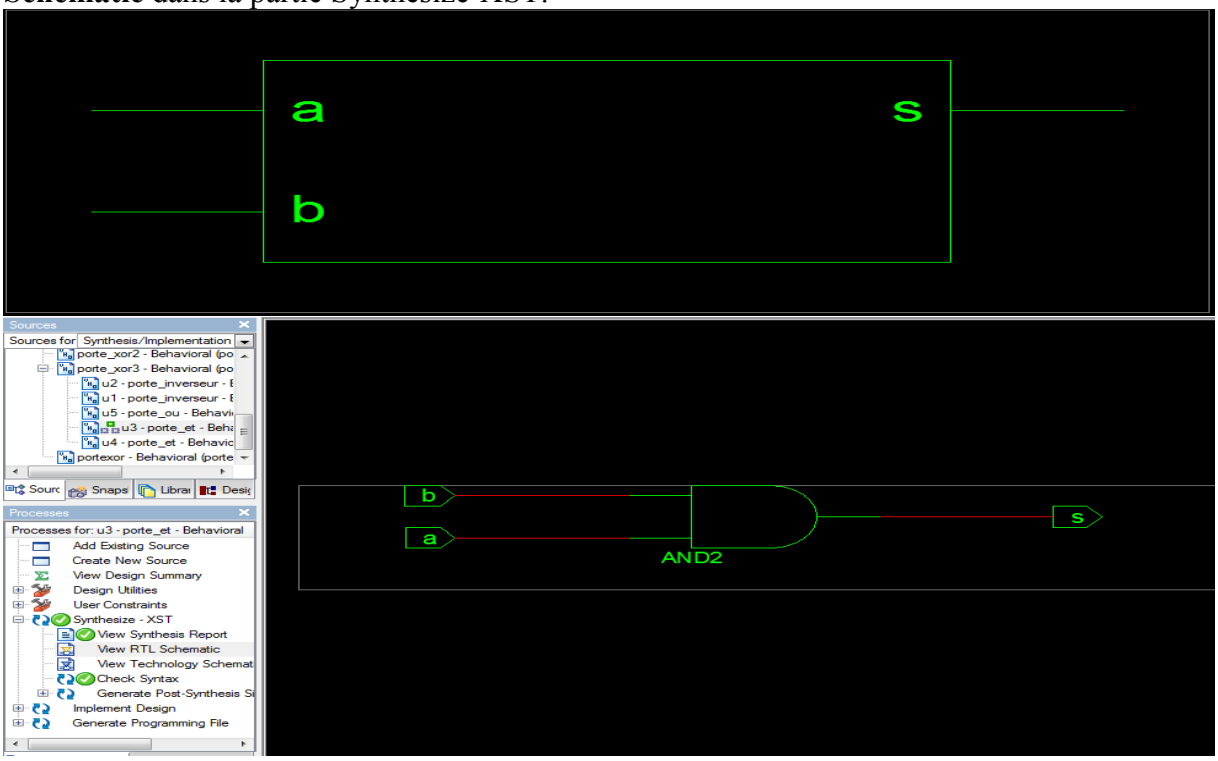


Figure 34 : le schéma de la porte_et

Et enfin dans la rubrique **behavioral Simulation**, on simule notre porte en donnant des valeurs pour l'entrée **a** et **b** pour voir la sortie **S** :

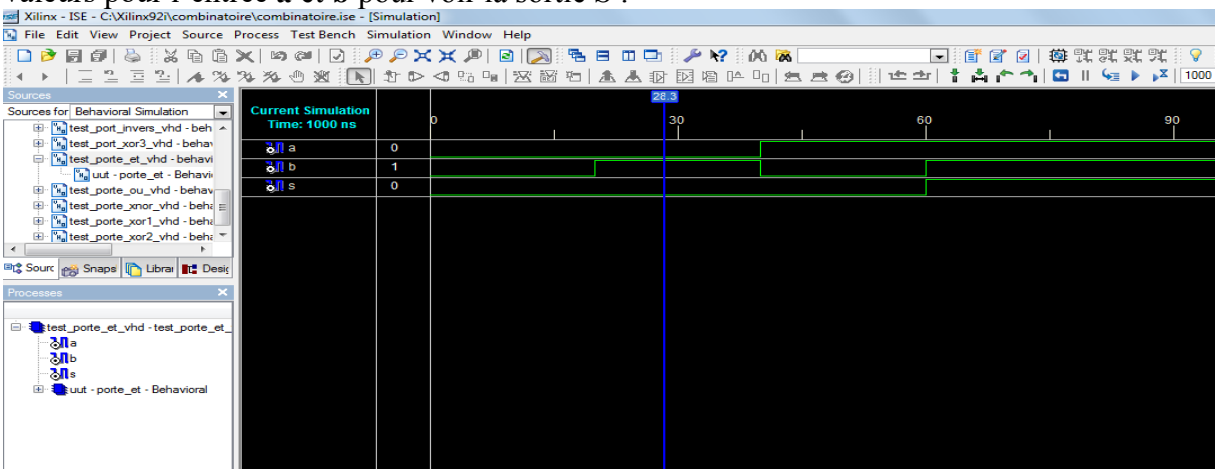


Figure 35 : la simulation de la porte_et

✓ Porte OR :

Alors le programme de la porte_ou est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity porte_ou is
```

```
        Port ( a : in  STD_LOGIC;
               b: in  STD_LOGIC;
               s : out  STD_LOGIC);
    end porte_ou;
    -- Déclaration de l'architecture avec une affectation simple.
    architecture Behavioral of porte_ou is
    begin
    s<=  a or b;
    end Behavioral;
```

Le fichier Test du VHDL : **test_porte_ou** nommé dans **file name** on change juste la partie du **tb :process**

```
    tb : process
    begin
    a<= '0'; b<= '0'; wait for 20 ns;
    a<= '1'; b<= '0'; wait for 20 ns;
   a<= '0'; b<= '1'; wait for 20 ns;
    a<= '1'; b<= '1'; wait for 20 ns;
    wait;
    end process;
end;
```

Et pour voir le schéma de la porte décrite, Donner la priorité à la **porte_ou** en cliquons dessus, bouton droit (**set as Top module**) ensuite dans la rubrique **Processes** cliquer sur **View RTL Schematic** dans la partie Synthesize-XST.

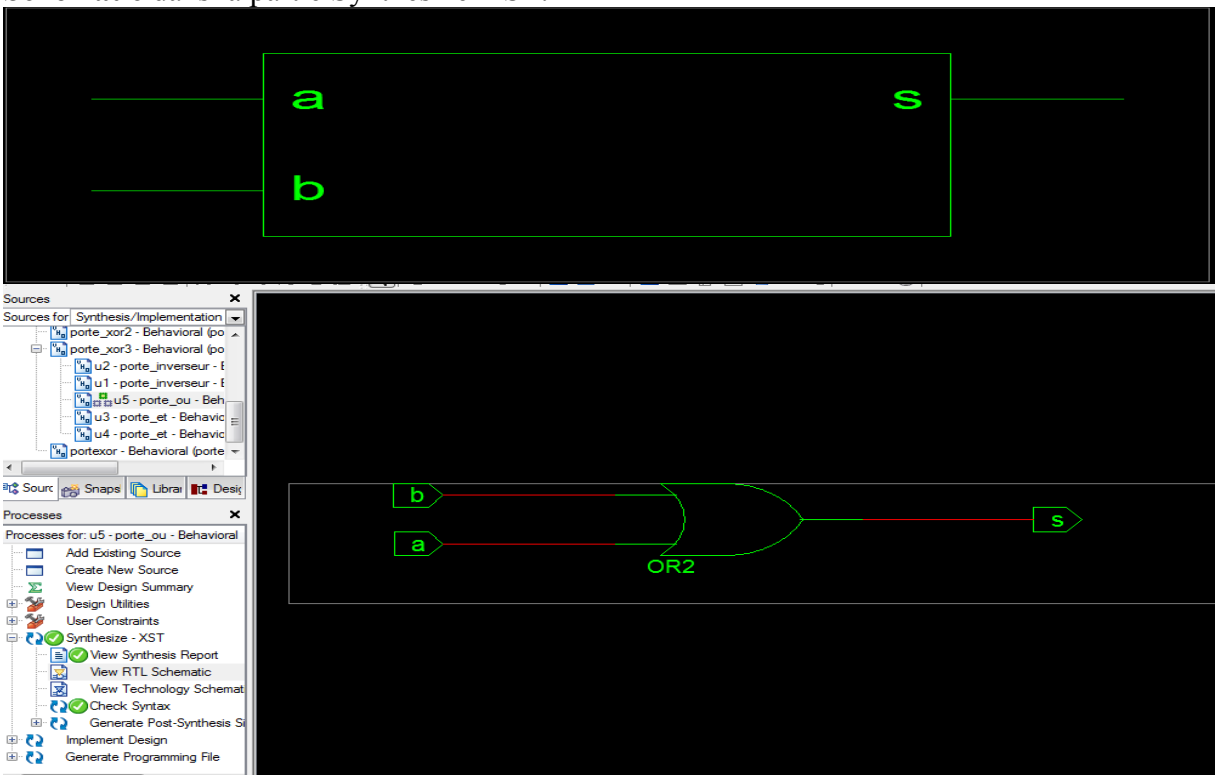


Figure 36 : le schéma de la porte_ou

Et enfin dans la rubrique **behavioral Simulation**, on simule notre porte en donnant des valeurs pour l'entrée **a** et **b** pour voir la sortie **S** :

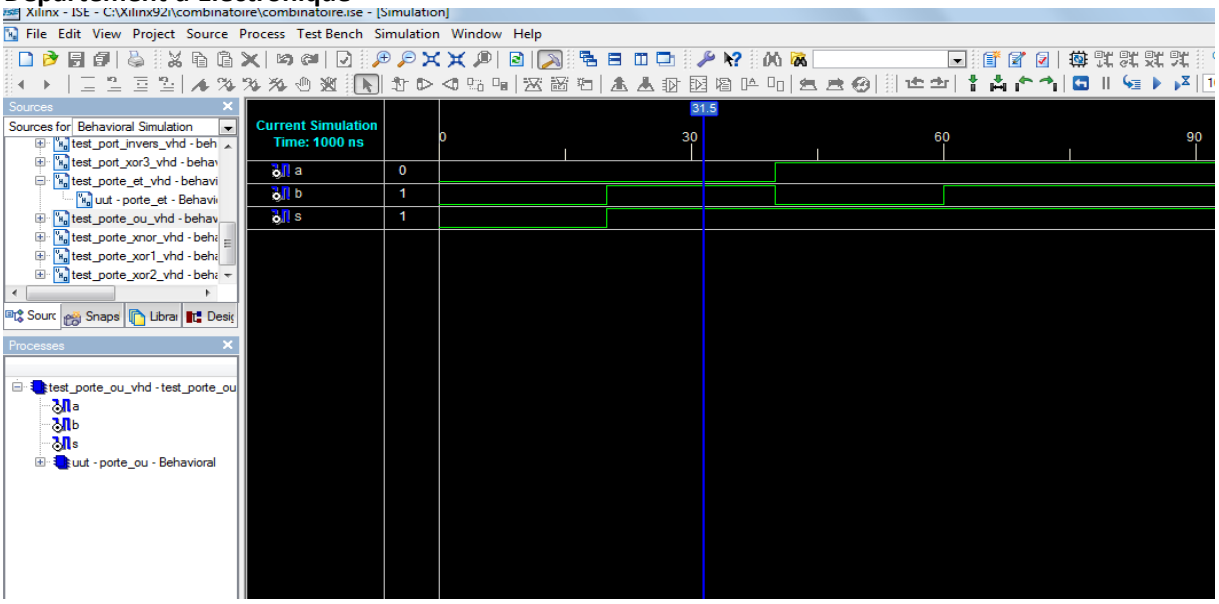


Figure 37 : la simulation de la porte_ou

✓ Porte NAND :

Alors le programme de la porte_non_et est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity porte_non_et is
   Port ( a : in  STD_LOGIC;
          b: in  STD_LOGIC;
          s : out  STD_LOGIC);
end porte_non_et;
-- Déclaration de l'architecture avec une affectation simple.
architecture Behavioral of porte_non_et is
begin
s<=  a nand b;
end Behavioral;
```

Le fichier Test du VHDL : **test_porte_non_et** nommé dans **file name** on change juste la partie du **tb :process**

```
tb : process
begin

a<= '0'; b<= '0'; wait for 20 ns;
a<= '1'; b<= '0'; wait for 20 ns;
a<= '0'; b<= '1'; wait for 20 ns;
a<= '1'; b<= '1'; wait for 20 ns;
wait;
end process;
```

```
end;
```

Et pour voir le schéma de la porte décrite, Donner la priorité à la **porte_non_et** en cliquons dessus, bouton droit (**set as Top module**) ensuite dans la rubrique **Processes** cliquer sur **View RTL Schematic** dans la partie Synthesize-XST.

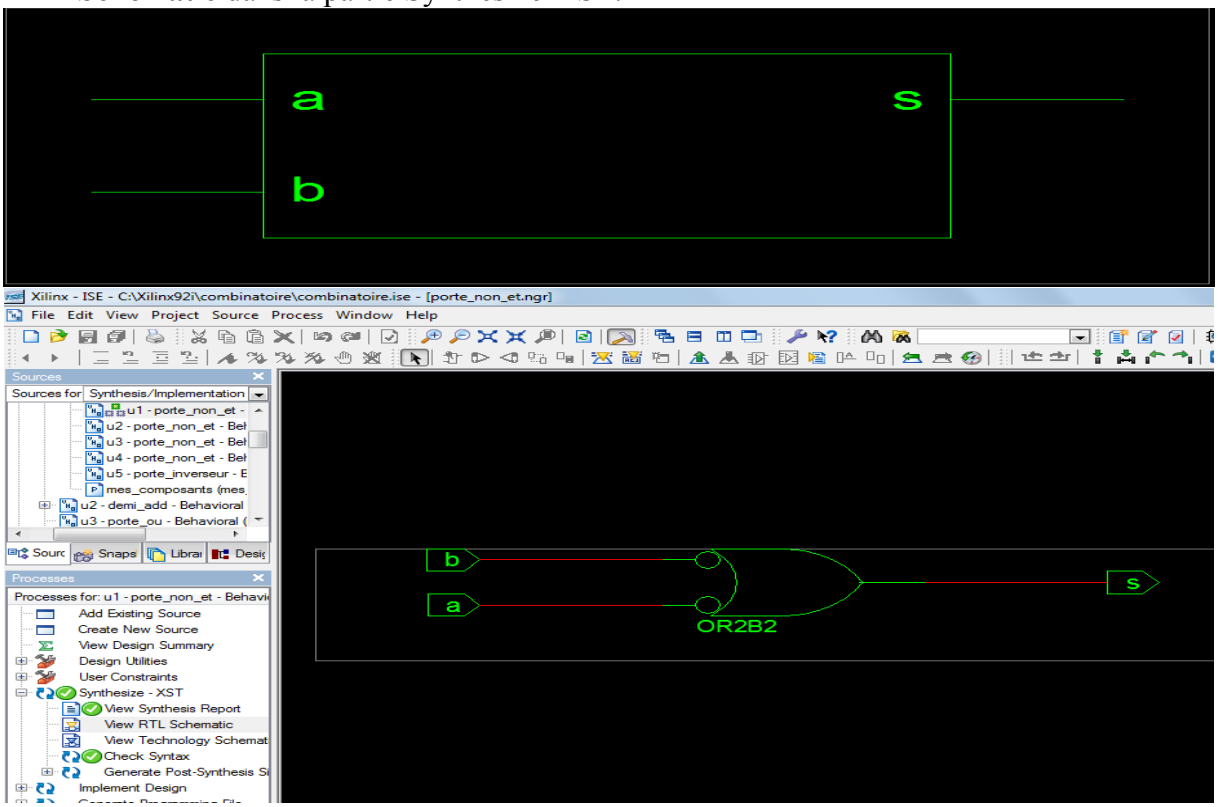


Figure 38 : le schéma de la porte_non_et

Et enfin dans la rubrique **behavioral Simulation**, on simule notre porte en donnant des valeurs pour l'entrée **a** et **b** pour voir la sortie **S** :

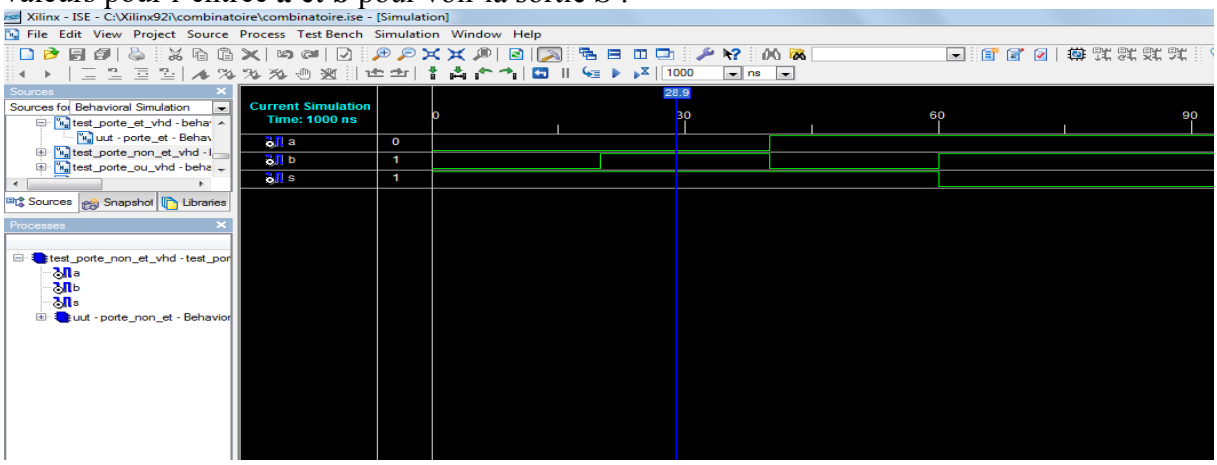


Figure 39 : la simulation de la porte_non_et

✓ Porte NOR :

Alors le programme de la porte_non_ou est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

```
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity porte_non_ou is
   Port ( a : in  STD_LOGIC;
          b: in  STD_LOGIC;
          s : out  STD_LOGIC);
end porte_non_ou;
-- Déclaration de l'architecture avec une affectation simple.
architecture Behavioral of porte_non_ou is
begin
s<=  a nor b;
end Behavioral;
```

Le fichier Test du VHDL : **test_porte_non_ou** nommé dans **file name** on change juste la partie du **tb :process**

```
tb : process
begin
a<= '0'; b<= '0'; wait for 20 ns;
a<= '1'; b<= '0'; wait for 20 ns;
a<= '0'; b<= '1'; wait for 20 ns;
a<= '1'; b<= '1'; wait for 20 ns;
wait;
end process;
end;
```

Et pour voir le schéma de la porte décrite, Donner la priorité à la **porte_non_ou** en cliquons dessus, bouton droit (**set as Top module**) ensuite dans la rubrique **Processes** cliquer sur **View RTL Schematic** dans la partie Synthesize-XST.

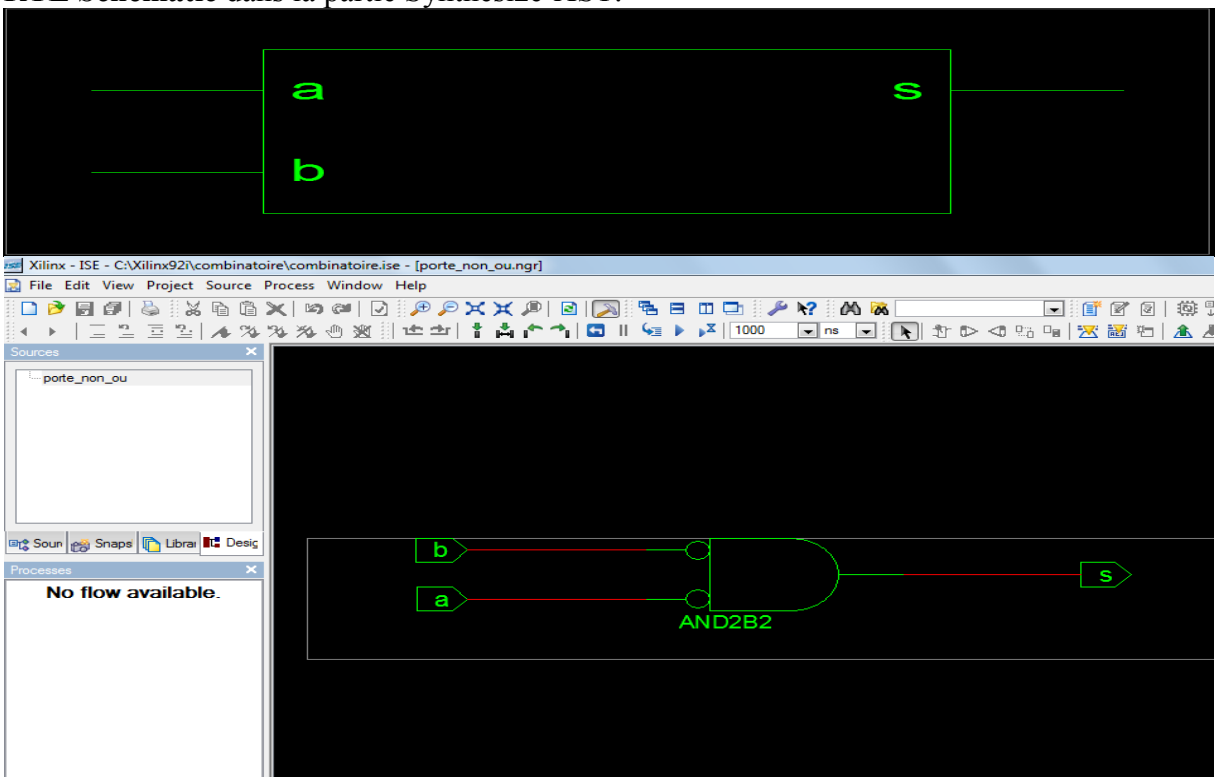


Figure 40 : le schéma de la porte_non_ou

Et enfin dans la rubrique **behavioral Simulation**, on simule notre porte en donnant des valeurs pour l'entrée **a** et **b** pour voir la sortie **S** :

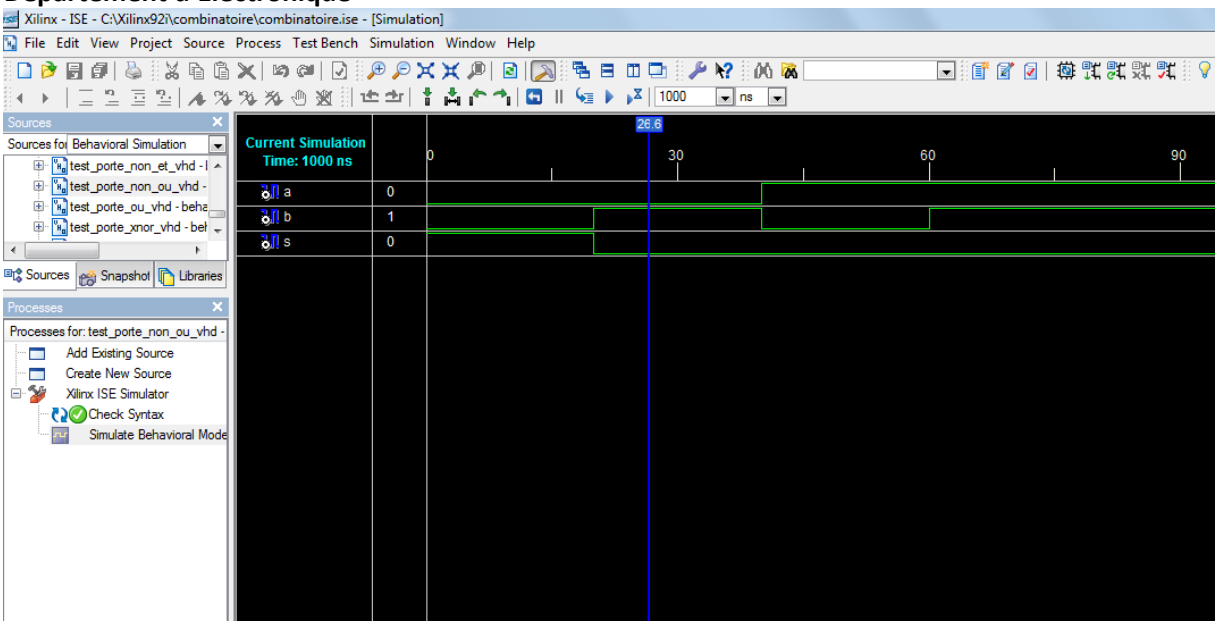


Figure 41 : la simulation de la porte_non_ou

1- Réaliser la porte XOR en utilisant les trois descriptions (flot de données, comportementale, structurelle) en faisant appel aux composants déjà réalisé en utilisant : **COMPONENT**.

✓ Porte XOR1 : flot de données

Alors le programme de la porte_xor1 est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity porte_xor1 is
   Port ( a : in  STD_LOGIC;
          b: in  STD_LOGIC;
          s : out  STD_LOGIC);
end porte_xor1;
-- Déclaration de l'architecture avec une affectation conditionnelle.
architecture Behavioral of porte_xor1 is
begin
s<= '0' when a=b else '1';
end Behavioral;
```

✓ Porte XOR2 : comportemental

Alors le programme de la porte_xor2 est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity porte_xor2 is
   Port ( a : in  STD_LOGIC;
          b: in  STD_LOGIC;
          s : out  STD_LOGIC);
```

```
end porte_xor2;
-- Déclaration de l'architecture avec une structure de test.
architecture Behavioral of porte_xor1 is
begin
process (a,b)
begin
if a=b then s<= '0';
else s<= '1';
end if;
end process;
end Behavioral;
```

✓ Porte XOR3 : structurelle

Alors le programme de la porte_xor3 est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity porte_xor3 is
   Port ( a : in  STD_LOGIC;
          b: in  STD_LOGIC;
          s : out  STD_LOGIC);
end porte_xor3;
-- Déclaration de l'architecture en structurelle.
architecture Behavioral of porte_xor3 is
signal s1,s2,s3,s4:std_logic;
component porte_inverseur
  port (a:in std_logic;
        s:out std_logic);
end component;
component porte_et
  port (a:in std_logic;
        b:in std_logic;
        s:out std_logic);
end component;
component porte_ou
  port (a:in std_logic;
        b:in std_logic;
        s:out std_logic);
end component;
begin
u2:porte_inverseur port map (b,s2);
u1:porte_inverseur port map (a,s1);
u5:porte_ou port map (s3,s4,s);
u3:porte_et port map (s1,b,s3);
u4:porte_et port map (a,s2,s4);
end Behavioral;
```

Le fichier Test du VHDL : **test_porte_xor 1,2 et 3** c'est le même on change juste la partie du **tb :process**

```
        tb : process
        begin
        a<= '0'; b<= '0'; wait for 20 ns;
        a<= '1'; b<= '0'; wait for 20 ns;
       a<= '0'; b<= '1'; wait for 20 ns;
        a<= '1'; b<= '1'; wait for 20 ns;
        wait;
        end process;
end;
```

Et pour voir le schéma de la porte décrite, Donner la priorité à la **porte_xor3** en cliquons dessus, bouton droit (**set as Top module**) ensuite dans la rubrique **Processes** cliquer sur **View RTL Schematic** dans la partie Synthesize-XST.

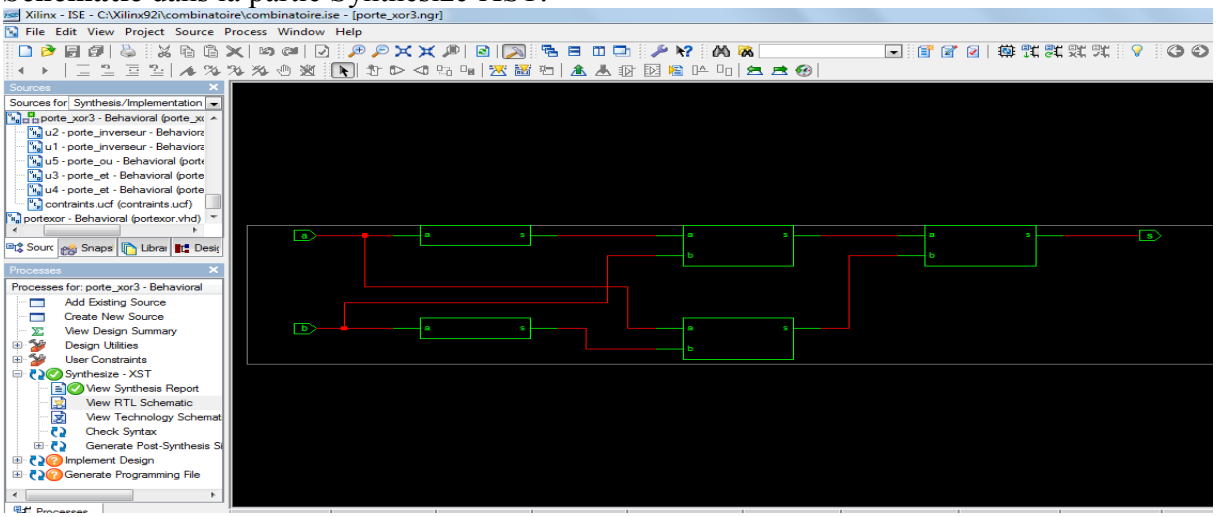


Figure 42 : le schéma de la porte_xor3

Et enfin dans la rubrique **behavioral Simulation**, on simule notre porte on donnant des valeurs pour l'entrée **a** et **b** pour voir la sortie **S** :

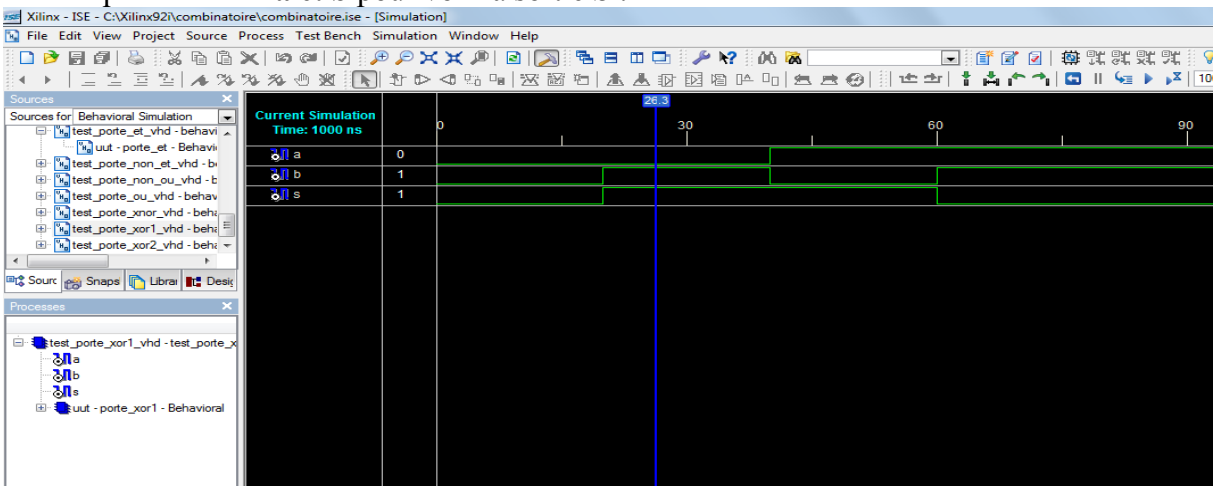


Figure 43 : la simulation de la porte_xor1,2 et 3

III. Multiplexeur

Réaliser un multiplexeur (2 vers 1) en utilisant les trois descriptions (flot de données, comportementale, structurelle).

✓ Porte Multiplexeur 1 : flot de données

Alors le programme de la porte_mux1 est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity porte_mux1 is
   Port ( in1 : in  STD_LOGIC;
          In2: in  STD_LOGIC;
          sel : in  STD_LOGIC;
          s : out  STD_LOGIC);
end porte_mux1;
-- Déclaration de l'architecture avec une affectation conditionnelle.
architecture Behavioral of porte_mux1 is
begin
s<= in1 when sel = '0';
           else s<= in2;
end Behavioral;
```

✓ Porte Multiplexeur 2 : comportemental

Alors le programme de la porte_mux2 est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity porte_mux2 is
   Port ( in1 : in  STD_LOGIC;
          In2: in  STD_LOGIC;
          sel : in  STD_LOGIC;
          s : out  STD_LOGIC);
end porte_mux2;
-- Déclaration de l'architecture avec une structure de test.
architecture Behavioral of porte_mux2 is
begin
process (in1,in2, sel)
begin
if sel = '0' then s <=in1;
else s<= in2;
end if;
end process;
end Behavioral;
```

✓ Porte Multiplexeur 3 : structurelle

Alors le programme de la porte_mux3 est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration du paquetage spécifique pour appeler les components.
use work.mes_composants.all;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
```

```
entity porte_mux3 is
   Port ( in1 : in  STD_LOGIC;
          In2: in  STD_LOGIC;
          sel : in  STD_LOGIC;
          s : out  STD_LOGIC);
end porte_mux3;
-- Déclaration de l'architecture en structurelle.
architecture Behavioral of porte_mux3 is
signal s1,s2,s3:std_logic;
begin
u1:porte_inverseur port map (sel,s1);
u4:porte_ou port map (s2,s3,s);
u2:porte_et port map (in1,s1,s2);
u3:porte_et port map (in2,sel,s3);
end Behavioral;
```

Le fichier Test du VHDL : **test_porte_xor 1,2 et 3** c'est le même on change juste la partie du **tb :process**

```
tb : process
begin
in1<='0'; in2<='0'; sel<='0'; wait for 20 ns;
in1<='0'; in2<='0'; sel<='1'; wait for 20 ns;
in1<='0'; in2<='1'; sel<='0'; wait for 20 ns;
in1<='0'; in2<='1'; sel<='1'; wait for 20 ns;
in1<='1'; in2<='0'; sel<='0'; wait for 20 ns;
in1<='1'; in2<='0'; sel<='1'; wait for 20 ns;
in1<='1'; in2<='1'; sel<='0'; wait for 20 ns;
in1<='1'; in2<='1'; sel<='1'; wait for 20 ns;
wait;
end process;
end;
```

Et pour voir le schéma de la porte décrite, Donner la priorité à la **porte_mux3** en cliquons dessus, bouton droit (**set as Top module**) ensuite dans la rubrique **Processes** cliquer sur **View RTL Schematic** dans la partie Synthesize-XST.

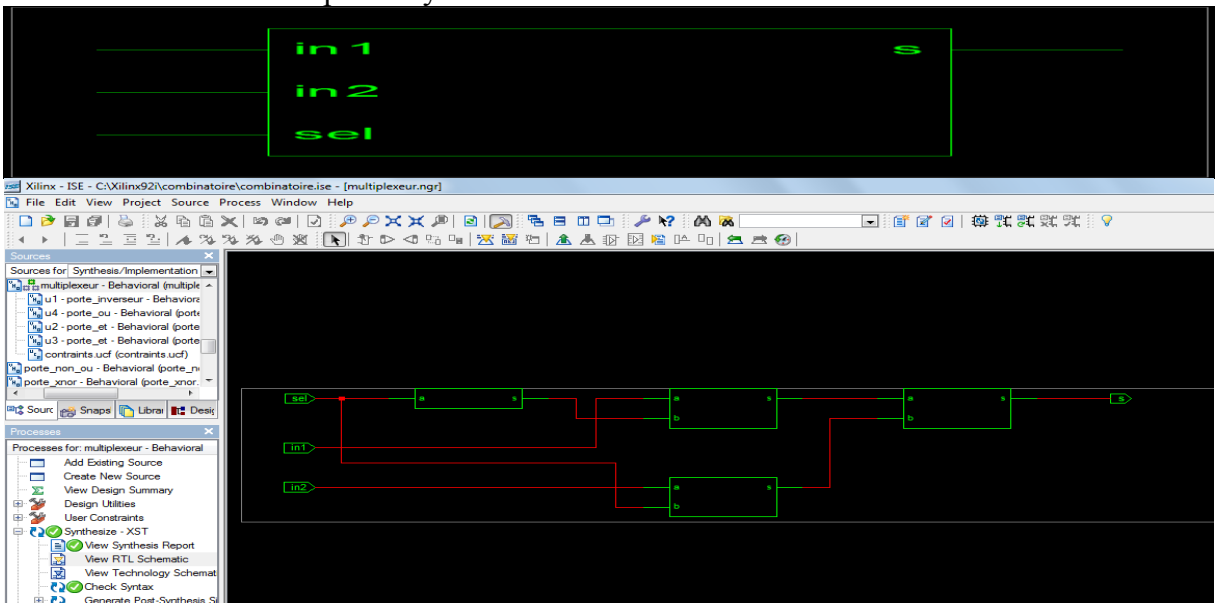


Figure 44 : le schéma de la porte_mux3

Et enfin dans la rubrique **behavioral Simulation**, on simule notre porte en donnant des valeurs pour l'entrée **in1, in2** et **sel** pour voir la sortie **S** :

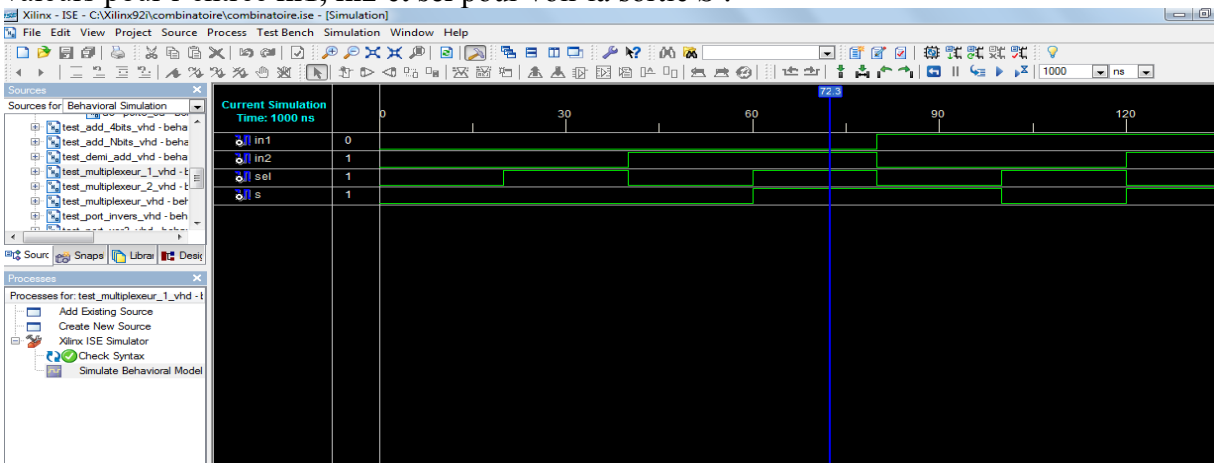


Figure 45 : la simulation de la porte_mux1,2 et 3

IV. Demi-additionneur

Réaliser un demi-additionneur en utilisant les portes NAND (NON-ET) et l'inverseur dans une description structurelle.

✓ Demi-additionneur : structurelle

Alors le programme du demi_add est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration du paquetage spécifique pour appeler les components.
use work.mes_composants.all;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity demi_add is
   Port ( a : in  STD_LOGIC;
          b: in  STD_LOGIC;
          s : out  STD_LOGIC;
          cb : out  STD_LOGIC);
end demi_add;
-- Déclaration de l'architecture en structurelle.
architecture Behavioral of demi_add is
signal s1,s2,s3:std_logic;
begin
u1:porte_non_et port map (a,b,s1);
u2:porte_non_et port map (a,s1,s2);
u3:porte_non_et port map (s1,b,s3);
u4:porte_non_et port map (s2,s3,s);
u5:porte_inverseur port map (s1,cb);
end Behavioral;
```

Le fichier Test du VHDL : **test_demi_add** c'est le même on change juste la partie du **tb :process**

```
tb : process
begin
a<= '0'; b<= '0'; wait for 20 ns;
```

```
        a<= '1'; b<= '0'; wait for 20 ns;
      a<= '0'; b<= '1'; wait for 20 ns;
        a<= '1'; b<= '1'; wait for 20 ns;
wait;
end process;
end;
```

Et pour voir le schéma de la porte décrite, Donner la priorité à la **demi_add** en cliquons dessus, bouton droit (**set as Top module**) ensuite dans la rubrique **Processes** cliquer sur **View RTL Schematic** dans la partie Synthesize-XST.

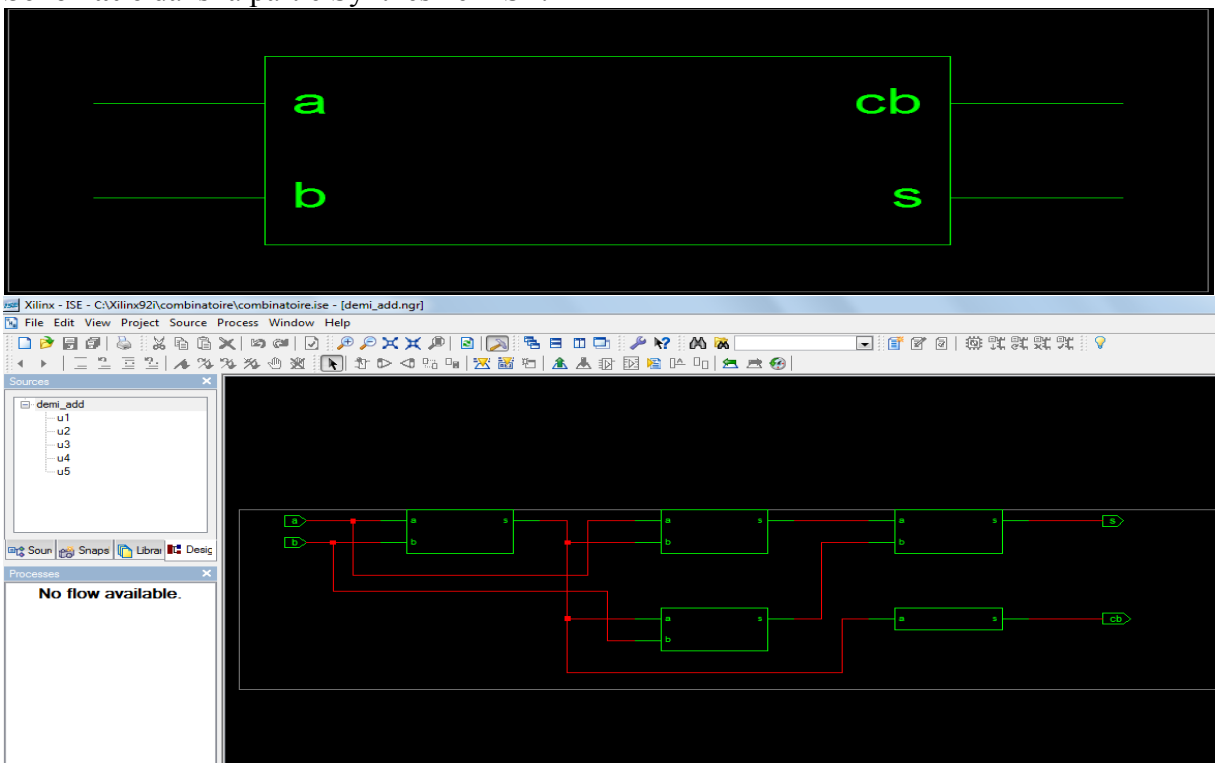


Figure 46 : le schéma du demi_add

Et enfin dans la rubrique **behavioral Simulation**, on simule notre porte en donnant des valeurs pour l'entrée **a** et **b** pour voir la sortie **S et cb** :

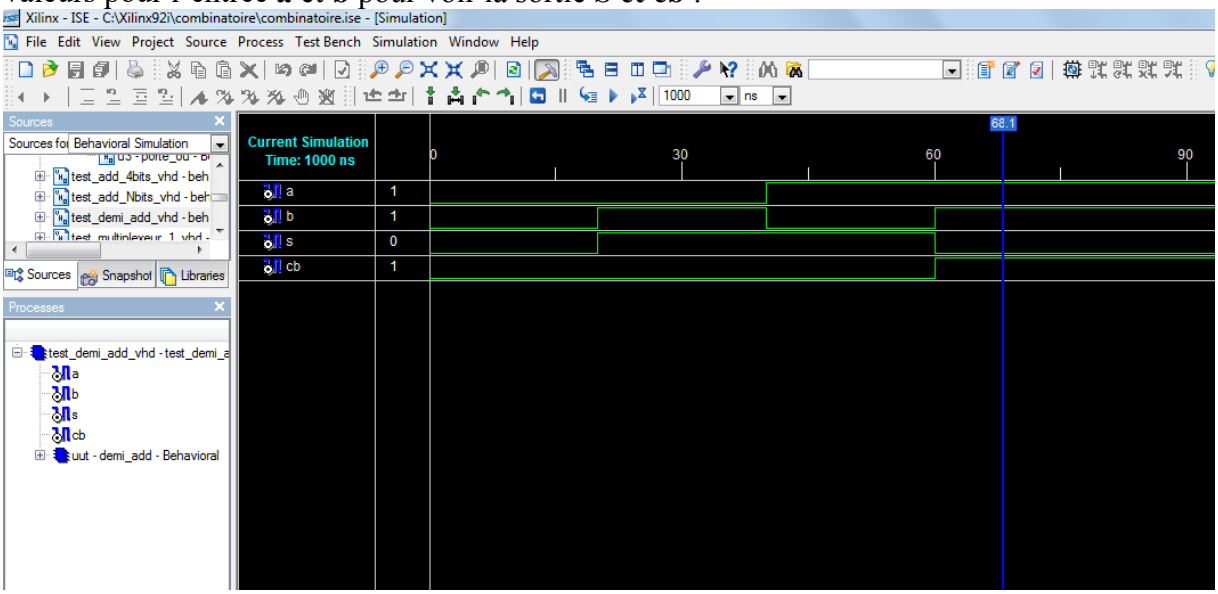


Figure 47 : la simulation du demi_add

V. Additionneur complet

Réaliser un additionneur complet de 1 bit en utilisant le demi_additionneur de 1bit ainsi qu'une retenue à l'entrée et sortie du système.

✓ Additionneur complet 1bit: structurelle

Alors le programme du add_1bit est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration du paquetage spécifique pour appeler les components.
use work.mes_composants.all;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity add_1bit is
   Port ( a : in  STD_LOGIC;
          b: in  STD_LOGIC;
          Cin : in  STD_LOGIC;
          Sortie : out  STD_LOGIC;
          Cout : out  STD_LOGIC);
end add_1bit;
-- Déclaration de l'architecture en structurelle.
architecture Behavioral of add_1bit  is
signal s1,s2,s3:std_logic;
begin
u1:demi_add port map (a,b,s1,S2);
u2:demi_add port map (Cin,s1,sortie,s3);
u3:porte_ou port map (s3,s2,Cout);
end Behavioral;
```

Le fichier Test du VHDL : **test_ add_1bit** c'est le même on change juste la partie du **tb :process**

```
tb : process
begin
a<='0'; b<='0'; Cin<= '0'; wait for 20 ns;
a<='0'; b<='0'; Cin<= '1'; wait for 20 ns;
a<='0'; b<='1'; Cin<= '0'; wait for 20 ns;
a<='0'; b<='1'; Cin<= '1'; wait for 20 ns;
a<='1'; b<='0'; Cin<= '0'; wait for 20 ns;
a<='1'; b<='0'; Cin<= '1'; wait for 20 ns;
a<='1'; b<='1'; Cin<= '0'; wait for 20 ns;
a<='1'; b<='1'; Cin<= '1'; wait for 20 ns;
wait;
end process;
end;
```

Et pour voir le schéma de la porte décrite, Donner la priorité à la **add_1bit** en cliquons dessus, bouton droit (**set as Top module**) ensuite dans la rubrique **Processes** cliquer sur **View RTL Schematic** dans la partie Synthesize-XST.

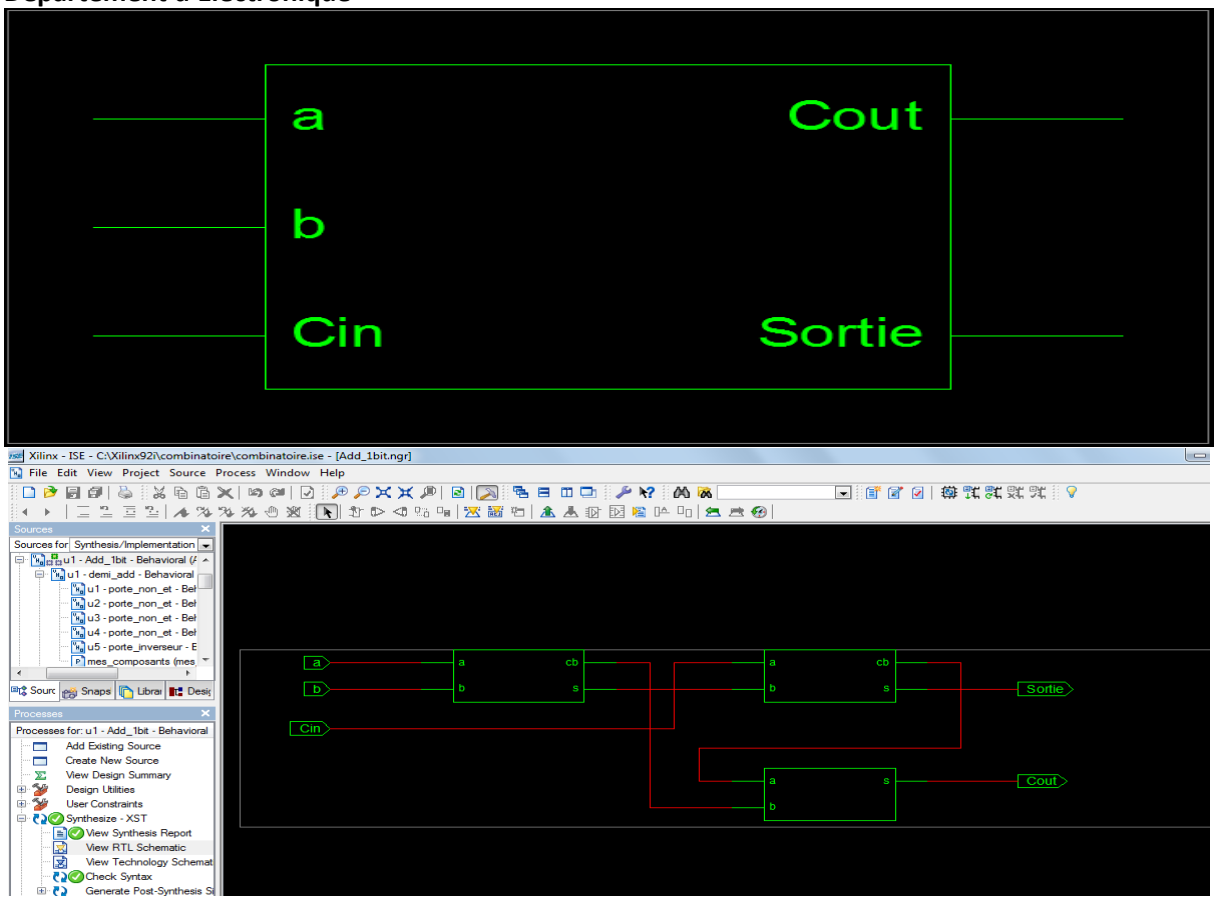


Figure 48 : le schéma d'add_1bit

Et enfin dans la rubrique **behavioral Simulation**, on simule notre porte en donnant des valeurs pour l'entrée **a, b** et **Cin** pour voir la sortie **Sortie et Cout** :

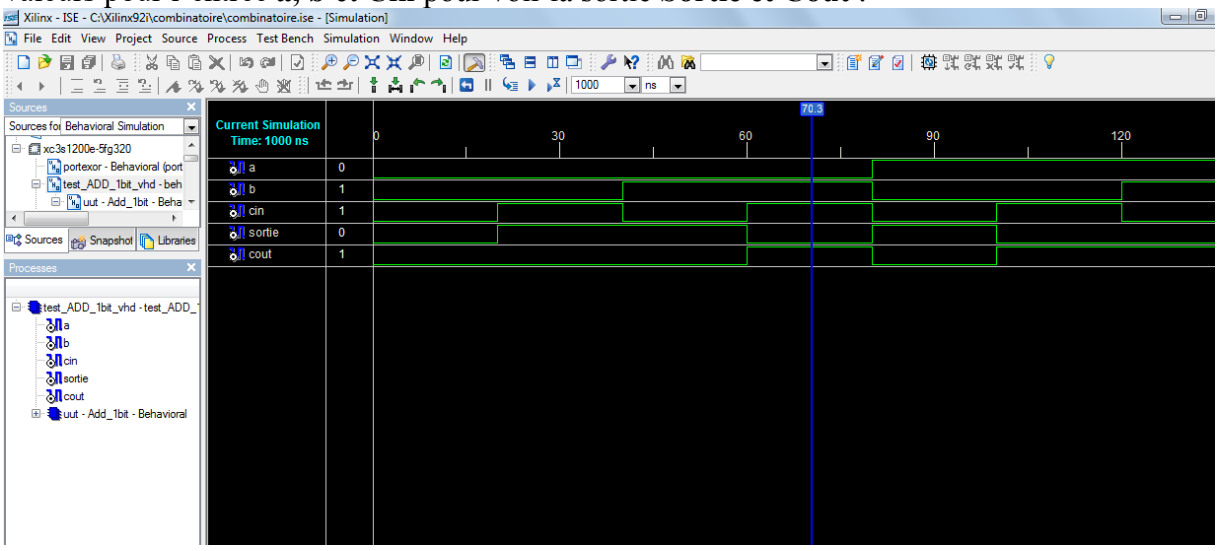


Figure 49 : la simulation d'add_1bit

1- Réaliser un additionneur complet de 4 bits en utilisant l'additionneur complet de 1bit. une description structurelle ainsi qu'un paquetage spécifique sont utilisés.

✓ Additionneur complet 4bits: structurelle

Nommé **les entrées/sorties** de l'**additionneur 4bits (**on choisit la direction de l'**entrée a** en **in** et la **sortie s** en **out**), le nom de l'entité (elle prendra par défaut le **nom du fichier**) et le nom de l'architecture (par défaut c'est **Behavioral**).

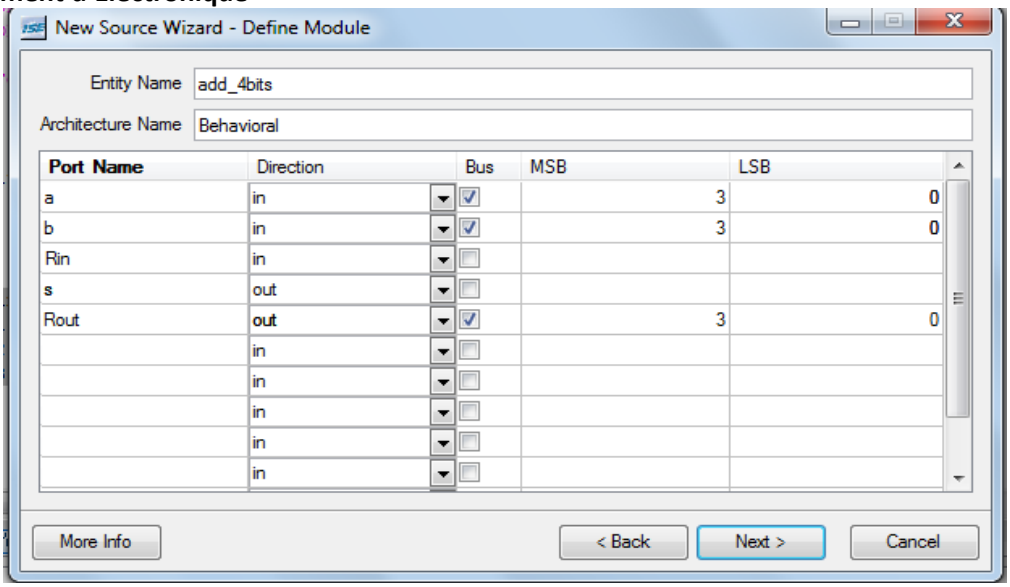


Figure 50 : la création d'un fichier VHDL des bits vecteurs

Alors le programme du add_4bits est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration du paquetage spécifique pour appeler les components.
use work.mes_composants.all;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity add_4bits is
   Port ( a : in  STD_LOGIC_VECTOR (3 downto 0);
          b: in  STD_LOGIC_VECTOR (3 downto 0);
          Rin : in  STD_LOGIC;
          s : out  STD_LOGIC_VECTOR (3 downto 0);
          Rout : out  STD_LOGIC);
end add_4bits;
-- Déclaration de l'architecture en structurelle.
architecture Behavioral of add_4bits  is
signal T1,T2,T3:std_logic;
begin
u1:Add_1bit port map (a(0),b(0),Rin,S(0),T1);
u2:Add_1bit port map (a(1),b(1),T1,S(1),T2);
u3:Add_1bit port map (a(2),b(2),T2,S(2),T3);
u4:Add_1bit port map (a(3),b(3),T3,S(3),Rout);
end Behavioral;
```

Le fichier Test du VHDL : **test_add_4bits** c'est le même on change juste la partie du **tb :process**

```
tb : process
begin
a<="0000"; b<="0000"; Rin<= '0'; wait for 20 ns;
a<="0100"; b<="0101"; Rin<= '1'; wait for 20 ns;
a<="1010"; b<="0110"; Rin<= '0'; wait for 20 ns;
a<="1100"; b<="0111"; Rin<= '1'; wait for 20 ns;
a<="0101"; b<="0111"; Rin<= '0'; wait for 20 ns;
```

```
a<="1000"; b<="0101"; Rin<= '1'; wait for 20 ns;
a<="0010"; b<="1011"; Rin<= '0'; wait for 20 ns;
a<="1110"; b<="0110"; Rin<= '1'; wait for 20 ns;
wait;
end process;
end;
```

Et pour voir le schéma de la porte décrite, Donner la priorité à la **add_4bits** en cliquons dessus, bouton droit (**set as Top module**) ensuite dans la rubrique **Processes** cliquer sur **View RTL Schematic** dans la partie Synthesize-XST.

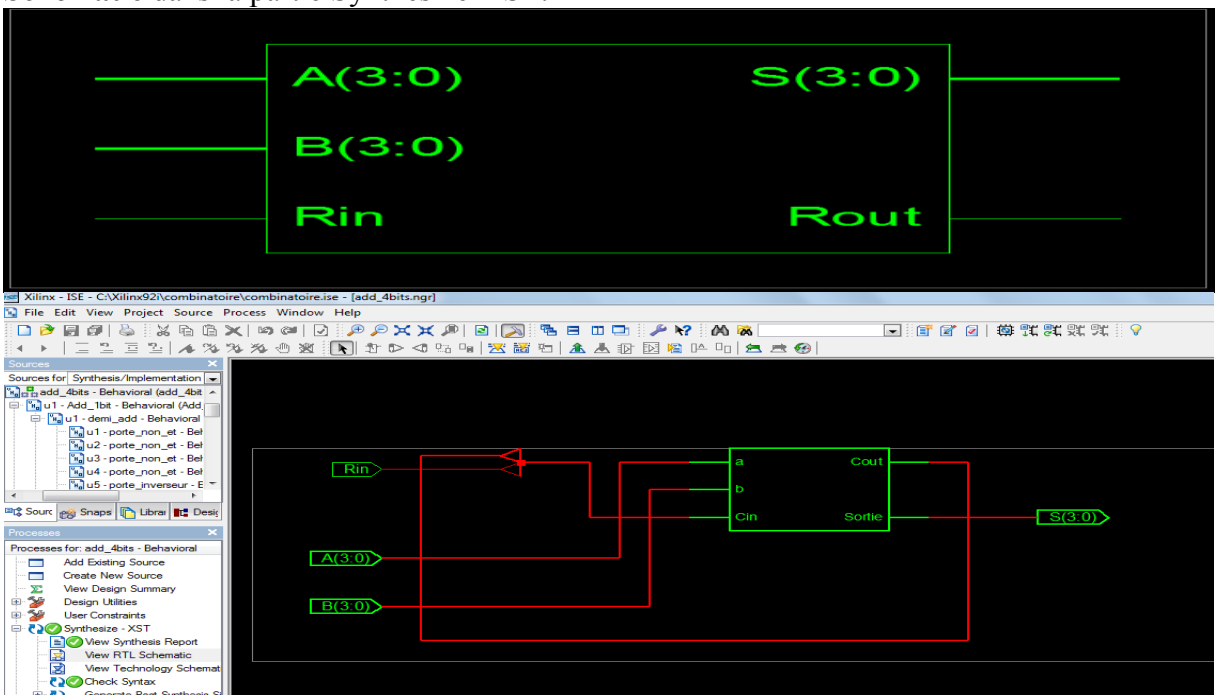


Figure 51 : le schéma d'add_4bits

Et enfin dans la rubrique **behavioral Simulation**, on simule notre porte on donnant des valeurs pour l'entrée **a,b** et **Rin** pour voir la sortie **S et Rout** :

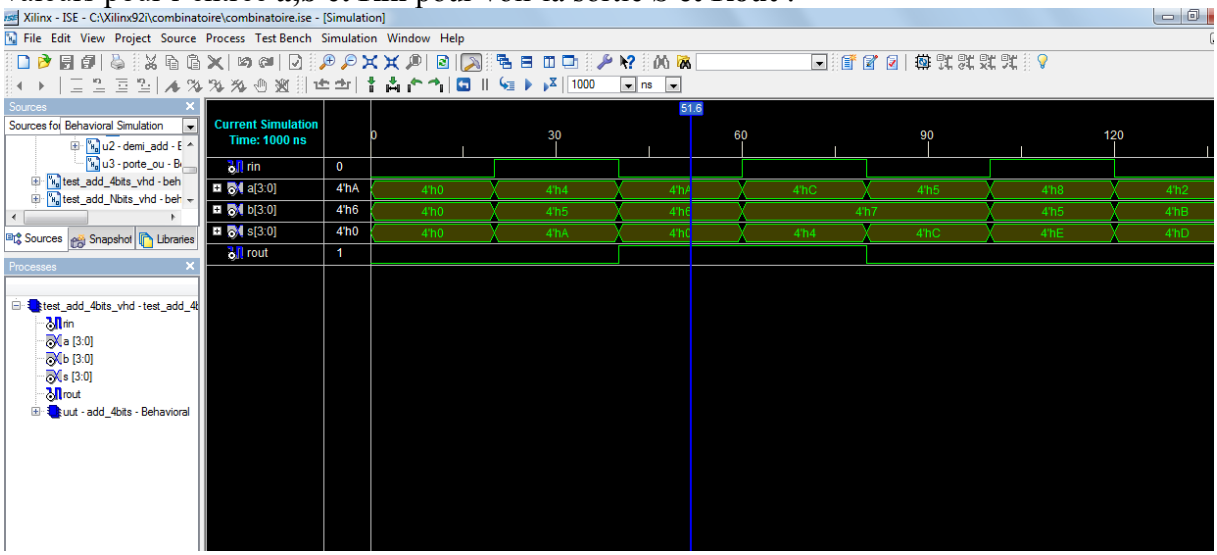


Figure 52 : la simulation d'add_4bits

Réaliser un additionneur complet de N bits en instanciant l'additionneur complet de 1bit. On doit déclarer le paramètre générique N dans l'entité et utiliser l'instruction GENERATE dans le programme principal de N bits.

✓ Additionneur complet Nbits: structurelle

Alors le programme du add_Nbits est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration du paquetage spécifique pour appeler les components.
use work.mes_composants.all;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity add_Nbits is
generic (n:natural:=8); -- (8 ou 16 ou AUTRES)
   Port ( a : in  STD_LOGIC_VECTOR (N-1 downto 0);
          b: in  STD_LOGIC_VECTOR (N-1  downto 0);
          Rin : in  STD_LOGIC;
          s : out  STD_LOGIC_VECTOR (N-1  downto 0);
          Rout : out  STD_LOGIC);
end add_Nbits;
-- Déclaration de l'architecture en structurelle.
architecture Behavioral of add_Nbits  is
signal t:std_logic_vector (n downto 0);
begin
u1:for i in 0 to n-1 generate
u2: add_1bit port map (a(i), b(i), t(i), s(i), t(i+1));
end generate;
t(0)<= Rin;
Rout <= t(n);
end Behavioral;
```

Le fichier Test du VHDL : **test_add_Nbits,** ICI N=8, la partie du **tb :process**

```
tb : process
begin
a<=x"FE"; b<=x"0D"; Rin<= '0'; wait for 20 ns;
a<=x"01"; b<=x"93"; Rin<= '1'; wait for 20 ns;
a<=x"A5"; b<=x"07"; Rin<= '0'; wait for 20 ns;
a<=x"C8"; b<=x"FF"; Rin<= '1'; wait for 20 ns;
wait;
end process;
end;
```

Et enfin dans la rubrique **behavioral Simulation**, on simule notre porte on donnant des valeurs pour l'entrée **a,b** et **Rin** pour voir la sortie **S** :

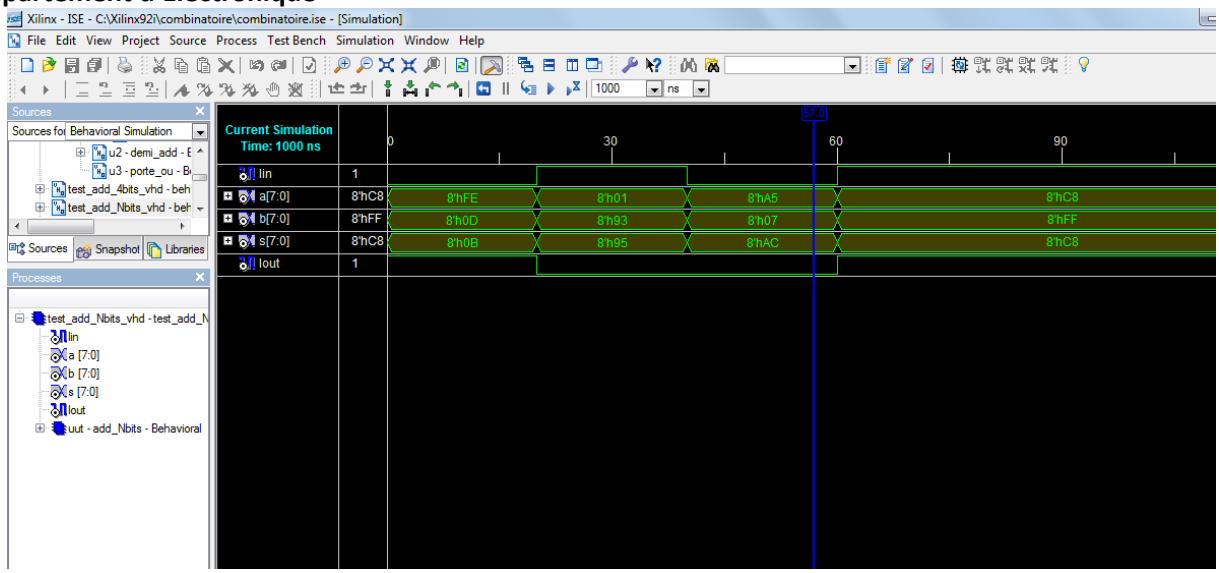


Figure 53 : la simulation d'add_Nbits (N=8)

VI. Bascule D

Réaliser la bascule D en utilisant un signal de remise à zéro synchrone active à l'état haut (Reset=1) et sensible au front montant de l'horloge Clk.

✓ Bascule D:

Alors le programme de la bascule_d est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity bascule_d is
    Port ( reset : in  STD_LOGIC;
           D: in  STD_LOGIC;
           Clk : in  STD_LOGIC;
           Q : out  STD_LOGIC;
           Qb : out  STD_LOGIC);
end bascule_d;
-- Déclaration de l'architecture avec une structure de test en séquentielle.
architecture Behavioral of bascule_d  is
begin
process (clk)
begin
if clk='1' and clk'event then
  if reset= '1' then Q<='0'; Qb<='1';
        else Q<= D; Qb<= not D;
end if;
end if;
end process;
end Behavioral;
```

Le fichier Test du VHDL : **test_bascule_d,** et le meme sauf pour la partie du **tb et tm process :**

```
tm: process
begin
```

```
clk<='0';wait for 10 ns;
clk<='1';wait for 10 ns;
end process;
tb : process
begin
reset<= '1'; D<= '0'; wait for 15 ns;
reset<= '1'; D<= '1'; wait for 15 ns;
reset<= '0'; D<= '0'; wait for 15 ns;
reset<= '0'; D<= '1'; wait for 15 ns;
wait;
end process;
end;
```

Et pour voir le schéma de la porte décrite, Donner la priorité à la **bascule_d** en cliquons dessus, bouton droit (**set as Top module**) ensuite dans la rubrique **Processes** cliquer sur **View RTL Schematic** dans la partie Synthesize-XST.

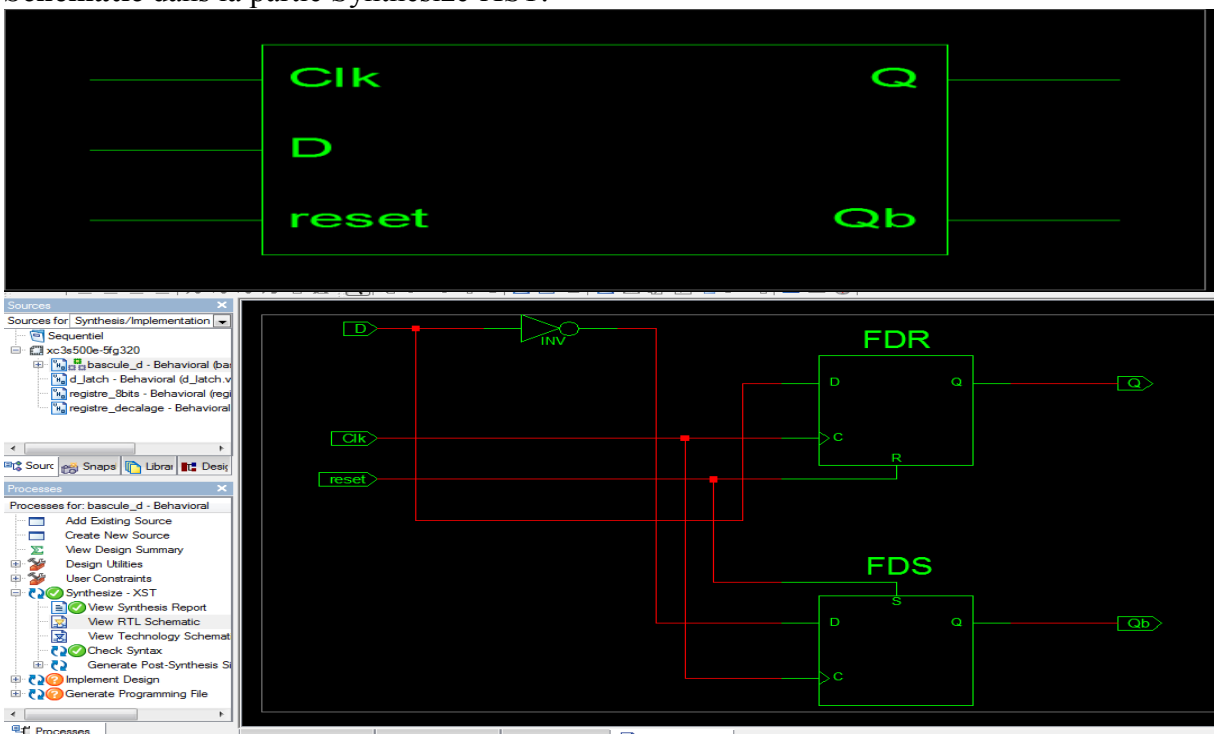


Figure 54 : le schéma de la bascule d

Et enfin dans la rubrique **behavioral Simulation**, on simule notre porte on donnant des valeurs pour l'entrée **reset, d** et **Clk** pour voir la sortie **q et qb** :

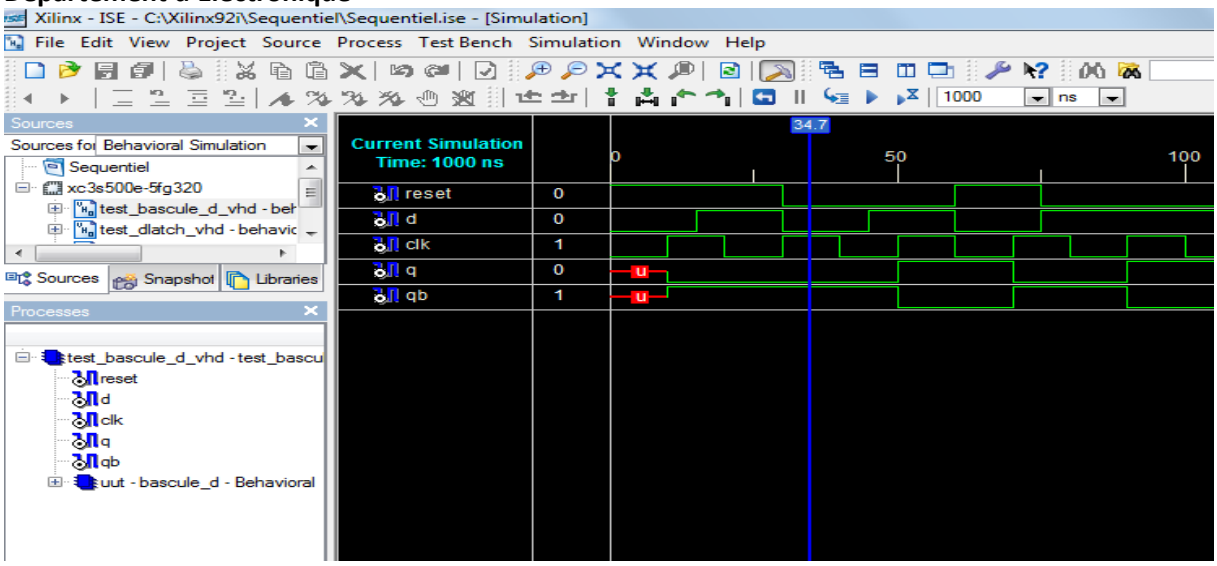


Figure 55 : la simulation de la bascule d

VII. REGISTRE

Réaliser un registre de 8 bits qui est constitué de bascules D avec un signal Reset asynchrone actif à l'état haut, le circuit sera sensible au front montant de l'horloge Clk.

✓ Registre à 8 bits:

Alors le programme d'un registre_8bits est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity registre_8bits is
   Port ( reset : in  STD_LOGIC;
          D: in  STD_LOGIC _VECTOR (7 downto 0);
          Clk : in  STD_LOGIC;
          Q : out  STD_LOGIC _VECTOR (7 downto 0);
          Qb : out  STD_LOGIC _VECTOR (7 downto 0));
end registre_8bits;
-- Déclaration de l'architecture avec une structure de test en séquentielle.
architecture Behavioral of registre_8bits  is
signal s,sb:std_logic_vector(7 downto 0);
begin
process (clk,reset,d)
begin

if reset= '1' then s<="00000000"; sb<="11111111";
else if clk='1' and clk'event then s<=d; sb<=not d;
else s<= s; sb<= sb;
end if;
end if;
end process;
q<=s; qb<=sb;
end Behavioral;
```

Le fichier Test du VHDL : **test_ registre_8bits,** et le même sauf pour la partie du **tb et tm process :**

```
tm: process
begin
clk<='0';wait for 10 ns;
clk<='1';wait for 10 ns;
end process;
tb : process
begin
reset<= '1'; D<= "01001001"; wait for 15 ns;
reset<= '1'; D<= "11111000"; wait for 15 ns;
reset<= '1'; D<= "00010101"; wait for 15 ns;
reset<= '0'; D<= "11001111"; wait for 15 ns;
reset<= '0'; D<= "00000111"; wait for 15 ns;
reset<= '0'; D<= "10101010"; wait for 15 ns;
wait;
end process;
end;
```

Et pour voir le schéma de la porte décrite, Donner la priorité au **registre_8bits** en cliquons dessus, bouton droit (**set as Top module**) ensuite dans la rubrique **Processes** cliquer sur **View RTL Schematic** dans la partie Synthesize-XST.

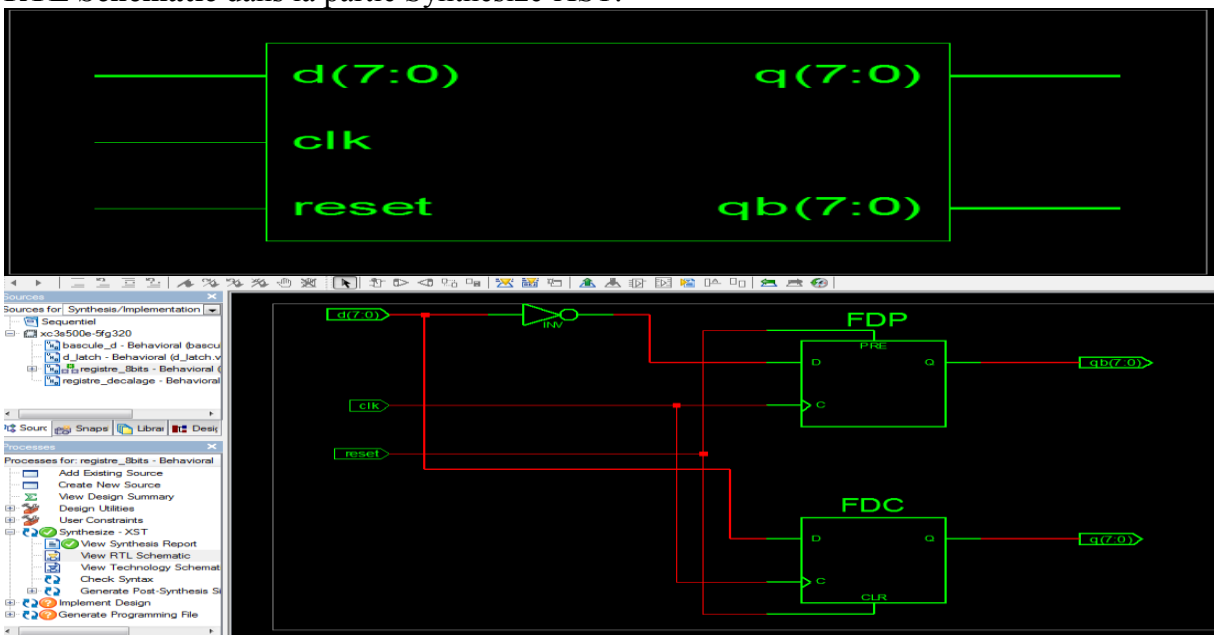


Figure 56 : le schéma du registre à 8bits

Et enfin dans la rubrique **behavioral Simulation**, on simule notre porte on donnant des valeurs pour l'entrée **reset, d** et **Clk** pour voir la sortie **q et qb** :

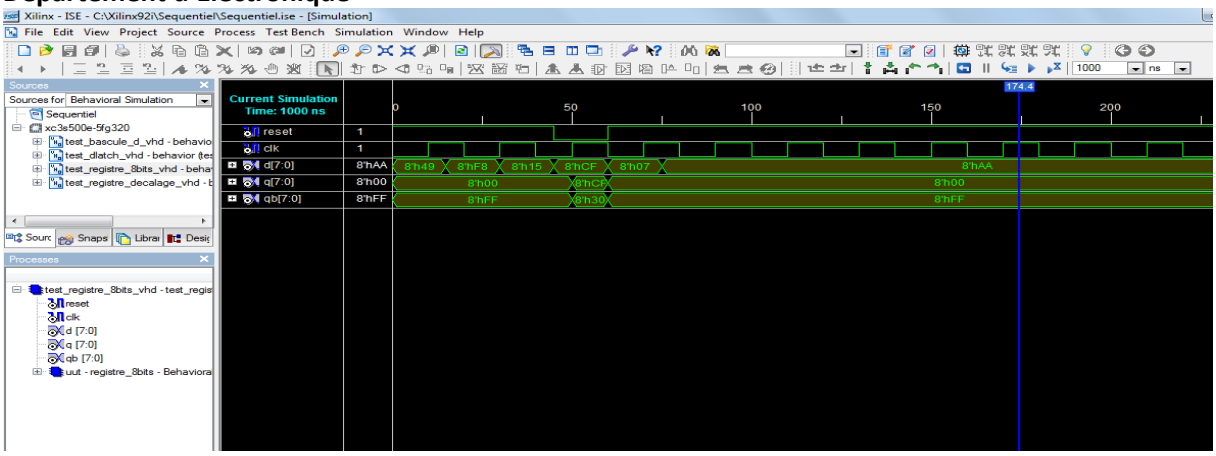


Figure 57 : la simulation du registre à 8bits

Réaliser un registre de N bits qui est constitué de bascules D avec un signal Reset asynchrone actif à l'état haut, le circuit sera sensible au front montant de l'horloge Clk.

✓ **Registre à N bits:**

Alors le programme d'un registre_Nbits est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity registre_Nbits is
   generic (n: natural:=16);
   Port ( reset : in  STD_LOGIC;
          D: in  STD_LOGIC _VECTOR (N-1 downto 0);
          Clk : in  STD_LOGIC;
          Q : out  STD_LOGIC _VECTOR (N-1 downto 0);
          Qb : out  STD_LOGIC _VECTOR (N-1 downto 0));
end registre_Nbits;
-- Déclaration de l'architecture avec une structure de test en séquentielle.
architecture Behavioral of registre_Nbits  is
signal s,sb:std_logic_vector(n-1 downto 0);
begin
process (clk,reset,d)
begin
if reset= '1' then s<=(others=>'0'); sb<=(others=>'1');
else if clk='1' and clk'event then s<=d; sb<=not d;
else s<= s; sb<= sb;
end if;
end if;
end process;
q<=s; qb<=sb;
end Behavioral;
```

Le fichier Test du VHDL : **test_ registre_Nbits,** et le même sauf pour la partie du **tb et tm process :**

```
tm: process
begin
clk<='0';wait for 10 ns;
```

```
clk<='1';wait for 10 ns;
end process;
tb : process
begin
reset<= '0'; D<= x"FF1E"; wait for 15 ns;
reset<= '0'; D<= x"2C35"; wait for 15 ns;
reset<= '1'; D<= x"1D6E"; wait for 15 ns;
reset<= '1'; D<= x"1123"; wait for 15 ns;
reset<= '0'; D<= x"097A"; wait for 15 ns;
reset<= '1'; D<= x"DE5A"; wait for 15 ns;
wait;
end process;
end;
```

Et pour voir le schéma de la porte décrite, Donner la priorité au **registre_Nbits** en cliquons dessus, bouton droit (**set as Top module**) ensuite dans la rubrique **Processes** cliquer sur **View RTL Schematic** dans la partie Synthesize-XST.

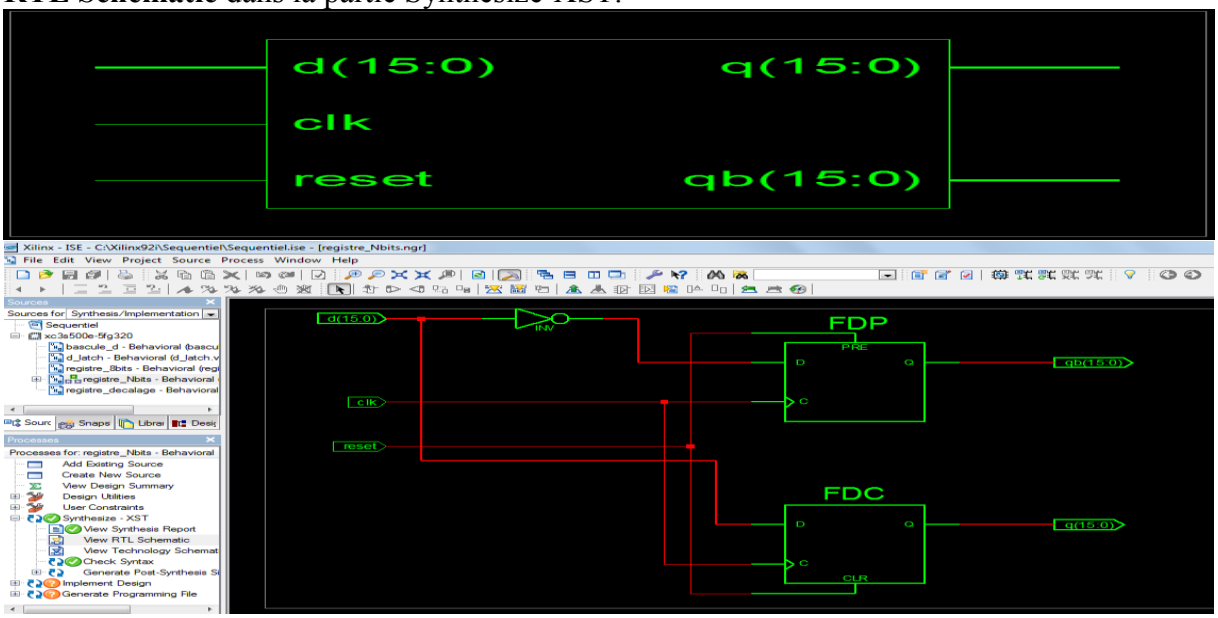


Figure 58 : le schéma du registre à Nbits (N=16)

Et enfin dans la rubrique **behavioral Simulation**, on simule notre porte on donnant des valeurs pour l'entrée **reset, d** et **Clk** pour voir la sortie **q et qb** :

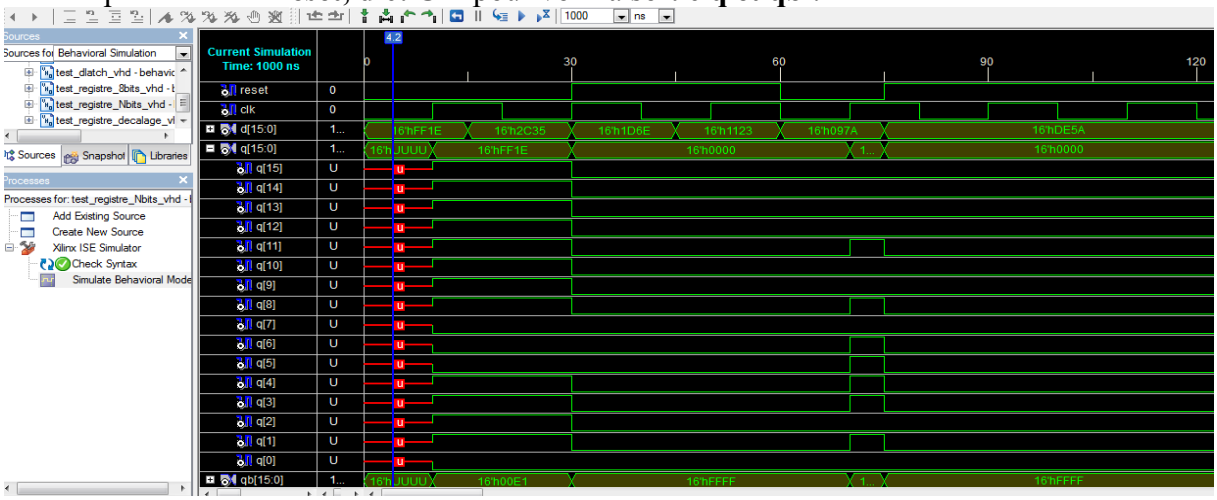


Figure 59 : la simulation du registre à Nbits (N=16)

VIII. Compteur/Décompteur

Réaliser un circuit compteur/décompteur de N bits (N est un paramètre générique) avec une remise à zéro asynchrone active à l'état haut. Le circuit sera sensible au front montant de l'horloge Clk, il fonctionnera comme compteur à l'état haut de Ud et comme décompteur à l'état bas de Ud.

✓ **Compteur/ décompteur à N bits:**

Alors le programme d'un compteur_decompteur_Nbits est comme suite :

```
-- Déclaration de la bibliothèque et des paquetages.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
-- Déclaration de l'entité avec les entrées/sorties correspondants.
entity compteur_decompteur_Nbits is
   generic (n: natural:=8);
   Port ( reset : in  STD_LOGIC;
          Ud: in  STD_LOGIC;
          Clk : in  STD_LOGIC;
          Q : out  STD_LOGIC _VECTOR (N-1 downto 0));
end compteur_decompteur_Nbits;
-- Déclaration de l'architecture avec une structure de test en séquentielle.
architecture Behavioral of compteur_decompteur_Nbits  is
signal s:std_logic_vector(n-1 downto 0);
begin
process (clk,reset,Ud)
begin
if reset= '1' then s<=(others=>'0');
else if clk='1' and clk'event then
if ud = '1' then S <= S+1;
else S <= S-1;
end if; end if; end if;
end process;
q<=s;
end Behavioral;
```

Le fichier Test du VHDL : **test_ compteur_decompteur_Nbits,** et le même sauf pour la partie du **tb et tm process :**

```
tm: process
begin
clk<='0';wait for 10 ns;
clk<='1';wait for 10 ns;
end process;
tb : process
begin
reset<= '0'; ud<= '1'; wait for 15 ns;
reset<= '1'; ud<= '1'; wait for 15 ns;
reset<= '0'; ud<= '1'; wait for 15 ns;
reset<= '1'; ud<= '0'; wait for 15 ns;
reset<= '0'; ud<= '0'; wait for 15 ns;
reset<= '1'; ud<= '1'; wait for 15 ns;
wait;
```

```
end process;
end;
```

Et pour voir le schéma de la porte décrite, Donner la priorité au **compteur_decompteur_Nbits** en cliquons dessus, bouton droit (**set as Top module**) ensuite dans la rubrique **Processes** cliquer sur **View RTL Schematic** dans la partie Synthesize-XST.

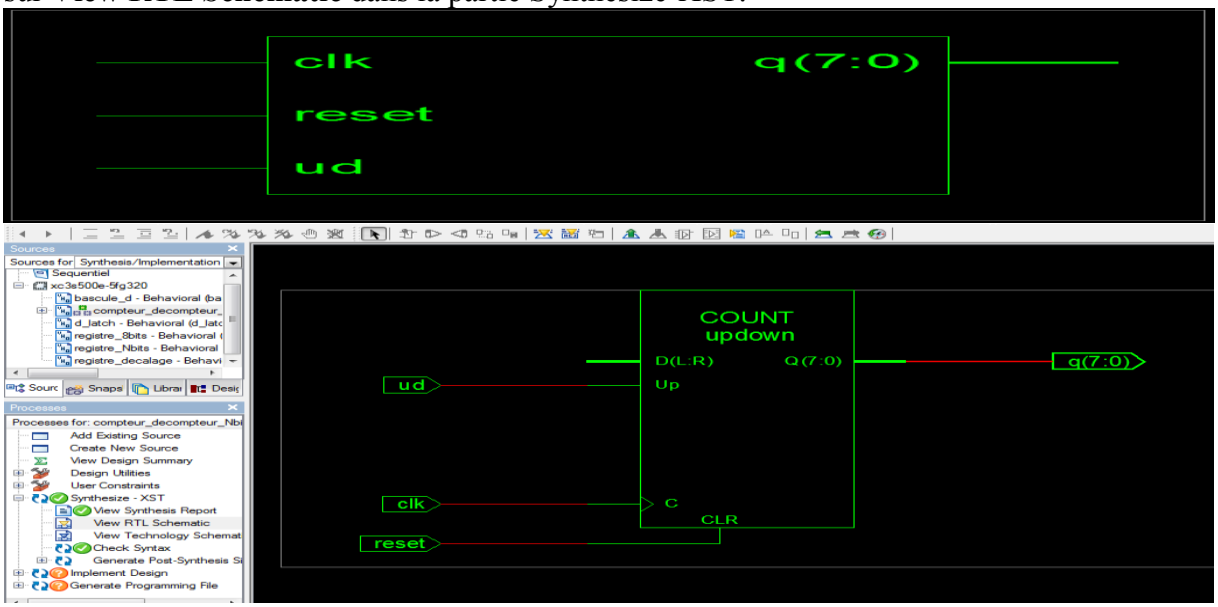


Figure 60 : le schéma du compteur_decompteur_Nbits (N=8)

Et enfin dans la rubrique **behavioral Simulation**, on simule notre porte on donnant des valeurs pour l'entrée **reset, d** et **Clk** pour voir la sortie **q**:

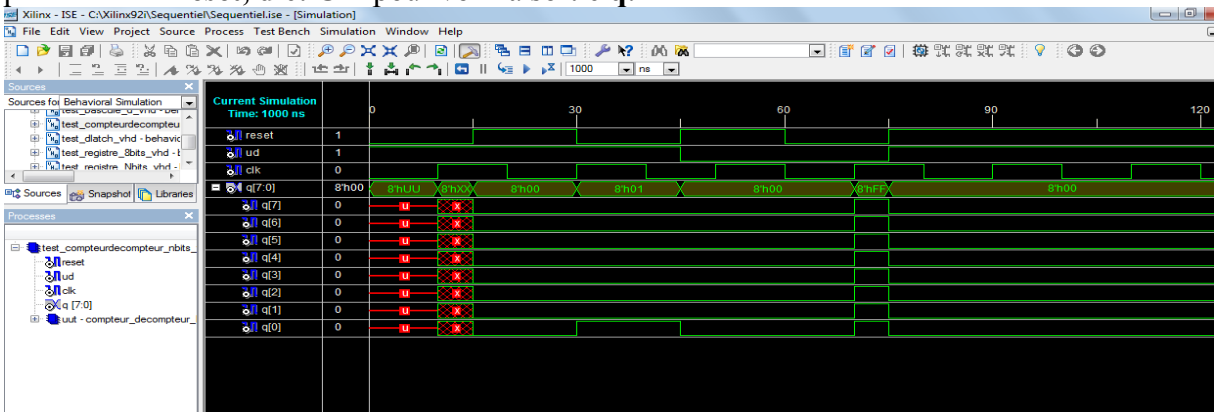


Figure 61 : la simulation du compteur_decompteur_Nbits (N=8)

Série TD n°1 (Circuits combinatoires)

Exercice 1

On donne la description VHDL suivante :

```
Library ieee ;
Use ieee.std_logic_1164.all;
Use work.std_arith.all;

Entity Portes is
Port (A, B: in std_logic;
      Y1, Y2, Y3, Y4, Y5, Y6, Y7: out std_logic);
end Portes ;

Architecture Arch_Portes of Portes is
begin
Y1<= A and B;
Y2<= A or B;
Y3<= A xor B;
Y4<= A not B;
Y5<= A nand B;
Y6<= A nor B;
Y7<= not (A xor B);
end Arch_Portes;
```

1) **A partir du fichier donner les noms des bibliothèques utilisées.**
2) **Quel est le nom de l'entité ? Quel est le nom de l'architecture ?**
3) **Représenter le schéma fonctionnel de la fonction. On placera naturellement les entrées à gauche et les sorties à droites.**

Exercice 2

B	A	S
0	0	1
0	1	1
1	0	1
1	1	0

1) **Faire la description VHDL de ce circuit ?**
2) **De quel circuit s'agit-il ?**
3) **Dessiner le schéma bloc de ce circuit ?**

Exercice 3

Entrées				Sorties	
A3	A2	A1	A0	S1	S0
0	1	0	1	1	1
0	1	1	0	0	1
1	1	0	1	1	0

1) **Faire la description VHDL de ce circuit ?**

On peut passer de std_logic vers std_logic_vector en présentant les entrées et les sorties comme des vecteurs respectivement.

2) **Ecrire une description VHDL qui utilise les entrées et les sorties en vecteurs dans une entité et plusieurs architectures qui contiennent différentes descriptions :**
   - ✓ **Affectation sélective (with… select… when) ?**
   - ✓ **Affectation conditionnelle (when… else) ?**
   - ✓ **Structure de test (if… then… else)?**
   - ✓ **Structure de choix (case… is… when) ?**

Exercice 4

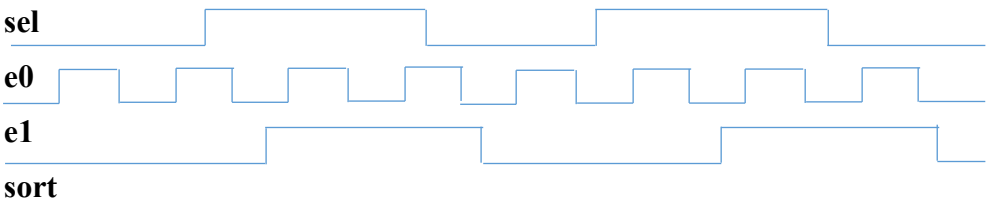
Soit la description VHDL suivante :

```
Library ieee ;
Use ieee.std_logic_1164.all;

Entity Fonction_mystere is
Port (e0, e1: in std_logic;
          sel: in std_logic;
         Sort: out std_logic);
end Fonction_mystere ;

Architecture V0 of Fonction_mystere is
begin
Sort <= e0 when sel= '0' else e1;
end V0;
```

1) **Quelle est la fonction réalisée par cette architecture ?**
2) **Réécrire la même fonction avec la structure de test ?**
3) **Pour cette architecture, recopier le chronogramme suivant et complétez le avec la trace du signal « sort » :**



Solution de la série TD n°1 (Circuits combinatoires)

Exercice 1

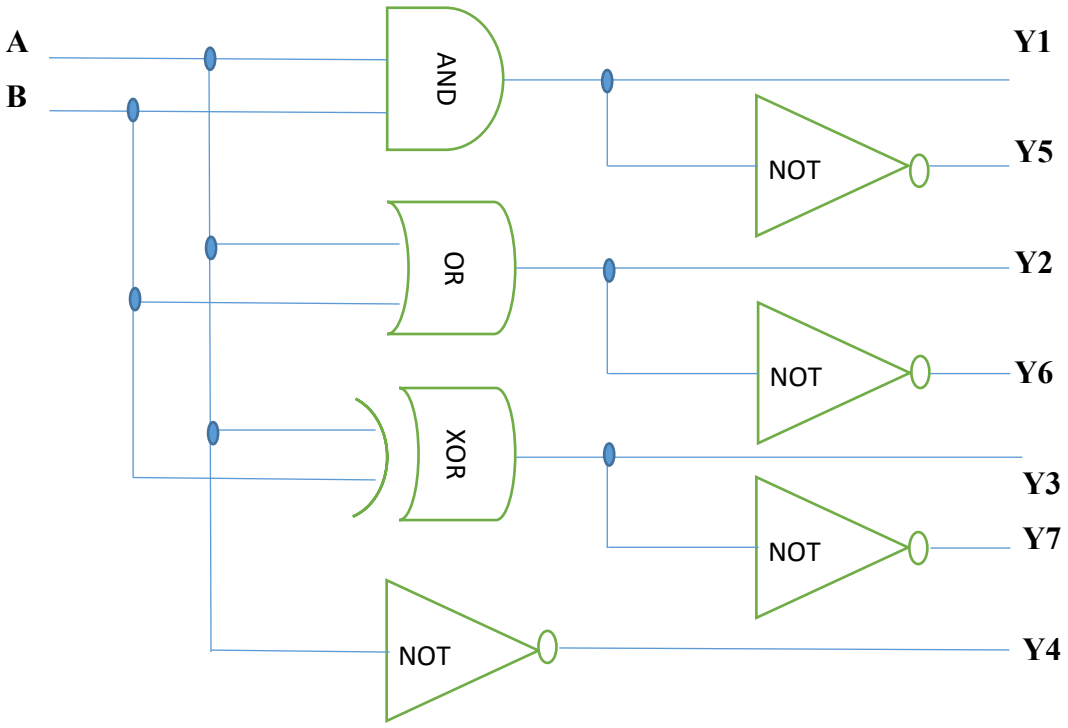
1) **les noms des bibliothèques utilisées.**

- ✓ La bibliothèque IEEE du standard VHDL,
- ✓ Le paquetage standard pour ieee.Std_logic_1164 pour typé les entrées/ sorties et les signaux internes,
- ✓ Le paquetage personnalisé par l'utilisateur pour work.std_arith pour spécifié les fonctions utilisées dans le programme VHDL.

2) **Quel est le nom de l'entité ?** Portes

Quel est le nom de l'architecture ? Arch_Portes

3) **Représenter le schéma fonctionnel de la fonction.**



Exercice 2

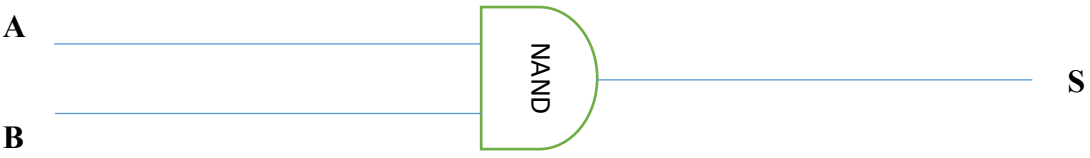
1) **Faire la description VHDL de ce circuit ?**

```
Library ieee ;
Use ieee.std_logic_1164.all ;
Use ieee.std_logic_arith.all ;
Entity Ex_2 is
Port (A, B: in std_logic;
      S: out std_logic);
end Ex_2 ;
```

```
Architecture arch_ex2 of Ex_2 is
begin
S <= '0' when A= '1' and B='1' else S <= '1';
end arch_ex2;
```

2) De quel circuit s'agit-il ? **Il s'agit du circuit NAND**

3) Le schéma bloc de ce circuit :



Exercice 3

1) Faire la description VHDL de ce circuit ?

```
Library ieee ;
Use ieee.std_logic_1164.all ;
Use ieee.std_logic_arith.all ;
Entity Ex_3 is
Port (A0, A1, A2, A3: in std_logic;
          S0, S1: out std_logic);
end Ex_3 ;
Architecture arch_ex3 of Ex_3 is
begin
    process (A0, A1, A2, A3)
    begin
    if (A0='1' and A0='0' and A0='1' and A0='0') then S0<= '1';S1<='1' ;
     else if (A0='0' and A0='1' and A0='1' and A0='0') then S0<= '1';S1<='0' ;
       else if (A0='1' and A0='0' and A0='1' and A0='1') then S0<= '0';S1<='1' ;
       end if;
     end if;
    end if;
   end process;
end arch_ex2;
```

2) Ecrire une description VHDL qui utilise les entrées et les sorties en vecteurs dans une entité et plusieurs architectures qui contiennent différentes descriptions :

- ✓ Affectation sélective (with… select… when) ?
- ✓ Affectation conditionnelle (when… else) ?
- ✓ Structure de test (if… then… else)?
- ✓ Structure de choix (case… is… when) ?

```
Library ieee ;
Use ieee.std_logic_1164.all ;
Use ieee.std_logic_arith.all ;
Entity vecteur is
Port (A: in std_logic_vector (3 downto 0);
        S: out std_logic_vector (1 downto 0));
end vecteur;
```

- ✓ **Architecture** with_select **of** vecteur **is**
begin
with A **select**
S<= "11" **when** "0101",
"01" **when** "0110",
"10" **when** "1101",
'0' **when others**;
end with_select**;**

- ✓ **Architecture** when_else **of** vecteur **is**
begin

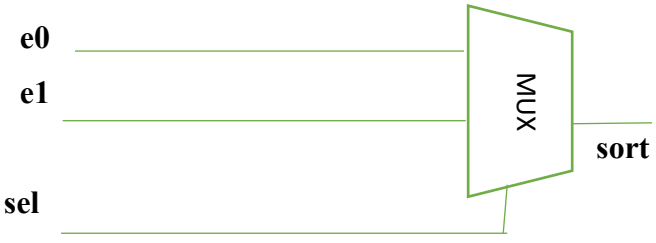
S<= "11" **when** A="0101" **else**
"01" **when** A="0110" **else**
"10" **when** A="1101" **else**
'0' **when others**;
end when_else**;**

- ✓ **Architecture** if_then **of** vecteur **is**
begin
process (A)
begin
if A="0101" **then** S<= "11" ;
else if A="0110" then S<= "01" ;
else if A="1101" then S<= "10" ;
end if;
end if;
end if;
end process;
end if_then**;**

- ✓ **Architecture** case_is **of** vecteur **is**
begin
process (A)
begin
case A **is**
when "0101" **=>** S<= "11" ;
when "0110" => S<= "01" ;
when "1101" => S<= "10" ;
end process;
end case_is**;**

Exercice 4

1) **Quelle est la fonction réalisée par cette architecture ?**



Sel	Sort
0	**E0**
1	**E1**

E1	E0	sel	sort
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Donc c'est un Multiplexeur à 2 entrées, une entrée de sélection et une sortie.

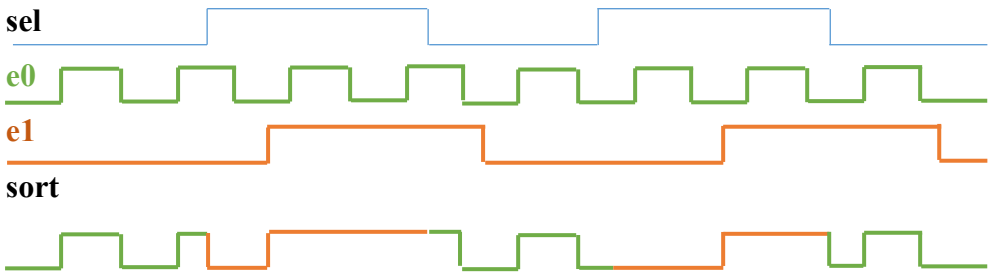
2) Réécrire la même fonction avec la structure de test ?

```
Library ieee ;
Use ieee.std_logic_1164.all;

Entity Fonction_mystere is
Port (e0, e1: in std_logic;
          sel: in std_logic;
         Sort: out std_logic);
end Fonction_mystere ;

Architecture structure_de_test of Fonction_mystere is
begin
process (e0, e1, sel)
begin
if sel= '0' then Sort <= e0; else e1;
end structure_de_test;
```

3) complétez le chronogramme suivant avec la trace du signal « sort » :



Série TD n°2 (Circuits combinatoires)

Exercice 1

Soit la description VHDL suivante :

```
Library ieee ;
Use ieee.std_logic_1164.all;

Entity fct is
Port (a, b: in std_logic;
          s: out std_logic);
end fct ;

Architecture archi_fct of fct is
begin
s<=  '1' when a=b else '0';
end archi_fct;
```

1) **Que réalise cette fonction ?**
2) **Donner le schéma de cette fonction ?**
3) **Donner une affectation sélective à cette architecture ?**

Dans le même projet, on écrit la description VHDL suivante :

```
Library ieee ;
Use ieee.std_logic_1164.all;

Entity circuit is
Port (e1, e2, e3, e4: in std_logic;
          s: out std_logic);
end circuit ;

Architecture archi_circuit of circuit is
Signal s1, s2, s3, s4: std_logic;
Componenet fct
Port (a, b: in std_logic;
          s: out std_logic);
end componenet ;
begin
cmp1: fct port map (e1, e2, s1);
cmp2: fct port map (e3, e4, s2);
cmp3: fct port map (e1, e3, s3);
end archi_circuit;
```

4) **Faite une analyse structurelle du circuit réalisé ?**
5) **Donner le schéma de ce circuit en utilisant les portes décrites dans le programme ?**

Exercice 2

Soit la description VHDL suivante :

```
Library ieee ;
Use ieee.std_logic_1164.all;
Use iee.std_logic_arith.all;

Entity fct1 is
Port (A, B: in std_logic;
      S1, S2, S3, S4: out std_logic);
end fct1 ;

Architecture Archi_fct1 of fct1 is
begin
S1<= '1' when (a= '0' and b= '0') else '0';
S2<= '1' when (a= '1' and b= '0') else '0';
S3<= '1' when (a= '0' and b= '1') else '0';
S4<= '1' when (a= '1' and b= '1') else '0';
end Archi_fct1;
```

1) **Déduire de ce programme le schéma bloc du circuit ?**
2) **De quel circuit s'agit-il ?**
3) **La structure électronique dans cette description est-elle combinatoire ou séquentielle ?**
4) **Ecrire cette même fonction avec une affectation sélective ?**

Exercice 3

Voici le schéma bloc d'un démultiplexeur 1à 4, une entrée de validation Enable (le circuit est actif à l'état haut de ce signal), une entrée de sélection E et 4 sorties.

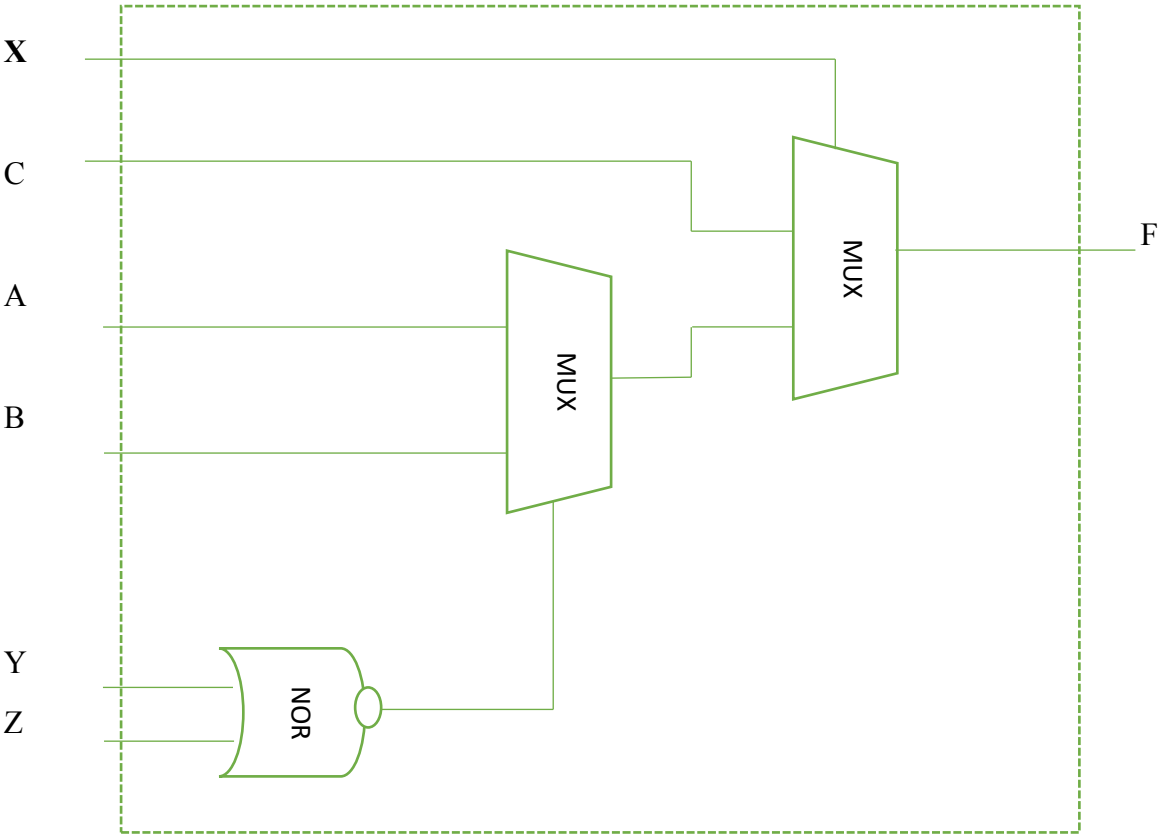


Ecrire une description VHDL de ce circuit en utilisant :

1) **Instruction de choix ? de quelle description s'agit-il ?**
2) **Affectation sélective ? de quelle description s'agit-il ?**

Exercice 4

Ecrire une description VHDL du schéma fonctionnel suivant, de quelle description avez-vous besoin ?



Solution de la série TD n°2 (Circuits combinatoires)

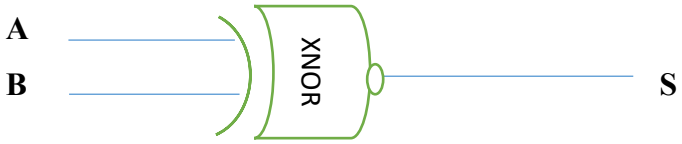
Exercice 1

1) **Que réalise cette fonction ?**

B	A	S
0	0	1
0	1	0
1	0	0
1	1	1

Donc cette fonction est un circuit XNOR

2) **Donner le schéma de cette fonction ?**



3) **Donner une affectation sélective à cette architecture ?**

```
Library ieee ;
Use ieee.std_logic_1164.all;

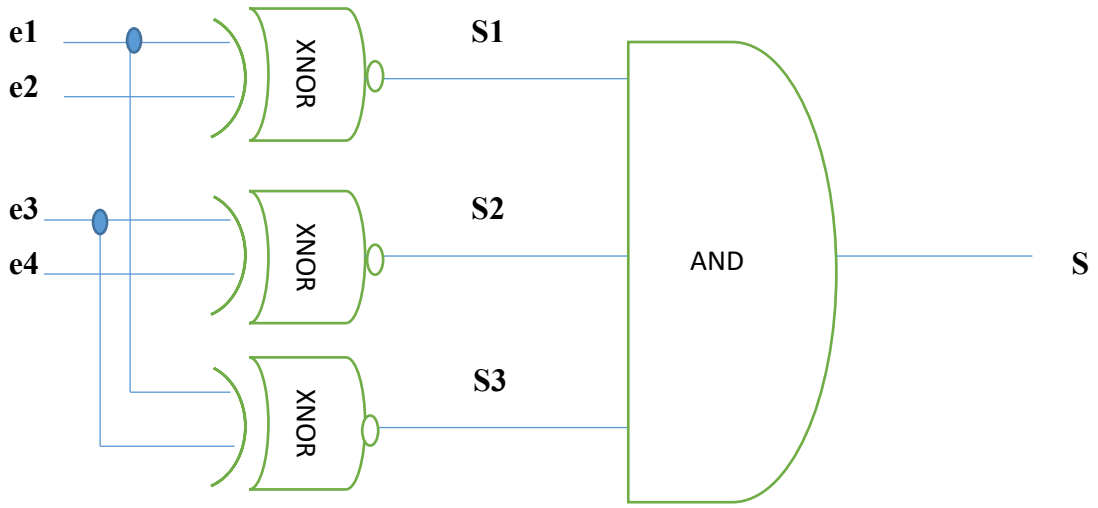
Entity fct_2 is
Port (a, b: in std_logic;
        s: out std_logic);
end fct_2 ;

Architecture archi_fct2 of fct_2 is
Signal ab: std_logic_vector (1 downto 0);
begin
ab <= a & b;
with ab select
      S<= '1' when "00",
          '0' when "01",
          '0' when "10",
          '1' when "11",
          '0' when others;
end archi_fct2;
```

4) **L'analyse structurelle du circuit réalisé**

- ✓ Dans cette description nous avons une bibliothèque et un paquetage standard ieee,
- ✓ Une entité « circuit » avec 4 entrées et une sortie
- ✓ Une architecture « archi_circuit » avec 3 signaux internes, un composant « fct » déjà décrit dans la première partie de l'exo 1 et trois étiquettes qui font appelle à la fonction xnor « fct » et enfin une affectation simple à trois entrées S1, S2 et S3.

5) **Le schéma de ce circuit en utilisant les portes décrites dans le programme**



Exercice 2

1) **Le schéma bloc du circuit**

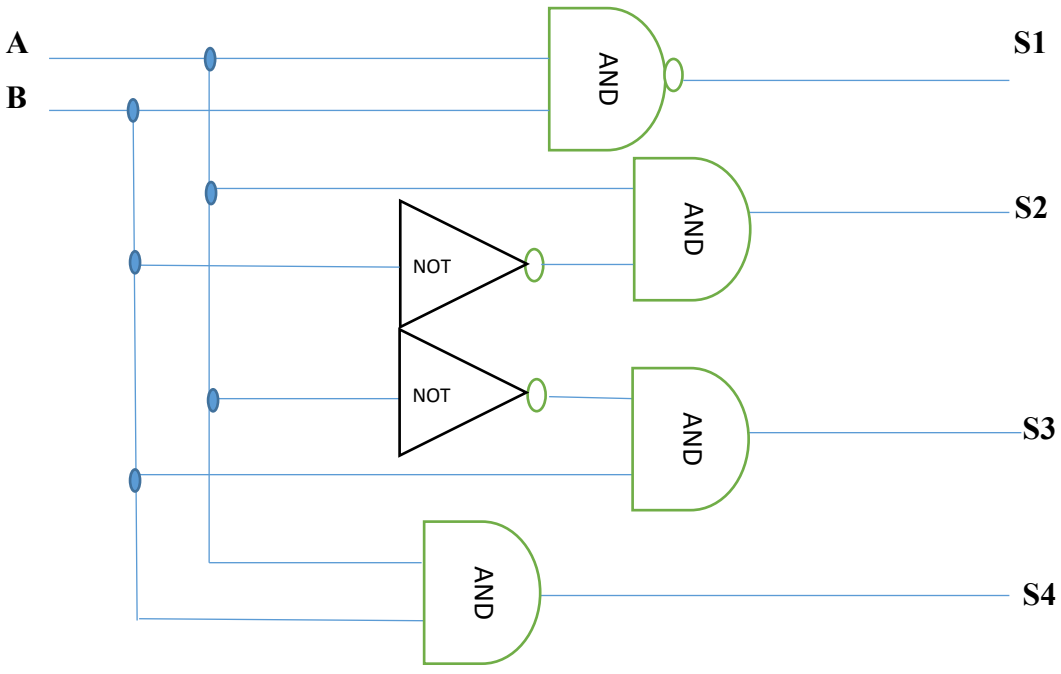
$S1 = \overline{A} \cdot B$

$S2 = A \cdot \overline{B}$

$S3 = \overline{A} \cdot B$

$S4 = A \cdot B$

B	A	S4	S3	S2	S1
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0



2) **De quel circuit s'agit-il ?**

Il s'agit d'un Décodeur 2×4 à 2 entrées et 4 sorties



3) **La structure électronique dans cette description est-elle combinatoire ou séquentielle ?**

La structure électronique est combinatoire car le Décodeur ne dépond que de l'état actuel des entrées et non pas de ses états précédents ou d'une horloge.

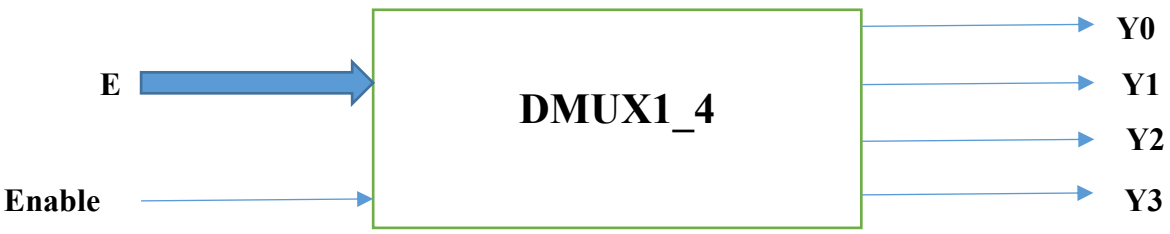
4) **Ecrire cette même fonction avec une affectation sélective ?**

```
Library ieee ;
Use ieee.std_logic_1164.all;
Use iee.std_logic_arith.all;

Entity fct2 is
Port (C: in std_logic_vector (1 downto 0);
        S: out std_logic_vector (3 downto 0));
end fct2 ;

Architecture Archi_fct2 of fct2 is
begin
with C select
S<= "0001" when "00",
      "0010" when "01",
      "0100" when "10",
      "1000" when "11",
        '0'    when others;
end Archi_fct2;
```

Exercice 3



Enable	E1	E0	Y3	Y2	Y1	Y0
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

Ecrire une description VHDL de ce circuit en utilisant :

1) **Instruction de choix ?**

```
Library ieee ;
Use ieee.std_logic_1164.all;
Use iee.std_logic_arith.all;

Entity EX3_1 is
Port (Enable: in std_logic;
            E: in std_logic_vector (1 downto 0);
            Y: out std_logic_vector (3 downto 0));
end EX3_1 ;

Architecture case_is of EX3_1 is
begin
   process (Enable, E)
   begin
   if Enable='0' then Y<= "0000" ;
    else case E is
   when "00" => S<= "0001" ;
   when "01" => S<= "0010" ;
   when "10" => S<= "0100" ;
   when "11" => S<= "1000" ;
   end if;
   end process;
end case_is;
```

✓ **De quelle description s'agit-il ? c'est une description comportementale avec l'utilisation d'une instruction séquentielle.**

2) **Affectation conditionnelle ?**

```
Library ieee ;
Use ieee.std_logic_1164.all;
Use iee.std_logic_arith.all;

Entity EX3_2 is
Port (Enable: in std_logic;
            E: in std_logic_vector (1 downto 0);
            Y: out std_logic_vector (3 downto 0));
end EX3_2 ;
```

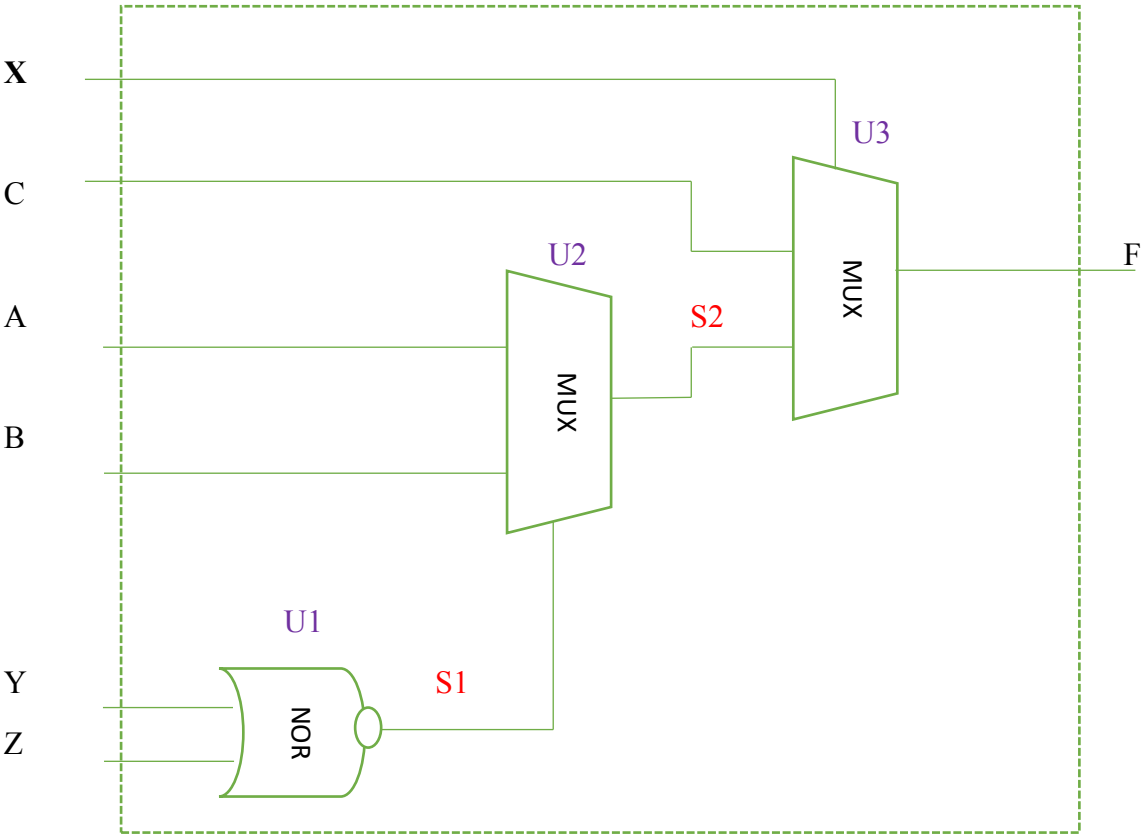
```
Architecture when_else of EX3_2 is
begin

Y<= "0000" when Enable='0' else
Y<= "0000" when Enable='1' and E= "00" else
Y<= "0000" when Enable='1' and E= "01" else
Y<= "0000" when Enable='1' and E= "10" else
Y<= "0000" when Enable='1' and E= "11";
end when_else;
```

✓ **De quelle description s'agit-il ? c'est une description flot de données avec l'utilisation d'une instruction concurrente.**

Exercice 4

De quelle description avez-vous besoin ? Nous avons besoin d'une description structurelle avec deux programmes en parallèle celui de la porte NOR et celui de la porte MUX.



✓ Porte NOR :

```
Library ieee ;
Use ieee.std_logic_1164.all ;
Use ieee.std_logic_arith.all ;
```

```
Entity porte_nor is
Port (a, b: in std_logic;
          s: out std_logic);
end porte_nor ;

Architecture arch_nor of porte_nor is
begin
s <= a NOR b;
end arch_nor;
```

✓ Porte MUX:

```
Library ieee ;
Use ieee.std_logic_1164.all ;
Use ieee.std_logic_arith.all ;
Entity porte_mux is
Port (a, b, sel: in std_logic;
          s: out std_logic);
end porte_mux ;

Architecture arch_mux of porte_mux is
begin
s <= a when sel= '0' else b;
end arch_mux;
```

✓ **La description structurelle :**

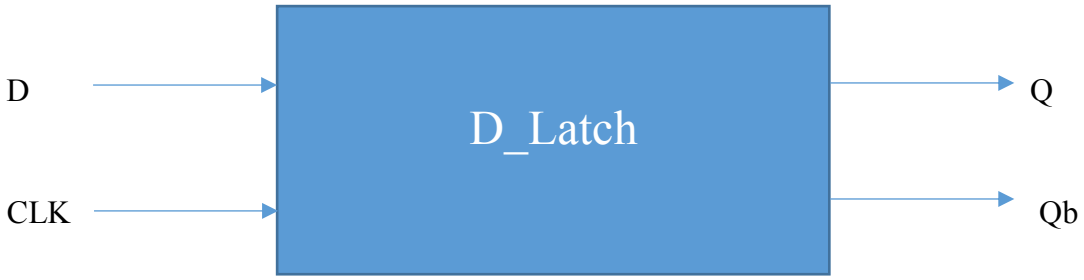
```
Library ieee ;
Use ieee.std_logic_1164.all ;
Use ieee.std_logic_arith.all ;
Entity structurelle is
Port (A, B, C, X, Y, Z: in std_logic;
                    F: out std_logic);
end structurelle ;

Architecture arch_struc of structurelle is
Signal S1, S2 : std_logic;
Component porte_nor
Port (a, b : in std_logic;
          s: out std_logic);
end component ;
Component porte_mux
Port (a, b, sel: in std_logic;
          s: out std_logic);
end component ;
begin
U1: porte_nor port map (Y, Z, S1);
U2: porte_mux port map (A, B, S1, S2);
U3: porte_mux port map (C, S2, X, F);
end arch_struc;
```

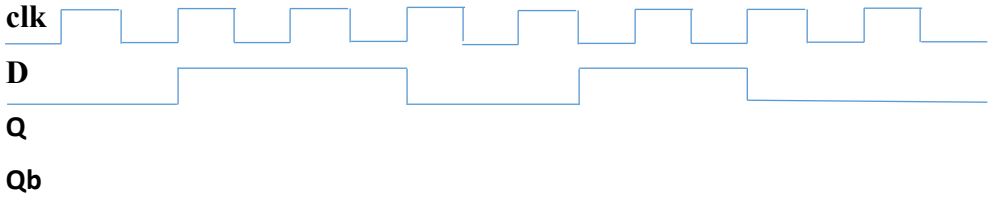
Série TD n°3 (Circuits séquentiels)

Exercices 1

1) **Ecrire une description VHDL selon le principe d'une bascule D latch**

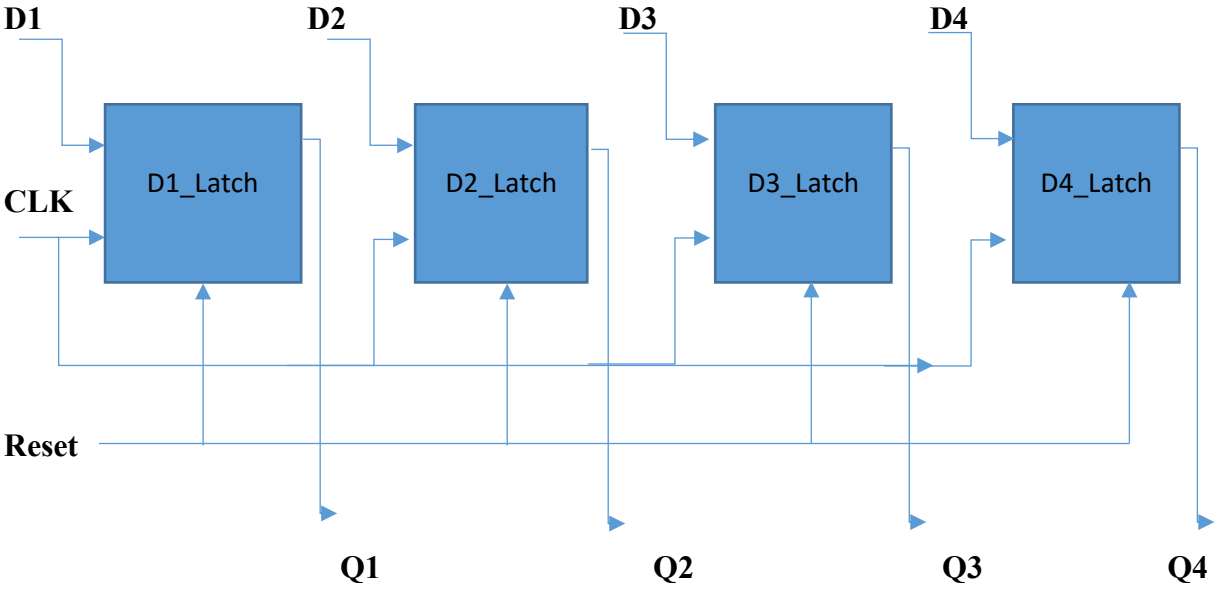


2) **Compléter le chronogramme suivant en se basant sur les données de D et CLK**



Exercice 2

Réaliser un registre composé de 4 bascules D latch.



Exercice 3

Réaliser un registre à décalage à droite de 4bits avec un reset asynchrone :

1) **Description simple.**
2) **Description en utilisant une boucle.**

Solution de la série TD n°3 (Circuits séquentiels)

Exercice 1

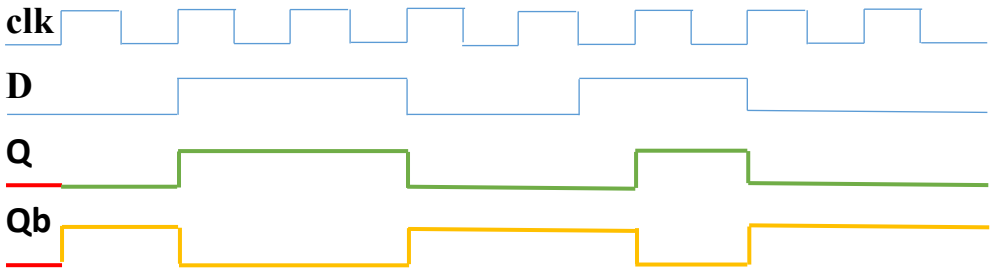
1) **Ecrire une description VHDL selon le principe d'une bascule D latch**

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity d_latch is
   port ( clk  : in  std_logic;
          d : in  std_logic;
          q : out  std_logic;
          qb : out  std_logic);
 end d_latch;
architecture Behavioral of d_latch is
signal t1, t2 : STD_LOGIC;
begin
process (clk)
begin
if clk= '1' then
t1<= d; t2<= not d;
else
t1<= t1; t2<= t2;
end if;
end process;
q<= t1; qb<= t2;
end Behavioral;
```

clk	d	q	qb
0	0	q^-	$\overline{q^-}$
0	1	q^-	$\overline{q^-}$
1	0	0	1
1	1	1	0

2) **Compléter le chronogramme suivant en se basant sur les données de D et CLK**



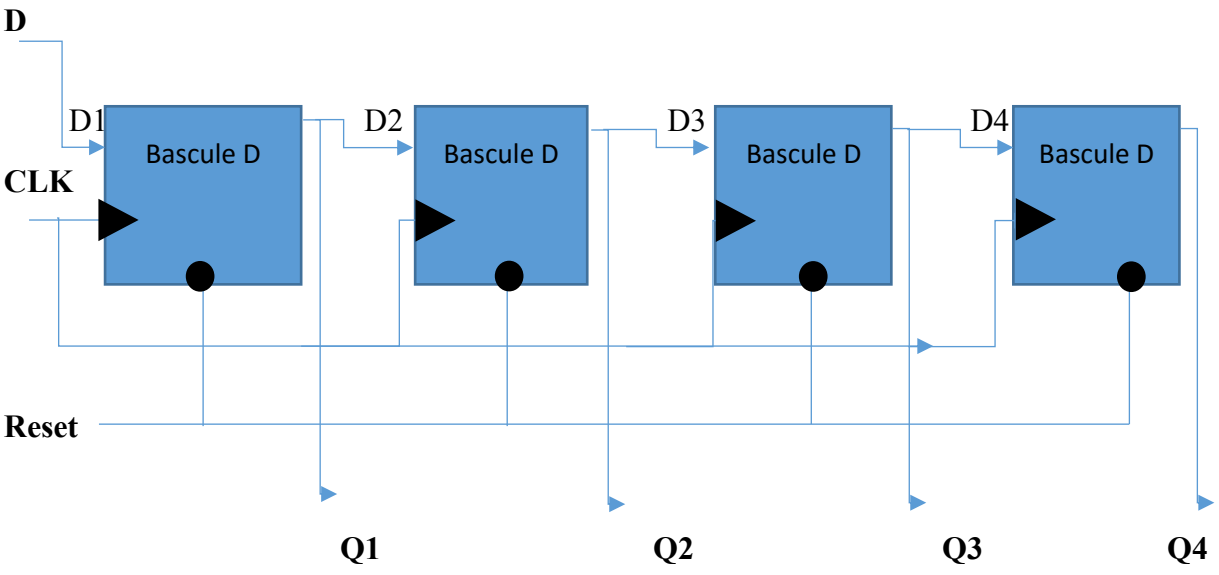
Exercice 2

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
```

```
entity registre_latch is
   port ( clk, reset  : in  std_logic;
          d : in  std_logic_vector (3 downto 0);
          q : out  std_logic_vector (3 downto 0);
          qb : out  std_logic_vector (3 downto 0));
end registre_latch;

architecture Behavioral of registre_latch is
signal E1, E2: std_logic_vector (3 downto 0);
begin
process (d, reset, clk)
begin
     if reset= '1' then E1<="0000"; E2<="1111";
     else if clk= '1' then
          E1<= d;  E2<= not d;
          else E1<= E1;  E2<= E2;
     end if;
     end if;
     end process;
     q<= E1;  qb<= E2;
end Behavioral;
```

Exercice 3




```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
```

```
entity registre_decalage is
   port ( clk, reset :in  std_logic;
        d : in  std_logic;
        q : out  std_logic_vector (3 downto 0));
end registre_decalage;
```

Description simple

```
architecture Description_simple of registre_decalage is
signal S: std_logic_vector (3 downto 0);
begin
process (d, reset, clk)
begin
    if reset= '1' then   S<="0000";
    else if clk'event and clk= '1' then
          S(3) <= d;
          S(2) <=  S(3);     ou    S(2 downto 0) <=  S(3 downto 1);
          S(1) <=  S(2);
          S(0) <=  S(1);
else S <=  S;
    end if;
    end process;
    q<= S;
end Description_simple;
```

Description en boucle

```
architecture Description_boucle of registre_decalage is
signal S: std_logic_vector (3 downto 0);
begin
process (d, reset, clk)
begin
    if reset= '1' then    S<="0000";
    else  if clk'event and clk= '1' then
          S(3) <= d;
          U1: for i in 0 to 2 loop
          S(i) <=  S(i + 1);
          End loop U1;
    end if;
    end if;
end process;
q<= S;
end Description_boucle;
```

Série TD n°4 (Circuits séquentiels)

Exercice 1

Réaliser un compteur de 3 bits dans les cas suivants :

1) **Compteur simple sans remise à zéro.**
2) **Compteur avec une Reset asynchrone.**

Exercice 2

Réaliser un compteur décimal en langage VHDL avec une remise à zéro asynchrone active à l'état bas.

Exercice 3

Réaliser un compteur/décompteur en langage VHDL avec une remise à zéro asynchrone active à l'état haut. Le circuit fonctionnera comme compteur à l'état haut du signal E et comme un décompteur à l'état bas du signal E.

Exercice 4

Réaliser un compteur modulo 6 en langage VHDL (le compteur compte de 0 à 5).

Exercice 5

Réaliser un décompteur modulo 12 en langage VHDL (le décompteur décompte de 11 à 0).

Solution de la série TD n°4 (Circuits séquentiels)

Exercice 1: Compteur de 3 bits

1. Compteur simple sans reset

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity compteur_simple is
   Port ( clk : in  std_logic;
          q : out  std_logic_vector (2 downto 0);
end compteur_simple;

architecture arch_simple of compteur_simple is
signal S: std_logic_vector (2 downto 0);
begin
process (clk)
begin
     if  rising_edge (clk) then      ou   if clk'event and clk= '1' then
          S <= S + 1;   else S<= S;
     end if;
end process;
q<= S;
end arch_simple;
```

2. Compteur avec reset asynchrone

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity compteur_reset is
   port ( clk, reset : in  std_logic;
          q : out  std_logic_vector (2 downto 0);
end compteur_reset;

architecture arch_reset of compteur_reset is
signal S: std_logic_vector (2 downto 0);
begin
process (reset, clk)
begin
```

```
    if reset= '1' then  S<="000";
    else  if clk'event and clk= '1' then
          S <= S + 1; else S<= S;
    end if; end if;
end process;
q<= S;
end arch_reset;
```

Exercice 2: Compteur decimal avec reset asynchrone active à l'état bas.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity compteur_decimal is
   port ( clk, reset : in  std_logic;
        digit : out  std_logic_vector (3 downto 0);
end compteur_decimal;

architecture arch_decimal of compteur_decimal is
signal temp: std_logic_vector (3 downto 0);
begin
process (reset, clk)
begin
    if reset= '0' then  temp<="0000";
    else  if clk'event and clk= '1' then
          if temp ="1001"  then  temp <="0000";
          else temp <= temp + 1
          end if;
    end if;
    end if;
end process;
digit<= temp;
end arch_decimal;
```

Exercice 3: Compteur / décompteur avec reset asynchrone active à l'état haut

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity compteur/decompteur is
```

```
   port ( clk, reset : in  std_logic;
          ud: in std_logic;
         digit : out  std_logic_vector (3 downto 0);
end compteur/decompteur;

architecture arch_compteur/decompteur of compteur/decompteur is
signal S: std_logic_vector (3 downto 0);
begin
process (reset, clk)
begin
     if reset= '1' then  S<="0000";
     else  if clk'event and clk= '1' then
             if ud = '1'then S <= S+1;
                             else S <= S -1;

           end if;
          end if;
end if;
end process;
digit <= S;
end arch_compteur/decompteur;
```

Exercice 4 : pour le compteur modulo 6

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity compteur_modulo6 is
   port ( clk, reset : in  std_logic;
         modul : out  std_logic_vector (2 downto 0);
end compteur_ modulo6;

architecture arch_ modulo6 of compteur_ modulo6 is
signal temp: std_logic_vector (2 downto 0);
begin
process (reset, clk)
begin
     if reset= '1' then  temp<="000";
     else  if clk'event and clk= '1' then
             if temp ="101"  then  temp <="000";
             else temp <= temp + 1
             end if;
```

```
    end if;
    end if;
end process;
modul<= temp;
end arch_ modulo6;
```

Exercice 5 : pour le décompteur modulo 12

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity decompteur_modulo12 is
   port ( clk, reset : in  std_logic;
        modulo : out  std_logic_vector (3 downto 0);
end decompteur_ modulo12;

architecture arch_ modulo12 of decompteur_ modulo12 is
signal temp: std_logic_vector (3 downto 0);
begin
process (reset, clk)
begin
    if reset= '1' then  temp<="1011";
    else  if clk'event and clk= '1' then
          if temp ="0000"  then  temp <="1011";
          else temp <= temp - 1
          end if;
    end if;
    end if;
end process;
modulo<= temp;
end arch_ modulo12;
```

Bibliographies

[1] :https://electrosttissemsilt.files.wordpress.com/2017/04/technologie-et-fabrication-des-circuits-intc3a9grc3a9s-partie-01.pdf

[2] : François Verdier, « Les circuits FPGA Concepts de base, architecture et applications », Université de Cergy-Pontoise Laboratoire ETIS - UMR CNRS 8051.

[3] :http://www.youscribe.com/catalogue/documents/ressources-professionnelles/informatique /cours-sm10-526067

[4] : Fabrice CAIGNET, « Etude des circuits logiques programmables Les FPGA », LAAS – CNRS

[5] : http://hdl.telecom-paristech.fr/

[6] : https://fr.wikipedia.org/wiki/Langage_de_description_de_mat%C3%A9riel

[7] : Jacques Weber, Sébastien Moutault, Maurice Meaudre, "Le langage VHDL : du langage au circuit, du circuit au langage", DUNOD, 2007.

[8] : le langage de description VHDL, T. BLOTIN Lycée Paul-Eluard 93206 SAINT-DENIS

[9] : Remacle Matthieu, Schmitz Thomas, Pierlot Vincent, « Le language VHDL », Microélectronique, 24 février 2016.

[10] : AMINE B. CHOUKRI ADEL C ; "Implémentation sur FPGA des méthodes MPP "P&O" et "floue optimisée par les Algorithmes Génétiques" ; école national polytechnique Alger ; Algérie juin 2006.